

Lido Lend Audit Report

Prepared by Cyfrin

Version 2.0

Lead Auditors

Alix40

Carrotsmuggler

Arnie

Alexzoid (Formal Verification)

October 26, 2026

Contents

1 About Cyfrin **4**

2 Disclaimer **4**

3 Risk Classification **4**

4 Protocol Summary **4**

5 Audit Scope **5**
5.0.1 Part 1 (July 6th - July 28th, 2026) . . . 5
5.0.2 Part 2 (August 12th - August 18th, 2026) . . . 6

6 In-Scope File Checksums **7**

7 Executive Summary **9**
7.1 Part 1 . . . 9
7.2 Part 2 . . . 10
7.3 Centralization Risks . . . 10
7.4 Defensive Improvements . . . 11

8 Findings **16**
8.1 Medium Risk . . . 16
8.1.1 A dust-debt settlement queue root reverts `LidoLendERC4626::fallbackExit` for every lender until it clears . . . 16
8.1.2 Banned trusted routers can keep bypassing collateral activation for fresh borrowers . . . 17
8.1.3 `LidoLend::settle` can settle positions even when idle funds are present . . . 19
8.1.4 `GuardianSignals::getOpinion` reads a stale cached suspect count instead of the live active count . . . 20
8.1.5 `LidoLendERC4626::totalAssets` understates backing when the wrapper is the fee recipient 21
8.1.6 `LidoLendMarketController::_handleSettle` blocks all settlement when a solvent queue root rounds to zero value . . . 22
8.1.7 `AddMarket` can record incorrect addresses due to lack of access control in `LidoLendMarketFactory` . . . 24
8.1.8 `CollateralActivationLib::credit, _drain` let an instant activation supply and equal withdrawal launder a borrower's throttled collateral into active with no delay . . . 25
8.2 Low Risk . . . 28
8.2.1 Permissionless `GuardianSignals::prune` lifts a count-derived ban and bypasses the `ACCESS_RESTORER_ROLE` unban path . . . 28
8.2.2 `CollateralActivationLib::debit` does not decrement market inflow when an activating remainder is demoted to dust . . . 28
8.2.3 `LidoLend::setFeeRecipient` does not accrue interest first so the outgoing recipient's earned fee shares are minted to the incoming recipient . . . 29
8.2.4 `LidoLendERC4626::settle, fallbackExit` emit success events with all-zero amounts when the settle loop makes no progress . . . 30
8.2.5 `RiskStewardDashboard::addMarketSteward, removeMarketSteward` emit `MarketStewardSet` on no-op calls and `revokeInstantActivation` emits nothing . . . 31
8.2.6 `GuardianSignals::syncQuorumBlock` advances the collateral-maturation quorum gate with no event emitted . . . 32
8.2.7 Debt-free `LidoLend::withdrawCollateral` reverts with an opaque panic instead of a descriptive lock error . . . 33
8.2.8 `LidoLendMarketController::positionOf` reports matured collateral as still activating until a hook flushes it . . . 34
8.2.9 `settle` can be called on any position by manipulating available liquidity . . . 34
8.2.10 `LidoLendERC4626::initialize` does not verify the canonical controller . . . 35
8.2.11 `IrmRouter::setMarketIrm` cannot replace a reverting current IRM route . . . 36

8.2.12 LidoLend::settle rounds canceled borrow shares up in the supply-anchored path, deviating from the pool-favoring convention, but the deviation is bounded to dust . . . 37
8.2.13 GuardianSignals::getOpinion can return a stale quorum frontier because nothing forces GuardianSignals::syncQuorumBlock before it is read . . . 37
8.2.14 GuardianSignals::_syncQuorumBlock cannot lower the quorum frontier after quorum is raised, silently keeping the weaker pre-raise assurance level . . . 38
8.2.15 GuardianSignals::_unsuspect can extend the suspect surcharge window instead of shrinking it . . . 38
8.2.16 CollateralActivationLib::credit checks the dust threshold before the instant flag, so small instant deposits are not activated immediately . . . 39
8.2.17 ExitBookBase::fillExitRequest lets a lender's exit fill at a degraded price with no lender-side floor . . . 39
8.2.18 ExitBookBase::fillExitRequest deposits the filler's full assets before sizing the fill, exposing the excess to being consumed by others . . . 40
8.2.19 CollateralActivationLib's sub-threshold demotion is inconsistent: seize skips it, and debit's version lets dust exceed minActivationAmount . . . 41
8.2.20 RiskStewardDashboard::increaseActivationDelays can revert on a legitimate activation-delay increase . . . 42
8.2.21 GuardianSignals::_unsuspect skips suspect snapshot re-computation after pruning an expired report, leaving a stale suspect count that inflates the activation delay . . . 43
8.2.22 Splitting a collateral deposit across fresh addresses under-throttles the early lots . . . 43
8.2.23 LidoLendMarketController::initialize never sets supplyCap, so every market launches uncapped until a separate governance call . . . 45
8.2.24 applyIrmConfig drops the parameter validation the original audit baseline enforced . . . 45
8.2.25 LidoLendMarketFactory does not bind a market id to a single controller, and it accepts caller-supplied initial risk config, so controller provenance must be verified off-chain before activation . . . 46
8.2.26 LidoLendERC4626::_deposit and LidoLendERC4626::_withdraw can reduce incumbent managed backing during an ordinary round trip . . . 46
8.2.27 GuardianSignals::setDelegate lets one delegate satisfy a multi-guardian quorum . . . 47
8.3 Informational . . . 49
8.3.1 ExitBookBase::_priorityFeeRate and the request deadline do not exclude market-pause time from their wall-clock windows . . . 49
8.3.2 MorphoExitBook::placeExitRequest omits the ZeroPricePerShare guard its sibling exit book enforces . . . 49
8.3.3 CollateralActivationLib::_drain emits CollateralDrained unconditionally and carries no drained amount . . . 50
8.3.4 GuardianSignals::setQuorum, prune, unflag flip effective ban status with no Banned or Unbanned event . . . 51
8.3.5 The adaptive-curve rate math is duplicated verbatim between AdaptiveCurveIrm and AdaptiveCurveIrmLib . . . 52
8.3.6 Zero-address validation is applied inconsistently across constructors and initializers . . . 53
8.3.7 The LidoLendMarketController guardian-registry binding is observable neither by a getter nor by an event . . . 55
8.3.8 AdaptiveCurveIrm::_borrowRate reverts when a market's curve config is unset, a failure mode the base IRM interface did not have . . . 56
8.3.9 AdaptiveCurveIrmLib does not detect a market that has migrated away from AdaptiveCurveIrm, returning misleading rates from stale config . . . 56
8.3.10 LinearKinkIrm::setMarketKinkParams can never write optimalUtilization = 0, so a market's config can never be unset . . . 57
8.3.11 GuardianSignals::setQuorum does not resync the quorum-scan frontier . . . 57
8.3.12 CollateralActivationLib::sync anchors the guardian-quorum fallback on activatingStart instead of unlockTime . . . 57
8.3.13 LidoLend::accrueInterest changes the Adaptive rate used by future borrowers while the market is empty . . . 58
8.3.14 LidoLend::setFeeRecipient, setOwner accept the zero address . . . 59

8.3.15 Single-step ownership transfer in `OwnerManagerBase::transferOwnership` and `LidoLend::setOwner` can permanently brick owner-only functions . . . 59
8.3.16 In `LidoLendERC4626::fallbackExit` event `FallbackExit` reports only the settle-leg burn, omitting the liquid-leg shares burned . . . 60
8.3.17 `RiskStewardDashboard::increaseActivationDelays` never checks `newMinActivationDelay` is at most `newMaxActivationDelay`, deferring to a confusing downstream revert . . . 60
8.3.18 `AssetRecoverer` recovery functions validate the recipient but not the token, yielding an obscure revert or a silent no-op . . . 60
8.3.19 Apply mechanical Solidity conventions consistently . . . 60
8.3.20 `LidoLendMarketFactory` omits fail-fast zero-address and code-size checks on the controller-only token inputs and on two constructor role recipients . . . 61
8.3.21 `LidoLendERC4626::settle` and `fallbackExit` transfer seized collateral to a caller-chosen receiver before updating vault accounting, with no reentrancy guard . . . 61
8.4 Gas Optimization . . . 62
8.4.1 Cache storage values read more than once instead of re-reading the slot . . . 62
8.4.2 `CollateralActivationLib::_computeDelay` copies the whole `CollateralActivationConfig` from storage but uses only slot 0 . . . 64
8.4.3 The storage `MarketParams` struct is re-encoded from storage on every core call instead of once per transaction . . . 65
8.4.4 Cache known storage values to prevent identical storage reads . . . 66
8.4.5 Use `calldata` for read-only external parameters . . . 66
8.4.6 Pack reorder-safe state variables into fewer storage slots . . . 67

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited. Unless the scope explicitly notes otherwise, the review is limited to the application layer and does not extend to the correctness of API/SDK/execution environment implementations, compilers, standard and third-party libraries, underlying operating systems and hardware.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Lido Lend is an over-collateralized lending protocol that pairs a Morpho Blue fork core with Lido-specific extensions. The core (`LidoLend`) keeps Morpho's isolated-market supply, borrow, collateral, liquidation, and flash-loan accounting unchanged, and adds owner-gated market creation, a `settle` primitive that cancels a borrower's debt against a lender's supply shares in exchange for collateral, and hooks that route every core action on a controlled market to its `LidoLendMarketController`.

The controller and its periphery add:

- a single-market ERC-4626 vault (`LidoLendERC4626`) that is the market's sole lender and exposes a settle-based `fallbackExit`
- a settlement queue that lets lenders of an illiquid market exit by settling the riskiest borrowers first, ahead of natural repayment
- an exit book where a lender posts an exit request that any filler closes from available liquidity for a Dutch-auction priority fee
- a collateral-activation throttle that delays newly supplied collateral from becoming borrowable
- a quorum-based guardian-signal system that can flag, suspect, and ban accounts per market
- a market factory (`LidoLendMarketFactory`) that deploys and wires each market's controller, vault, and guardian registry ahead of the governance activation motion
- bundler adapters (`EthereumGeneralAdapter1`, `LidoLendVaultAdapter`) that batch native-ETH staking, wstETH wrapping, and vault entry into a single transaction
- oracle adapters pricing wstETH (`WstEthRateOracle`) and USDY (`UsdyStableOracle`) as collateral

Interest is priced by an adaptive-curve model or a linear-kink model selected per market through an IRM router. The core, controller, vault, guardian registry, IRM router, and oracles are upgradeable behind ossifiable proxies

(`OssifiableProxy`); the IRM implementations, exit books, and governance facades are not. The launch market borrows WETH against Lido wstETH collateral on Ethereum mainnet; a USDY-collateral stablecoin market is designed but not yet deployed. No production deployment existed at the time of review.

5 Audit Scope

This report covers two audit phases on the same codebase. The scope for each phase was:

5.0.1 Part 1 (July 6th - July 28th, 2026)

```
lido-lend-markets/src/controllers/LidoLendMarketController.sol
lido-lend-markets/src/LidoLendERC4626.sol
lido-lend-markets/src/GuardianSignals.sol
lido-lend-markets/src/oracle/WstEthRateOracle.sol
lido-lend-markets/src/oracle/UsdyStableOracle.sol
lido-lend-markets/src/libs/CollateralActivationLib.sol
lido-lend-markets/src/libs/SettlementQueueLib.sol
lido-lend-markets/src/libs/ExitBookLib.sol
lido-lend-markets/src/libs/MaxHeapLib.sol
lido-lend-markets/src/libs/Constants.sol
lido-lend-markets/src/types/SettlementTypes.sol
lido-lend-markets/src/types/CollateralActivationTypes.sol
lido-lend-markets/src/types/ExitBookTypes.sol
lido-lend-markets/src/types/GuardianSignalsTypes.sol
lido-lend-markets/src/periphery/ExitBookBase.sol
lido-lend-markets/src/periphery/Erc4626ExitBook.sol
lido-lend-markets/src/periphery/LidoLendOwner.sol
lido-lend-markets/src/periphery/MorphoExitBook.sol
lido-lend-markets/src/periphery/RiskStewardDashboard.sol
lido-lend-markets/src/utils/PausableUntil.sol
lido-lend-markets/src/utils/AssetRecoverer.sol
lido-lend-markets/src/proxy/OssifiableProxy.sol
lido-lend/src/LidoLend.sol
lido-lend/src/libraries/MarketParamsLib.sol
lido-lend/src/libraries/MathLib.sol
lido-lend/src/libraries/SharesMathLib.sol
lido-lend/src/libraries/ConstantsLib.sol
lido-lend/src/libraries/ErrorsLib.sol
lido-lend/src/libraries/UtilsLib.sol
lido-lend/src/libraries/EventsLib.sol
lido-lend/src/libraries/SafeTransferLib.sol
lido-lend/src/libraries/periphery/MorphoStorageLib.sol
lido-lend/src/libraries/periphery/MorphoBalancesLib.sol
lido-lend/src/libraries/periphery/MorphoLib.sol
lido-lend-irm/src/adaptive-curve-irm/AdaptiveCurveIrm.sol
lido-lend-irm/src/adaptive-curve-irm/AdaptiveCurveConfig.sol
lido-lend-irm/src/adaptive-curve-irm/libraries/periphery/AdaptiveCurveIrmLib.sol
lido-lend-irm/src/adaptive-curve-irm/libraries/ExpLib.sol
lido-lend-irm/src/adaptive-curve-irm/libraries/MathLib.sol
lido-lend-irm/src/adaptive-curve-irm/libraries/ErrorsLib.sol
lido-lend-irm/src/adaptive-curve-irm/libraries/UtilsLib.sol
lido-lend-irm/src/common/OwnerManagerBase.sol
lido-lend-irm/src/common/OwnerManagerErrors.sol
lido-lend-irm/src/irm-router/IrmRouter.sol
lido-lend-irm/src/irm-router/libraries/ErrorsLib.sol
lido-lend-irm/src/linear-kink-irm/LinearKinkIrm.sol
lido-lend-irm/src/linear-kink-irm/libraries/ErrorsLib.sol
```

5.0.2 Part 2 (August 12th - August 18th, 2026)

```
lido-lend-markets/src/controllers/LidoLendMarketController.sol
lido-lend-markets/src/LidoLendERC4626.sol
lido-lend-markets/src/GuardianSignals.sol
lido-lend-markets/src/adapters/EthereumGeneralAdapter1.sol
lido-lend-markets/src/adapters/LidoLendVaultAdapter.sol
lido-lend-markets/src/libs/CollateralActivationLib.sol
lido-lend-markets/src/libs/ExitBookLib.sol
lido-lend-markets/src/libs/MaxHeapLib.sol
lido-lend-markets/src/libs/SettlementQueueLib.sol
lido-lend-markets/src/types/ExitBookTypes.sol
lido-lend-markets/src/types/GuardianSignalsTypes.sol
lido-lend-markets/src/types/SettlementTypes.sol
lido-lend-markets/src/periphery/ExitBookBase.sol
lido-lend-markets/src/periphery/Erc4626ExitBook.sol
lido-lend-markets/src/periphery/MorphoExitBook.sol
lido-lend-markets/src/periphery/LidoLendMarketFactory.sol
lido-lend-markets/src/periphery/LidoLendOwner.sol
lido-lend-markets/src/periphery/RiskStewardDashboard.sol
lido-lend-markets/src/utils/AssetRecoverer.sol
lido-lend/src/LidoLend.sol
lido-lend/src/libraries/ErrorsLib.sol
lido-lend-irm/src/adaptive-curve-irm/AdaptiveCurveIrm.sol
lido-lend-irm/src/adaptive-curve-irm/libraries/AdaptiveCurveMathLib.sol
lido-lend-irm/src/adaptive-curve-irm/libraries/periphery/AdaptiveCurveIrmLib.sol
lido-lend-irm/src/irm-config-factory/IrmConfigFactory.sol
lido-lend-irm/src/linear-kink-irm/LinearKinkConfig.sol
lido-lend-irm/src/linear-kink-irm/LinearKinkIrm.sol
```

6 In-Scope File Checksums

These keccak256 digests cover the exact bytes of every in-scope source file at the final audited state, with all fixes applied. A single changed byte changes the digest.

Public repository verification (October 26, 2026). The repositories lido-lend, lido-lend-irm and lido-lend-markets are now published as lend, lend-irm and lend-markets, respectively. Cyfrin independently recomputed all 53 file hashes at the following public commits and confirmed that every digest matches:
https://github.com/lidofinance/lend `4d55f1dd98a0f7712543965c66e03c963a004263`
https://github.com/lidofinance/lend-irm `97af4c39c51f1e023d8025319f34a1658036066c`
https://github.com/lidofinance/lend-markets `23416c43c4038ea3c00a13d6203949c8f88905cc`

Files use src/... in each repository; lend-markets dependencies are under lib/lend/ and lib/lend-irm/. The aggregate retains the original paths: src/... for markets, lib/lido-lend/... and lib/lido-lend-irm/... .

Aggregate root: keccak256 of rows sorted by original path, each digest + two spaces + path, joined by newlines without a trailing newline:
`0x84be32b38acdf912b6ae7fbaabe235398f03d682394d121dcdc41af042b4ecc4`

`lido-lend-markets` (25 files)

File	keccak256
src/GuardianSignals.sol	0x394c7c7420c828717a5d644442ac2ae9ea3e6c37cb3b0afc6a765f5d39da7f28
src/LidoLendERC4626.sol	0xaf2223efeb83069af02774b0b9c64c1eb950fd909b1a7b34345b0a798b8fe8d6
src/adapters/EthereumGeneralAdapter1.sol	0xa21c89addca6293d18be340a4255b721cc098c144bdbc8ff83d5d62f839a43d9
src/adapters/LidoLendVaultAdapter.sol	0x49ab46019a882ad147564534717a03d67976db41a1828b812e7da0f53bb8f5f9
src/controllers/LidoLendMarketController.sol	0x373eaab44d8ebe4711a9c5fee5f5030aca5475c2805d02f1075a1e6e174d8f0f
src/libs/CollateralActivationLib.sol	0xd8c2acee361f6fd8a907825c2332ec80471db9e46925f8f6b3806a2ac6880e75
src/libs/Constants.sol	0x2898cb8a036aef5d7d66501926b6d028cf29bdc47baad01f037955d7380050a0
src/libs/ExitBookLib.sol	0x2ccdcdfdb9b8949b05b85f0c6aa748eb75c48e87456f040ffdc4d365ea8b38f0
src/libs/MaxHeapLib.sol	0x268bbf3e5a8c40aa38933faa4d5bed7f3a00dda0dda936f5eb04dda29e41763e
src/libs/SettlementQueueLib.sol	0x5acbdaf61af72bbc283cb4b2470a24c84f6423a10d207bb85dc45b409b167147
src/oracle/UsdyStableOracle.sol	0x82caeb8b1e50738dc979cd9469a9da63f34d86c15010d4934aefd767495a2825
src/oracle/WstEthRateOracle.sol	0xa7eb3dca147913db68745c9b66c7e28a42a93f0ca19e2390fed69e0d6c451114
src/periphery/Erc4626ExitBook.sol	0xa85f0a11fa3625a81ffa32b64af5c8a964ea23cc7190bf009472333bba643c14
src/periphery/ExitBookBase.sol	0x412160d3b187b34552ab8cd47b17dea4b569e32b6e88c16c255acd44ad937d97
src/periphery/LidoLendMarketFactory.sol	0xe680a0610bd7d43ada1406e1387fe9aa1a741c03835273dbeac6eb90d55104c5
src/periphery/LidoLendOwner.sol	0xcdf03ccaef93ea6b6a6a57e6800a955ba9ca0bfe13918b14238bbf0086f9db72
src/periphery/MorphoExitBook.sol	0xf0da6a947793953ce0cd4135716c60ce81bca88a8aab8c845a2db89f71917e21

File	keccak256
src/periphery/RiskStewardDashboard.sol	0x90d7d44e7e62868693e4c4f4ce4322ae9631a875b4b2f876685fac6fda6d2e9c
src/proxy/OssifiableProxy.sol	0x4cd5b76eac5248fa2e7631eb081094e21afbcdb84fe39d195120d35eaa6bdebe
src/types/CollateralActivationTypes.sol	0xeeeb1486296e0adc4d4844665cee1fc393713d9b6efa5ea801d081bc32603c7b
src/types/ExitBookTypes.sol	0xfd114855a04301d306f62d98096fa8d38730c339180b6c05eb8048c5f93d3d8b
src/types/GuardianSignalsTypes.sol	0xf7cb4606fa6355ecd162a9f1355b4abd3ba19210189a96bfe5c25a62a8dc3f15
src/types/SettlementTypes.sol	0xb11bc798eda3262e576c61ee69abdbb43ca672a4a96a4930eb43031363347849
src/utils/AssetRecoverer.sol	0x5ad79a83e23c26dcbd3c5c64d090d8faf05898cc5eb34d6faaecfa960f075bea
src/utils/PausableUntil.sol	0xc4f4a94353ff81a7351d1d7758ab63cd179ad70e031fa6244a9b9696ce0f0ca0

lido-lend **(12 files)**

File	keccak256
src/LidoLend.sol	0xc9f69ab05c6672a1571b4b91d0ad99b27e87bc64c6296c67a73c41a8071de1df
src/libraries/ConstantsLib.sol	0x34f33fef6867defa982f42d0d543ec23d62a17580ba628dd109ad3a547e58637
src/libraries/ErrorsLib.sol	0x9d9ca277430b17d8781dafa12a0f3f02b0d8401a343eb0ec380d8d9ae34a359d
src/libraries/EventsLib.sol	0x7a8f5e2416ba393f5113a43e051cc5257c1975cb47171babc78d143dd4ffbde8
src/libraries/MarketParamsLib.sol	0x6c8e25952be253b960283e8caecf8fe3138e1b2ab0595e67055d56a3eeabeffc
src/libraries/MathLib.sol	0xa7354cbbcecef7bc0c94b61061c4e5da75515056b8e2db65e826b00d7369744a
src/libraries/SafeTransferLib.sol	0x120117a636a3a46d6ba7e30ba00ece4e6c284fc25d7d990a841f4abaced430b1
src/libraries/SharesMathLib.sol	0xe6d1c0ee97239bcda3ca8429fdbad2486faf4192312ce6c4b322772e038f01a8
src/libraries/UtilsLib.sol	0x9a98b6d9cd11924d5a15906046b7c26816a74c61abf9a3d51240a11a78983276
src/libraries/periphery/MorphoBalancesLib.sol	0x37dd5f303f0d36a211fdf6da9ad98d7f4efeaf6f81183bcf20bf4202d4515f26
src/libraries/periphery/MorphoLib.sol	0x7d59323645f4977e73c388cdb3aa7aeb393fc18251e9249e30ad6ece3b92908a
src/libraries/periphery/MorphoStorageLib.sol	0x5d2b8dbbdf1c7aa09202b8026d548cdd4ef19a60e0f400f709f76e061d1fd4c1

lido-lend-irm **(16 files)**

File	keccak256
src/adaptive-curve-irm/AdaptiveCurveConfig.sol	0x06ee168c37530dfb8ef6cdb16f67b8234d709624f1698f232126307dcdf16aef
src/adaptive-curve-irm/AdaptiveCurveIrm.sol	0xe289c4166783038c6c9304a5f882b38860a04a4f01682f9801e588876b1020af
src/adaptive-curve-irm/libraries/AdaptiveCurveMathLib.sol	0x441ef1a01e6aa1db49b56207779037b8f86c794d804e64eaf31f7f5a2af4a44e
src/adaptive-curve-irm/libraries/ErrorsLib.sol	0xb1c732964472582571c74884e9903e102cf9e531b1429699803e08eeb2b994a9
src/adaptive-curve-irm/libraries/ExpLib.sol	0x32e269f72ad4f1938f8c78e4ecfce741ede5358b08897154d8b5662307a8aa49
src/adaptive-curve-irm/libraries/MathLib.sol	0x0cde8ff301e1544641526b64fc48316cb2c37d6eb5e3b067444d4a1c4aa52aed
src/adaptive-curve-irm/libraries/UtilsLib.sol	0xd4b7d2e137bd50fd5176213b5ae9af6e7dc3fcc305ccd005734bb0158594ba4d
src/adaptive-curve-irm/libraries/periphery/AdaptiveCurveIrmLib.sol	0xf44fcf95334f381705d105bc4c3a2d3d136c4874cc400e7a884c09b10226c5b5
src/common/OwnerManagerBase.sol	0x4bf1527d81d47d4732fb40ab4bbe192d551979df63c43bb8cc47d45ae70115cb
src/common/OwnerManagerErrors.sol	0xd54420aecdcdbd3102ebeb5fb897e8719899f8ac716ffdc0ae4e9b8d5b3b7bf5
src/irm-config-factory/IrmConfigFactory.sol	0x4f643c3d9c111fd54b6e655f695fdbf50284a5d7bb6a64a440c3ae27d8fb81c4
src/irm-router/IrmRouter.sol	0x1f0a8a118827fe8d16434cf20505bdc4898b64a2835371eb426fbdb845d43beb
src/irm-router/libraries/ErrorsLib.sol	0x700ea721dceac328b83ea7ac772d0c8af8fc76bf329be9334e03162cf04a7a4a
src/linear-kink-irm/LinearKinkConfig.sol	0x39f8ec370a84976346fb2ad3b5acb2bc9d9c6c598332a486674a428547a33ae7
src/linear-kink-irm/LinearKinkIrm.sol	0x23385aec7fcb622627671871b2035c05cf49cb9451ab60dfe90edf7c5a262e6e
src/linear-kink-irm/libraries/ErrorsLib.sol	0x7e15cb72d5068b3889b98ec1ad4929548dcbcd10fc0c0da70d747125d07cfbbe

7 Executive Summary

Over the course of 22 days across two audit phases, the Cyfrin team conducted security audits on the Lido Lend smart contracts provided by Lido. In this period, a total of 62 issues were found. The findings consist of 8 Medium & 27 Low severity issues with the remainder being informational and gas optimizations.

7.1 Part 1

Part 1 was the initial review of the full protocol across all three repositories. The six medium-severity findings concentrated on the settlement path and the vault's shared accounting: a dust-debt queue root could revert `LidoLendERC4626::fallbackExit` for every lender until it cleared, `LidoLend::settle` could be driven against a position even while idle liquidity was available, and `LidoLendMarketController::_handleSettle` could block all settlement when a solvent queue root rounded to zero value. Alongside these, `LidoLendERC4626::totalAssets` understated backing when the wrapper was the market's fee recipient, the guardian-signal system read a stale suspect count

when sizing activation surcharges, and a banned trusted router could keep bypassing the collateral-activation window for fresh borrowers. The twenty low-severity findings centered on the guardian and collateral-activation state machines, exit-book pricing and deposit sequencing, and event and rounding hygiene, while the thirteen informational and three gas findings covered validation consistency, missing events, IRM configuration edge cases, and storage-read and struct-packing optimizations.

7.2 Part 2

Part 2 reviewed the follow-up changes, most notably the new `LidoLendMarketFactory` and the bundler adapters, together with fixes to Part 1 findings. The two medium-severity findings were both on the newly introduced surface: `LidoLendMarketFactory` could record market addresses without adequate access control, and an instant-activation supply followed by an equal withdrawal could launder a borrower's throttled collateral into an active state with no delay. The seven low-severity findings addressed controller-to-market binding and factory input validation, the re-introduced `applyIrmConfig` parameter checks, an uninitialized supply cap, guardian-delegate uniqueness and a stale suspect snapshot, and a deposit/withdrawal round-trip that could reduce incumbent backing. The eight informational findings covered the settle-leg reentrancy ordering, the `FallbackExit` event's incomplete accounting, zero-address and single-step-ownership hazards, factory fail-fast checks, and mechanical convention consistency, while the three gas findings continued the storage-read caching and struct-packing optimizations from Part 1.

7.3 Centralization Risks

Protocol governance holds broad, trusted control over every in-scope contract, and users must trust these role holders when engaging with the protocol.

- Governance can replace the logic of every proxied contract at will: the `OssifiableProxy` admin controls `OssifiableProxy::proxy__upgradeTo`, `proxy__upgradeToAndCall`, and `proxy__changeAdmin`, and can permanently lock a proxy via `proxy__ossify`. An upgrade can rewrite any accounting, oracle, or access-control logic and thereby reach all user collateral and debt; no on-chain timelock or multisig requirement is enforced in scope.
- Governance can halt a market: `PAUSER_ROLE` / `RESUMER_ROLE` pause and resume it through `LidoLendMarketController::pauseFor, resume` (indefinitely if desired), and `FREEZER_ROLE` / `UNFREEZER_ROLE` freeze and unfreeze it. Every core action on a paused market reverts, so borrower defenses such as repay and collateral top-up are blocked alongside liquidation and interest accrual; a freeze blocks only new borrows, debt-bearing collateral withdrawals, and collateral maturation.
- Governance can block individual users per market: `ACCESS_LIMITER_ROLE` and `ACCESS_RESTORER_ROLE` ban and unban accounts through `GuardianSignals`, and its `MANAGER_ROLE` adds and removes guardians and sets the ban quorum and suspect window. A ban hard-blocks borrowing for the targeted account and freezes the maturation of its activating collateral; a ban also latches automatically once guardian flags reach quorum.
- Governance sets all market economics: `MARKET_CREATOR_ROLE` on `LidoLendOwner` creates markets; `MANAGER_ROLE` on the market controller sets the supply cap, collateral-activation config, settlement config, and instant-activation grants; market stewards appointed by `STEWARD_ADMIN_ROLE` on `RiskStewardDashboard` can tighten activation delays and locks; and the IRM `owner` / `manager` set each market's interest-rate curve parameters, which are validated on-chain only for well-formedness bounds, not economic soundness.
- Governance controls fees: `FEE_MANAGER_ROLE` sets the per-market protocol fee and the fee recipient through `LidoLendOwner`, and each exit book's `owner` sets its service fee through `ExitBookBase::setServiceFee`, capped at `MAX_SERVICE_FEE` (25%).
- The exit-book `owner` can sweep any asset held by an exit book via `AssetRecoverer::recoverERC20, recoverERC721, recoverERC1155, recoverEther`. The books hold nothing between fills, so this reaches stuck or donated assets rather than custodied user principal.

Beyond the inherent admin powers, the exit-book service fee is read at fill time rather than snapshotted when the lender's exit request was created, so an owner fee change (bounded only by the maximum) can reduce the reward of a filler whose fill transaction is already in flight. The lender's proceeds are unaffected.

7.4 Defensive Improvements

The following are defense-in-depth improvements. Each describes a protection the protocol deliberately omits (in some cases matching upstream Morpho behavior by design) rather than an exploitable bug; adopting them would harden the protocol against upstream and operational failure modes.

- **Oracle adapters omit price-feed staleness and completeness guards** (`lido-lend-markets/src/oracle/UsdyStableOracle.sol:118-124`, `lido-lend-markets/src/oracle/WstEthRateOracle.sol:57-59`). `UsdyStableOracle::price` checks only that the Ondo NAV is non-zero and the loan stablecoin's Chainlink answer is positive, applying no round-freshness or completeness guard, an omission documented as matching Morpho's stock oracle; `WstEthRateOracle::price` returns Lido's internal accounting rate with no market-price leg, a documented stETH/ETH par assumption. Because these adapters are protocol-authored and governance-wired rather than permissionless market-creator choices, add a bounded-staleness check (`updatedAt`, `answeredInRound`, and a per-feed heartbeat) to the stablecoin feed, a sanity bound on the NAV, and a bounded-deviation guard on the wstETH rate so a frozen feed or a stETH market depeg cannot price liquidations at a stale value.
- **Interest-rate parameter setters do not accrue interest before repricing** (`lido-lend-irm/src/adaptive-curve-irm/AdaptiveCurveIrm.sol:76-101`, `lido-lend-irm/src/linear-kink-irm/LinearKinkIrm.sol:52-65`). `AdaptiveCurveIrm::setMarketCurveParams` re-points the market at a new immutable config and `LinearKinkIrm::setMarketKinkParams` overwrites the kink in place, so the next lazy accrual prices the entire window since the market's last update at the new parameters; the specification instructs operators to call `accrueInterest` immediately before changing, and `IrmRouter::setMarketIrm` already does so on-chain. Accrue inside the setters themselves, by giving the IRMs a core reference or routing the change through the router, so the elapsed window is always settled at the rate that was actually in force.
- **Proxy initializers are unauthenticated** (`lido-lend-markets/src/oracle/UsdyStableOracle.sol:100-111`). `UsdyStableOracle::initialize` binds the stablecoin feed and decimal scale behind a one-shot `initializer` with no caller restriction, and the same pattern applies to `LidoLendMarketController`, `LidoLendERC4626`, `GuardianSignals`, `IrmRouter`, and `LidoLend`. The deployer initializes those inside the proxy constructor, but no deploy path exists for `UsdyStableOracle`; if it is deployed and initialized in separate transactions an attacker can front-run the call and bind the oracle to an attacker-controlled feed. Bind initializers to the deployer, or deploy-and-initialize atomically, to close the front-run window.

Summary

Project Name	Lido Lend
Repository	`lido-lend-markets`
Repository	`lido-lend`
Repository	`lido-lend-irm`
Overall Timeline	July 6th - August 18th, 2026
Methods	Manual Review, Formal Verification

Audit Phases

	Part 1	Part 2
lido-lend-markets: Initial	0e3e16af06c6...	d70a17018453...
lido-lend-markets: Fix	bdf9e5047525...	bdf9e5047525...
lido-lend: Initial	ec7171654bdb...	490f9d7b3aba...
lido-lend: Fix	422d589f9d09...	422d589f9d09...
lido-lend-irm: Initial	ce459215fd04...	998de75d9470...
lido-lend-irm: Fix	566d31f7dfdf...	566d31f7dfdf...
Timeline	July 6th - July 28th, 2026	August 12th - August 18th, 2026

These commit hashes identify the audited revisions in the pre-publication repositories and are shown as plain text: the repository history was rewritten before publication, so commit links would not resolve. For the exact `keccak256` of the audited contracts, see Section 6 (In-Scope File Checksums).

Issues Found

Severity	Found	Resolved	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	8	4	4
Low Risk	27	17	10
Informational	21	8	13
Gas Optimizations	6	2	4
Total	62	31	31

Summary of Findings

[M-1] A dust-debt settlement queue root reverts `LidoLendERC4626::fallbackExit` for every lender until it clears	Resolved
[M-2] Banned trusted routers can keep bypassing collateral activation for fresh borrowers	Acknowledged
[M-3] `LidoLend::settle` can settle positions even when idle funds are present	Resolved
[M-4] `GuardianSignals::getOpinion` reads a stale cached suspect count instead of the live active count	Acknowledged
[M-5] `LidoLendERC4626::totalAssets` understates backing when the wrapper is the fee recipient	Resolved
[M-6] `LidoLendMarketController::_handleSettle` blocks all settlement when a solvent queue root rounds to zero value	Acknowledged
[M-7] `AddMarket` can record incorrect addresses due to lack of access control in `LidoLendMarketFactory`	Resolved
[M-8] `CollateralActivationLib::credit, _drain` let an instant activation supply and equal withdrawal launder a borrower's throttled collateral into active with no delay	Acknowledged
[L-01] Permissionless `GuardianSignals::prune` lifts a count-derived ban and bypasses the `ACCESS_RESTORER_ROLE` unban path	Resolved

[L-02] `CollateralActivationLib::debit` does not decrement market inflow when an activating remainder is demoted to dust	Resolved
[L-03] `LidoLend::setFeeRecipient` does not accrue interest first so the outgoing recipient's earned fee shares are minted to the incoming recipient	Acknowledged
[L-04] `LidoLendERC4626::settle, fallbackExit` emit success events with all-zero amounts when the settle loop makes no progress	Resolved
[L-05] `RiskStewardDashboard::addMarketSteward, removeMarketSteward` emit `MarketStewardSet` on no-op calls and `revokeInstantActivation` emits nothing	Resolved
[L-06] `GuardianSignals::syncQuorumBlock` advances the collateral-maturation quorum gate with no event emitted	Resolved
[L-07] Debt-free `LidoLend::withdrawCollateral` reverts with an opaque panic instead of a descriptive lock error	Resolved
[L-08] `LidoLendMarketController::positionOf` reports matured collateral as still activating until a hook flushes it	Acknowledged
[L-09] `settle` can be called on any position by manipulating available liquidity	Acknowledged
[L-10] `LidoLendERC4626::initialize` does not verify the canonical controller	Acknowledged
[L-11] `IrmRouter::setMarketIrm` cannot replace a reverting current IRM route	Acknowledged
[L-12] `LidoLend::settle` rounds canceled borrow shares up in the supply-anchored path, deviating from the pool-favoring convention, but the deviation is bounded to dust	Resolved
[L-13] `GuardianSignals::getOpinion` can return a stale quorum frontier because nothing forces `GuardianSignals::syncQuorumBlock` before it is read	Resolved
[L-14] `GuardianSignals::_syncQuorumBlock` cannot lower the quorum frontier after quorum is raised, silently keeping the weaker pre-raise assurance level	Acknowledged
[L-15] `GuardianSignals::_unsuspect` can extend the suspect surcharge window instead of shrinking it	Resolved
[L-16] `CollateralActivationLib::credit` checks the dust threshold before the instant flag, so small instant deposits are not activated immediately	Resolved
[L-17] `ExitBookBase::fillExitRequest` lets a lender's exit fill at a degraded price with no lender-side floor	Resolved
[L-18] `ExitBookBase::fillExitRequest` deposits the filler's full assets before sizing the fill, exposing the excess to being consumed by others	Resolved
[L-19] `CollateralActivationLib`'s sub-threshold demotion is inconsistent: `seize` skips it, and `debit`'s version lets `dust` exceed `minActivationAmount`	Resolved
[L-20] `RiskStewardDashboard::increaseActivationDelays` can revert on a legitimate activation-delay increase	Resolved
[L-21] `GuardianSignals::_unsuspect` skips suspect snapshot recomputation after pruning an expired report, leaving a stale suspect count that inflates the activation delay	Resolved
[L-22] Splitting a collateral deposit across fresh addresses under-throttles the early lots	Acknowledged
[L-23] `LidoLendMarketController::initialize` never sets `supplyCap`, so every market launches uncapped until a separate governance call	Resolved
[L-24] `applyIrmConfig` drops the parameter validation the original audit baseline enforced	Acknowledged

[L-25] `LidoLendMarketFactory` does not bind a market id to a single controller, and it accepts caller-supplied initial risk config, so controller provenance must be verified off-chain before activation	Acknowledged
[L-26] `LidoLendERC4626::_deposit` and `LidoLendERC4626::_withdraw` can reduce incumbent managed backing during an ordinary round trip	Acknowledged
[L-27] `GuardianSignals::setDelegate` lets one delegate satisfy a multi-guardian quorum	Resolved
[I-01] `ExitBookBase::_priorityFeeRate` and the request deadline do not exclude market-pause time from their wall-clock windows	Acknowledged
[I-02] `MorphoExitBook::placeExitRequest` omits the `ZeroPricePerShare` guard its sibling exit book enforces	Resolved
[I-03] `CollateralActivationLib::_drain` emits `CollateralDrained` unconditionally and carries no drained amount	Resolved
[I-04] `GuardianSignals::setQuorum, prune, unflag` flip effective ban status with no `Banned` or `Unbanned` event	Acknowledged
[I-05] The adaptive-curve rate math is duplicated verbatim between `AdaptiveCurveIrm` and `AdaptiveCurveIrmLib`	Resolved
[I-06] Zero-address validation is applied inconsistently across constructors and initializers	Resolved
[I-07] The `LidoLendMarketController` guardian-registry binding is observable neither by a getter nor by an event	Resolved
[I-08] `AdaptiveCurveIrm::_borrowRate` reverts when a market's curve config is unset, a failure mode the base IRM interface did not have	Acknowledged
[I-09] `AdaptiveCurveIrmLib` does not detect a market that has migrated away from `AdaptiveCurveIrm`, returning misleading rates from stale config	Acknowledged
[I-10] `LinearKinkIrm::setMarketKinkParams` can never write `optimalUtilization = 0`, so a market's config can never be unset	Acknowledged
[I-11] `GuardianSignals::setQuorum` does not resync the quorum-scan frontier	Acknowledged
[I-12] `CollateralActivationLib::sync` anchors the guardian-quorum fallback on `activatingStart` instead of `unlockTime`	Acknowledged
[I-13] `LidoLend::accrueInterest` changes the Adaptive rate used by future borrowers while the market is empty	Acknowledged
[I-14] `LidoLend::setFeeRecipient, setOwner` accept the zero address	Acknowledged
[I-15] Single-step ownership transfer in `OwnerManagerBase::transferOwnership` and `LidoLend::setOwner` can permanently brick owner-only functions	Acknowledged
[I-16] In `LidoLendERC4626::fallbackExit` event `FallbackExit` reports only the settle-leg burn, omitting the liquid-leg shares burned	Resolved
[I-17] `RiskStewardDashboard::increaseActivationDelays` never checks `newMinActivationDelay` is at most `newMaxActivationDelay`, deferring to a confusing downstream revert	Acknowledged
[I-18] `AssetRecoverer` recovery functions validate the recipient but not the token, yielding an obscure revert or a silent no-op	Acknowledged
[I-19] Apply mechanical Solidity conventions consistently	Partially Resolved
[I-20] `LidoLendMarketFactory` omits fail-fast zero-address and code-size checks on the controller-only token inputs and on two constructor role recipients	Resolved

[I-21] `LidoLendERC4626::settle` and `fallbackExit` transfer seized collateral to a caller-chosen receiver before updating vault accounting, with no reentrancy guard	Acknowledged
[G-1] Cache storage values read more than once instead of re-reading the slot	Resolved
[G-2] `CollateralActivationLib::_computeDelay` copies the whole `CollateralActivationConfig` from storage but uses only slot 0	Resolved
[G-3] The storage `MarketParams` struct is re-encoded from storage on every core call instead of once per transaction	Acknowledged
[G-4] Cache known storage values to prevent identical storage reads	Acknowledged
[G-5] Use `calldata` for read-only external parameters	Acknowledged
[G-6] Pack reorder-safe state variables into fewer storage slots	Acknowledged

8 Findings

8.1 Medium Risk

8.1.1 A dust-debt settlement queue root reverts `LidoLendERC4626::fallbackExit` for every lender until it clears

Description: `LidoLendERC4626::fallbackExit` and `LidoLendERC4626::settle` route their settlement leg through `_settle`, with no `try/catch` around the core call:

```
function _settle(...) internal returns (uint256 cutSupplyShares, uint256 collateralSeized, uint256
↪  debtCanceled) {
    cutSupplyShares = Math.min(supplyShares, budgetSupplyShares);
    if (cutSupplyShares == 0) return (0, 0, 0);
    ...
    (collateralSeized, debtCanceled,, cutSupplyShares) =
        LIDO_LEND.settle(_marketParams, supplyShares, borrowShares, address(this), borrower, receiver);
}
```

Core `settle` reverts when the seized collateral rounds to zero:

```
seizedAssets = borrowAssetsCanceled.mulDivDown(ORACLE_PRICE_SCALE, collateralPrice);
require(seizedAssets != 0, ErrorsLib.SETTLEMENT_SLICE_TOO_SMALL);
```

Any revert here unwinds the whole outer transaction, including `fallbackExit`'s already-completed free-liquidity redeem leg. Because `fallbackExit` always targets the settlement queue root, a reverting root blocks the automatic illiquid-exit path for every lender until the root changes or its debt is repaid. On a market whose oracle price exceeds `ORACLE_PRICE_SCALE` (`1e36`, as with wstETH priced in WETH), an ordinary permissionless `repay` down to a 1- or 2-wei residual leaves a borrower queued as the root with a debt small enough that the forced full close seizes zero collateral, tripping the revert.

Impact: A dust-debt queue root blocks every lender's automatic illiquid exit while it persists. The block is a liveness failure, not a loss of funds: any account can permissionlessly repay the residual debt to clear the root, and ordinary liquid `withdraw`/`redeem` remain available for the covered portion of a position.

Proof of Concept:

```
// SPDX-License-Identifier: GPL-2.0-or-later
pragma solidity 0.8.35;

import {MockPriceOracle, MockERC20} from "../setup/Mocks.sol";
import {LidoLendSetup} from "../setup/LidoLendSetup.sol";
import {Market} from "../setup/Types.sol";
import {MarketActions} from "../setup/MarketActions.sol";

contract PoC_Validate_DustSettlementSlice is LidoLendSetup {
    using MarketActions for Market;

    function _m() internal view returns (Market storage) {
        return protocol.markets[0];
    }

    function test_dustResidualDebtAtRoot_bricksFallbackExit() public {
        MockERC20 loan = new MockERC20("Loan", "LOAN", 18);
        MockERC20 collateral = new MockERC20("Collateral", "COLL", 18);
        _deployProtocol(_defaultConfig(address(loan), address(collateral), address(collateral)));
        Market storage m = _m();

        // Price above ORACLE_PRICE_SCALE (1e36), like wstETH/WETH.
        MockPriceOracle(address(m.oracle)).setPrice(2e36);

        address lender = _actor("lender");
```

```
        _fund(m.loanToken, lender, 100 ether);
        m.vaultDeposit(lender, 100 ether);

        address borrower = _actor("borrower");
        _fund(m.collateralToken, borrower, 100 ether);
        m.supplyCollateral(borrower, 100 ether);
        m.activateCollateral();
        m.borrow(borrower, 15 ether);

        // Ordinary, permissionless repay down to a 1-wei residual: stays queued as the root, not bad
        ↪  debt.
        _fund(m.loanToken, borrower, 15 ether);
        vm.prank(borrower);
        m.loanToken.approve(address(m.core), type(uint256).max);
        vm.prank(borrower);
        m.core.repay(m.params, 15 ether - 1, 0, borrower, "");

        (address root,,,) = m.controller.nextSettlementBorrower();
        assertEq(root, borrower, "the dust borrower is still the queue root");

        uint256 lenderShares = m.vault.balanceOf(lender);
        vm.prank(lender);
        vm.expectRevert(bytes("settlement slice too small"));
        m.vault.fallbackExit(lenderShares, lender, lender, 0);
    }
}
```

Run with `forge test --match-path "test/scenario/ZZ_PoC_Validate_DustSettlementSlice.t.sol" -vv` from the repository root (needs no mainnet fork). Observed output:

```
Ran 1 test for test/scenario/ZZ_PoC_Validate_DustSettlementSlice.t.sol:PoC_Validate_DustSettlementSlice
[PASS] test_dustResidualDebtAtRoot_bricksFallbackExit() (gas: 23379166)
Logs:
  residual borrowShares after ordinary repay: 1000000
Suite result: ok. 1 passed; 0 failed; 0 skipped
```

Recommended Mitigation: Reject a slice that cannot seize at least one collateral unit before calling the core, so `fallbackExit` can preserve its completed liquid leg:

```
+ uint256 price = IOracle(_marketParams.oracle).price();
+ if (Math.mulDiv(cutAssets, ORACLE_PRICE_SCALE, price) == 0) return (0, 0, 0);
+
  (collateralSeized, debtCanceled,, cutSupplyShares) =
      LIDO_LEND.settle(_marketParams, supplyShares, borrowShares, address(this), borrower, receiver);
```

Alternatively, catch only the core's `SETTLEMENT_SLICE_TOO_SMALL` failure rather than a blanket catch, which would hide unrelated accounting or policy errors. Enforcing a minimum nonzero residual debt in `LidoLendMarketController::_handleRepay` provides defense in depth.

Lido: Fixed in commit `bb712a27`.

Cyfrin: Verified. The empty-slice guard makes the `SETTLEMENT_SLICE_TOO_SMALL` revert unreachable, so `fallbackExit` no longer revert.

8.1.2 Banned trusted routers can keep bypassing collateral activation for fresh borrowers

Description: The market is explicitly designed to support trusted routers. `LidoLendMarketController::grantInstantActivation` is documented as a whitelist for trusted caller contracts, such as routers supplying collateral on behalf of users. The architecture spec also states that a trusted router can qualify for instant activation while users themselves remain subject to the normal timer.

That trust boundary is not fully enforced once a router is banned.

When collateral is supplied, `LidoLendMarketController::_handleSupplyCollateral` syncs the guardian state for the borrower, then applies instant activation using only the supply caller's allowlist bit:

```
_activation.sync(isFrozen, borrower, _guardianSignals.getOpinion(id, borrower));
_activation.credit(borrower, hookData.assets, _instantActivationCallers[hookData.caller]);
```

There is no check that `hookData.caller` is banned. There is also no freeze condition in the instant activation decision.

As a result, if a trusted instant activation router is compromised and guardians ban it, the router can no longer borrow directly because `LidoLendMarketController::_handleBorrow` checks the borrow caller, receiver, and borrower:

```
if (_guardianSignals.isBanned(id, hookData.caller)) revert AddressBanned();
if (_guardianSignals.isBanned(id, hookData.receiver)) revert AddressBanned();

GuardiansOpinion memory opinion = _guardianSignals.getOpinion(id, hookData.onBehalf);
if (opinion.banned) revert AddressBanned();
```

However, the same banned router can still supply collateral for fresh attacker controlled borrowers and have that collateral become active immediately. `LidoLend::supplyCollateral` accepts an arbitrary `onBehalf` account and does not require authorization from that account. The router only needs to provide the collateral.

This matters because router support is not incidental. It is an intended part of the market design. The docs say instant activation is mainly granted to looping contracts that help borrowers build leveraged positions, and the test suite includes a leverage router scenario where instant activation is required for the loop to compound.

The same issue also affects freeze handling. `LidoLendMarketController::freeze` blocks borrowing while the market is frozen, but it does not remove instant activation. A trusted instant activation router can prepare fresh positions during the freeze by supplying collateral for attacker controlled accounts. Because instant collateral never enters the activation timer, those positions are already active when `LidoLendMarketController::unfreeze` is called. `LidoLendMarketController::pauseFor` is different because pause blocks the hook entrypoint before collateral can be supplied.

Impact: A compromised trusted router can continue creating immediately borrowable collateral positions for fresh accounts even after guardians ban the router.

The attacker can route collateral into a new unbanned borrower, skip the activation delay, then have that borrower call `LidoLend::borrow` directly. The borrow succeeds as long as the borrower, receiver, and caller are not banned and the position satisfies LLTV.

This does not create an undercollateralized borrow by itself. The lender risk comes from removing the review window that collateral activation is meant to provide for stolen collateral, abnormal inflows, sanctions response, compromised accounts, or fast collateral risk events.

During a freeze, the attacker cannot borrow immediately. However, the router can still pre-position already active collateral for fresh accounts. Once the market is unfrozen, those accounts can borrow without waiting through the activation period that ordinary collateral would have faced.

The issue is best described as a containment bypass. Guardian ban stops the compromised router from borrowing, but it does not stop the router from exporting its instant activation privilege to unlimited fresh accounts.

Proof of Concept:

1. Governance grants instant activation to router `R` through `LidoLendMarketController::grantInstantActivation`.
2. Router `R` is compromised.
3. Guardians ban `R` through `GuardianSignals::ban`.
4. `R` can no longer call `LidoLend::borrow` directly, because `LidoLendMarketController::_handleBorrow` checks `hookData.caller` with `GuardianSignals::isBanned`.

5. The instant activation mapping is not changed by the ban. `LidoLendMarketController::hasInstantActivation` still returns true for `R`.
6. The compromised router calls `LidoLend::supplyCollateral` with `onBehalf` set to a fresh attacker controlled account `A`.
7. `LidoLend::supplyCollateral` does not require authorization from `A`. It records collateral for `A` and transfers the collateral from `R`.
8. `LidoLendMarketController::_handleSupplyCollateral` calls `CollateralActivationLib::credit` with `instant` set from `_instantActivationCallers[R]`.
9. Since `R` is still allowlisted, `CollateralActivationLib::credit` treats the supplied collateral as active immediately. It does not enter `dust` or `activating`.
10. Account `A` calls `LidoLend::borrow` directly with `caller`, `onBehalf`, and `receiver` all set to `A`.
11. The borrow ban checks pass because `A` is not banned.
12. The borrow uses the collateral that `R` supplied as active collateral, even though `R` was already banned.
13. The attacker repeats the same flow with fresh accounts.

Recommended Mitigation: `LidoLendMarketController::_handleSupplyCollateral` should only apply instant activation when both the borrower and the supply caller are allowed to use the privilege.

The collateral supply does not need to revert. It can remain risk reducing. The safer behavior is to remove the instant flag for that deposit and send it through the normal activation path.

A direct fix is:

```
GuardiansOpinion memory opinion = _guardianSignals.getOpinion(id, borrower);

_activation.sync(isFrozen, borrower, opinion);

bool instant = _instantActivationCallers[hookData.caller]
    && !isFrozen
    && !opinion.banned
    && !_guardianSignals.isBanned(id, hookData.caller);

_activation.credit(borrower, hookData.assets, instant);
```

This preserves trusted router UX in the normal case, while ensuring a banned router cannot keep bypassing the activation window for fresh borrowers.

Operationally, incident response should still revoke the router through `LidoLendMarketController::revokeInstantActivation`, but the safety of guardian ban should not depend on a second manual action being executed in the same incident window.

Lido: Acknowledged.

Conditional on keeping the feature disabled. The risk is acknowledged, so no trusted instant activators will be enabled without additional constraints and further audits.

8.1.3 `LidoLend::settle` can settle positions even when idle funds are present

Description: The `settle` functionality in LidoLend is meant to kick-in if not enough idle funds are present, and thus user-positions need to be wound down to free up withdrawers. However, the implementation isnt correct, and allows users to `settle` positions even when funds are actually present.

The settlement check is done in the following snippet:

```
require(borrowAssetsCanceled <= _settleableSupply(id, onBehalf),
↪  ErrorsLib.SETTLEMENT_EXCEEDS_SETTLEABLE_SUPPLY);
```

`_settleableSupply` measures the gap between the withdrawer's **TOTAL** position and the available liquidity.

```
uint256 onBehalfSupplyAssets = position[id][onBehalf].supplyShares
        .toAssetsDown(market[id].totalSupplyAssets, market[id].totalSupplyShares);
        return UtilsLib.zeroFloorSub(onBehalfSupplyAssets, availableLiquidity);
```

But there is nothing enforcing the idle liquidity to be settled first, even though the `LidoLend::settle` function takes in a `supplyShares` parameter indicating how much funds to settle, not just the total amount.

So a supplier can have 1000 funds supplied, and the pool can have 100 funds idle. Even in this state, the Supplier can call `settle(100)` and settle for 100 funds, even though the entire amount was already idle. So the idle funds will never be touched, and the settlement will go through.

This is an even bigger issue due to the `LidoLendERC4626` contract, which acts as a concentrated supplier to the market. So `LidoLendERC4626` can be expected to hold a massive supply of tokens, and thus have a very large `position[id][onBehalf].supplyShares`, which means it can always settle positions as long as the total supply amount of the 4626 contract is higher than the free liquidity.

So even if the contract has 1000s of tokens available, users can settle positions of 10s of tokens as long as the `LidoLendERC4626` holds more than 1000s of tokens of supply.

Impact: Positions can be settled even if there is enough liquidity available, since `_settleableSupply` always expects the maximum amount possible to be settled, even though that is not true.

Proof of Concept:

1. The market has 1000 tokens free liquidity
2. Alice supplies 10 tokens to `LidoLendERC4626`, which supplies 1200 tokens into the market.
3. Alice then calls `settle` for the 10 tokens.
4. Since the supplier is `LidoLendERC4626`, whose total position (1210) exceeds the free liquidity (1010), `settle` call goes through.

Recommended Mitigation: In `_settleableSupply`, consider using the passed in settlement amount instead of the full `position[id][onBehalf].supplyShares`

Lido: Fixed in fe636f7. `LidoLendERC4626.settle()` no longer reserves free market liquidity against the request: a borrower with a positive personal settlement-LTV slice can be settled at any liquidity level.

Cyfrin: Fix validated. Settlement is now opt-in, unless the borrower is heap root (worst health borrower).

8.1.4 `GuardianSignals::getOpinion` reads a stale cached suspect count instead of the live active count

Description: `GuardianSignals::getOpinion` reads the cached `signal.suspectCount` rather than recomputing it against the current time.

```
(opinion.strikeCount, opinion.suspectCount, opinion.suspectExpiryTimestamp) = _signalCounters(marketId,
↪  account);
```

`_signalCounters` returns the count exactly as it was at the last `_snapshotSuspects` call, so individual suspect reports that have since aged past `_suspectWindow` still count as long as the snapshot as a whole has not lapsed and at least one report was live when it was taken.

So a borrower can be charged a surcharge for guardian reports that have individually expired, simply because no guardian has since called suspect or unsuspect on that account to force a refresh.

Impact: Borrowers can be charged an inflated, stale suspect-count surcharge on their collateral-activation delay that does not reflect which suspect reports are actually still live.

Proof of Concept:

1. Guardians X, Y, Z suspect account A at t=0, 10, 20 respectively with `suspectWindow=30`; the third call snapshots `suspectCount=3`, `suspectReportedAt=20`.

2. At t=35, X's report (0+30=30 < 35) has individually expired, so a fresh scan would give `suspectCount=2`, but no suspect/unsuspect call has happened since t=20.
3. `getOpinion(marketId, A)` still returns the cached `suspectCount=3`, and since `suspectExpiryTimestamp=50` is still ahead of the relevant lot's `activatingStart`, the full stale count of 3 is applied as surcharge instead of the true live count of 2.

Recommended Mitigation: Recompute the active suspect count on read (or lazily refresh the cache on access) instead of relying solely on the last write-time snapshot, so `getOpinion` always reflects currently-live reports.

Lido: Acknowledged.

Acceptable conservative tradeoff. Stale reports can extend activation delays without increasing borrowing capacity.

8.1.5 `LidoLendERC4626::totalAssets` understates backing when the wrapper is the fee recipient

Description: `LidoLendERC4626::totalAssets` obtains the wrapper's expected supply balance from the expected market totals. Those totals account for pending interest and aggregate fee shares, but the calculation does not add the pending fee shares to the wrapper's individual position.

If the wrapper itself is configured as the core fee recipient, `LidoLendERC4626::totalAssets` understates the wrapper's claim on the market until interest is accrued. `ERC4626` deposits and withdrawals can then be priced against a value that excludes pending assets belonging to the wrapper.

```
// MorphoBalancesLib::expectedMarketBalances
market.totalSupplyShares += feeShares.toUint128();

// LidoLendERC4626::totalAssets
uint256 supplyShares = _lidoLendSupplyShares();
(uint256 totalSupplyAssets, uint256 totalSupplyShares) = _marketSupplyTotals();
return supplyShares.toAssetsDown(totalSupplyAssets, totalSupplyShares);
```

Impact: When users enter or leave while the value is understated, the resulting share conversions can transfer part of the pending fee value between existing and transacting users. The effect requires a positive market fee, pending interest, and `LidoLendERC4626` being the configured fee recipient.

Proof of Concept:

1. Configure `LidoLendERC4626` as the market's `feeRecipient`.
2. Give the wrapper an existing core supply position and existing wrapper shareholders.
3. Configure a positive market fee, open positive debt, and allow interest and fees to accumulate without triggering core accrual.
4. Record the wrapper value returned by `LidoLendERC4626::totalAssets`.
5. Have a new user call `LidoLendERC4626::deposit`.
6. The entry calculation values expected aggregate fee dilution but omits the pending fee shares from the wrapper's individual position, so the user receives more wrapper shares than a fee-aware calculation would mint.
7. `LidoLendERC4626::_deposit` calls `LidoLend::supply`, which reaches `LidoLend::_accrueInterest`.
8. The pending core fee shares are now minted to the wrapper after the new user's wrapper shares already exist.
9. Compare the user's post-accrual claim with a control deposit priced after first accruing the market. The first user captures part of the fees earned before entry.

Recommended mitigation:

Do not configure `LidoLendERC4626` as the market fee recipient. If that configuration is required, make `LidoLendERC4626::totalAssets` include the fee shares that would be minted during pending accrual, using the same rounding and accounting rules as the core.

Lido: Fixed in commit fb2c88d

Cyfrin: Verified, vaults will not be used as core fee recipients. Additionally, `LidoLendERC4626` now tracks only vault-originated core supply shares as managed backing, preventing directly credited shares from causing the reported accrual-timing issue.

8.1.6 `LidoLendMarketController::_handleSettle` blocks all settlement when a solvent queue root rounds to zero value

Description: `LidoLendMarketController::_isBadDebt` compares debt and collateral after each value has already been rounded to a whole loan token unit.

Debt is calculated through `SharesMathLib::toAssetsUp`, which rounds the position's share value up. Collateral is calculated through `LidoLendMarketController::_collateralValue`, which rounds its loan value down.

```
function _isBadDebt(MarketView memory marketView, PositionView memory position)
    internal
    pure
    returns (bool)
{
    if (position.borrowShares == 0) return false;

    return _debtAssets(marketView, position)
        > _collateralValue(marketView, position.collateral);
}
```

For borrow shares `s`, collateral `c`, total borrow assets `A`, total borrow shares `S`, oracle price `P`, and virtual shares `V`, the unrounded values are:

```
debt       = s * (A + 1) / (S + V)
collateral = c * P / ORACLE_PRICE_SCALE
```

A position can satisfy the exact solvency condition:

```
s * (A + 1) * ORACLE_PRICE_SCALE
    <= c * P * (S + V)
```

while the controller still observes:

```
ceil(debt) > floor(collateral)
```

The same rounding also prevents the root from being settled. `SettlementQueueLib::allowedSlice` returns zero as soon as the rounded collateral value is zero, before applying `sliceFloorAssets`, `sliceFloorBps`, `ltvStepBps`, or the full close rule.

```
(uint256 debtAssets, uint256 collateralValue) =
    _positionValues(marketView, position);

if (debtAssets == 0 || collateralValue == 0) return 0;
```

`SettlementQueueLib::settlementSlice` consequently returns an all zero tuple for a borrower that still has nonzero debt shares and collateral.

This leaves the borrower in the queue while making it impossible for `LidoLendERC4626::fallbackExit` to cut the root. The fallback loop receives a zero cut and stops, leaving the lender's illiquid shares unserved.

Targeting another borrower does not bypass the root. Every settlement reaches `LidoLendMarketController::_handleSettle`, which checks the current queue root and reverts with `SettlementBadDebt` when the rounded verdict reports bad debt.

This differs from Issue 33. In Issue 33, the controller returns a positive slice and `LidoLend::settle` later reverts because the collateral seizure rounds to zero. Here no positive slice reaches `LidoLend::settle`. Settlemen-

`tQueueLib::allowedSlice` returns zero first, `LidoLendERC4626::fallbackExit` stops without reverting, and the root's bad debt verdict separately blocks settlement of every other borrower.

Impact: A position worth less than one loan token base unit can block the illiquid settlement path for every lender even though the position owes less than its collateral is worth in exact arithmetic.

`LidoLendERC4626::fallbackExit` still pays the portion covered by available loan token liquidity, but the settlement leg makes no progress and the lender retains the remaining vault shares.

`LidoLendERC4626::settle` cannot be used to reach another eligible borrower because `LidoLendMarketController::_handleSettle` rejects the call based on the dust root.

There is no direct loss of funds. Any funded account can clear the root through `LidoLend::repay` or a full share anchored liquidation. In the demonstrated state, cleanup costs one loan token base unit plus gas.

Proof of Concept: Violated: https://prover.certora.com/output/52567/899ae0ed32a94eae9b6c817791b8683a/?anonymousKey=c8199c26c5e982a4b557942e6966fba3bd74eb43

```
// MC-HL-47: the settle gate calls bad debt only on a root that is underwater in exact arithmetic.
// The predicate rounds the debt up and the collateral value down, and a verdict those two roundings
// alone produce closes the gate over a position that owes less than it holds.
// FORMULA: root = nextSettlementBorrower(), shares[root] * (totalBorrowAssets + 1) * SCALE <=
↪  collateral[root] * price * (totalBorrowShares + VIRTUAL_SHARES) => !hasBadDebt()
rule hlMcBadDebtVerdictIsNotRounding(env e) {

    setupLidoLendMarketControllerValidState(e);
    address root; uint256 sNext; uint256 bsNext; uint256 rNext;
    root, sNext, bsNext, rNext = nextSettlementBorrower(e);

    mathint shares = to_mathint(ghostLlCvlPosBorrowShares[root]);
    mathint collateral = to_mathint(ghostLlCvlPosCollateral[root]);
    require(root != 0 && shares != 0 && collateral != 0,
        "SAFE: only a position with debt and collateral is queued, and an empty queue has no root");

    require(to_mathint(ghostOraclePrice256[ghostLlCvlOracle]) >= ORACLE_PRICE_SCALE_MC_CVL() /
    ↪  10000000000000
            && to_mathint(ghostOraclePrice256[ghostLlCvlOracle]) <= ORACLE_PRICE_SCALE_MC_CVL() * 10,
        "SCOPE: a price a deployed adapter reports, parity scaled by the pair's decimal gap, with a
        ↪  decade of slack either side: UsdyStableOracle about 1.05e24, WstEthRateOracle about
        ↪  1.21e36");

    require(to_mathint(ghostLlCvlTotalBorrowShares) >= to_mathint(ghostLlCvlTotalBorrowAssets)
            && to_mathint(ghostLlCvlTotalBorrowShares)
                <= to_mathint(ghostLlCvlTotalBorrowAssets) * VIRTUAL_SHARES_MC_CVL(),
        "SCOPE: a borrow index inside the band a market runs in. A first borrow mints VIRTUAL_SHARES
        ↪  per asset, which is the upper edge, and only interest moves the ratio down toward the lower
        ↪  one. The residue a zeroFloorSub can leave at the very bottom (no borrow assets against live
        ↪  shares) is a degenerate market, not the one this rule is about");

    require(shares * (to_mathint(ghostLlCvlTotalBorrowAssets) + 1)
            * ORACLE_PRICE_SCALE_MC_CVL()
        <= collateral * to_mathint(ghostOraclePrice256[ghostLlCvlOracle])
            * (to_mathint(ghostLlCvlTotalBorrowShares) + VIRTUAL_SHARES_MC_CVL()),
        "the root owes no more than its collateral is worth, cross-multiplied so neither side rounds;
        ↪  the +1 is the virtual asset toAssetsUp adds");

    assert(!hasBadDebt(e), "the gate reads bad debt off the position, not off its own rounding");
}
```

Violated: https://prover.certora.com/output/52567/69504c3ce81d4e42a74acdb01fddaab2/?anonymousKey=47fd62a7850ccfda84c6a037a3b19848a3a48dc6

```
// MC-HL-48: the borrower the queue names as next to settle is one a lender can cut.
```

```
// The waterfall asks for the root and nothing else, so a root the sizing prices to nothing is where
// it stops, and the illiquid exit behind it stops with it.
// FORMULA: root = nextSettlementBorrower(), root != 0 AND shares[root] != 0 AND collateral[root] != 0
↪  AND sliceFloorAssets != 0 => nextSettlementBorrower().borrowShares != 0
rule hlMcNextToSettleIsAlwaysCuttable(env e) {

    setupLidoLendMarketControllerValidState(e);
    address root; uint256 sNext; uint256 bsNext; uint256 rNext;
    root, sNext, bsNext, rNext = nextSettlementBorrower(e);

    require(root != 0 && ghostLlCvlPosBorrowShares[root] != 0 && ghostLlCvlPosCollateral[root] != 0,
        "SAFE: only a position with debt and collateral is queued, and an empty queue has no root");

    require(ghostMcSetSliceFloorAssets96 != 0,
        "SCOPE: a configured slice floor, as the deployment sets it. With every policy field zero the
        ↪  root is uncuttable by design rather than by rounding");

    require(to_mathint(ghostOraclePrice256[ghostLlCvlOracle]) >= ORACLE_PRICE_SCALE_MC_CVL() /
    ↪  10000000000000
            && to_mathint(ghostOraclePrice256[ghostLlCvlOracle]) <= ORACLE_PRICE_SCALE_MC_CVL() * 10,
        "SCOPE: a price a deployed adapter reports, the same band MC-HL-47 carries");

    require(to_mathint(ghostLlCvlTotalBorrowShares) >= to_mathint(ghostLlCvlTotalBorrowAssets)
            && to_mathint(ghostLlCvlTotalBorrowShares)
                <= to_mathint(ghostLlCvlTotalBorrowAssets) * VIRTUAL_SHARES_MC_CVL(),
        "SCOPE: a borrow index inside the band a market runs in, the same pin MC-HL-47 carries");

    assert(bsNext != 0, "the borrower the queue names as next is one the waterfall can cut");
}
```

Recommended Mitigation: `LidoLendMarketController::_isBadDebt` should compare debt and collateral without independently rounding them to whole loan token units. The comparison should preserve the virtual asset and virtual share terms and use full precision arithmetic that cannot overflow.

Correcting the bad debt verdict alone is not sufficient. `SettlementQueueLib::allowedSlice` would still return zero for the same root, so `LidoLendERC4626::fallbackExit` would continue stopping at it.

The settlement design should also define an explicit cleanup path for a nonzero position whose collateral value is below one loan token base unit. This can be handled by preventing forced operations from leaving such residuals or by providing a bounded dust close that removes the debt and queue entry while explicitly assigning the subunit rounding difference.

Lido: The report identifies a reachable sub-unit consequence of the documented rule that a bad-debt root blocks all settlement.

Unresolved. Deferral requires reliable cleanup. It's estimated as a rare case with a manual and permissionless way to resolve. So the risk can be accepted. The way to resolve is documented.

8.1.7 `AddMarket` can record incorrect addresses due to lack of access control in `LidoLendMarketFactory`

Description: `AddMarket::run` writes the deployment artifact after the broadcast block closes.

```
(marketId, controller, erc4626Wrapper) =
    LidoLendMarketFactory(input.factory).deployMarketComponents(input.market, input.erc4626WrapperSpec);
vm.stopBroadcast();
...
_record(input, marketId, controller, erc4626Wrapper, deployedBlock, context.artifacts)
```

Foundry runs the script body as a simulation, then sends the collected transactions. `controller` and `erc4626Wrapper` therefore hold simulated addresses. `_record` writes the file in that same simulation pass, before a miner includes the real transaction.

`LidoLendMarketFactory::_deployProxy` uses plain `CREATE`. Each address depends on the nonce of the factory, not on the market parameters.

```
proxy = address(new OssifiableProxy(implementation, ADMIN, initData));
```

The deployment functions of the factory have no access control. Any address can increase that nonce. If a miner includes a third-party deployment first, it takes nonce `N`, and the components of the script get nonce `N+k`. The artifact then records an address that holds the contract of the third party.

`marketId` stays correct, because `MarketParamsLib::id` hashes the market parameters. The artifact therefore holds a correct market id with a controller and a vault that the script did not deploy. `GuardianSignalsConfigFile::writeDeployed` records the guardian registry in the same way.

So if the simulation results in address `A`, but the actual execution results in address `B`, due to an attacker deploying using the factory contract before the transaction is mined, the script stores the value `A`, which had params selected by the attacker and not the correct address `B`. If this stored config file is used, it can lead to enabling incorrect markets with incorrect configurations.

Impact: The artifact can record a controller that the script did not deploy, and `LidoLendMarketFactory::validateMarketController` does not check its activation and settlement configuration. Governance can therefore bind an attacker-configured controller in `LidoLend::createMarket` while all on-chain checks pass.

Proof of Concept:

1. Lido runs `AddMarket`. In the simulation the nonce of the factory is 20. The script computes the controller at nonce 20 and the vault at nonce 21.
2. `_record` writes both addresses to the artifact file in that simulation pass.
3. Foundry sends the broadcast transaction to the public mempool.
4. An attacker reads it and sends an earlier transaction to `LidoLendMarketFactory::deployMarketComponents`. It uses the same market parameters, but sets `minActivationDelay` to 0 and `sliceFloorBps` to 10000.
5. The transaction of the attacker takes nonces 20 and 21. The components of Lido get nonces 22 and 23. The artifact now records the components of the attacker.
6. An operator checks the recorded controller with `LidoLendMarketFactory::validateMarketController`. The check passes. Governance calls `LidoLend::createMarket` with that address. The market starts with no activation delay and a full-slice settlement floor.

Recommended Mitigation: Do not write the addresses that the simulation returns. Move the artifact write to a step that runs after the broadcast, and read the addresses from the `MarketComponentsDeployed` and `GuardianSignalsDeployed` events in the mined block.

Or consider suing `CREATE2` in the factory, with a salt from the market id, removes the dependence on the nonce and makes the simulated address correct. Or put access control in the factory functions.

The script may also compare the recorded addresses with the broadcast receipt, and fail if they differ.

Lido: Fixed in commit 04d9d76

Cyfrin: Fix verified. Contract now uses create2 for deployments.

8.1.8 `CollateralActivationLib::credit, _drain` let an instant activation supply and equal withdrawal launder a borrower's throttled collateral into active with no delay

Description: For an instant-activation caller, `CollateralActivationLib::credit` books nothing into the dust or activating buckets and counts the deposit as active at once. Withdrawal and seizure go through `_drain`, which empties `activating` before `dust`, and instant deposits are never tracked separately.

So a borrower sitting on a throttled `activating` lot can convert it to active in one transaction:

1. Instant-supply an equal amount. `credit` books nothing, so raw collateral rises but the throttled bucket is untouched.

2. Withdraw the same amount. `_drain` empties `activating` first, so it consumes the throttled lot instead.

Raw collateral ends unchanged, but the throttled lot is now active with zero elapsed delay. The supply and withdrawal net to zero, so the round trip can be flash-funded and costs only gas.

Impact: The activation throttle and the guardian review window it provides are fully bypassed for a chosen lot. Latent as shipped: no deployed caller holds the instant-activation grant, and the design forbids granting it to a permissionless helper. It is a bug to fix before instant activation is enabled.

Proof of Concept: Foundry test on the shipped default config, with an enabled instant caller (the intended router integration).

```
contract PoC_InstantActivationLaundering is LidoLendSetup {
    address internal router;    // an enabled instant-activation caller
    address internal bob;       // borrower whose throttled lot is laundered
    uint256 internal constant LOT = 100 ether;

    function setUp() public {
        MockERC20 loan = new MockERC20("Loan", "LOAN", 18);
        MockERC20 coll = new MockERC20("Coll", "COLL", 18);
        _deployProtocol(_defaultConfig(address(loan), address(coll), address(coll)));
        router = _actor("router");
        bob = _actor("bob");
        _m().controller.setInstantActivation(router, true);   // documented, intended integration
    }

    function _m() internal view returns (Market storage) { return protocol.markets[0]; }

    function test_instantSupplyThenWithdraw_laundersThrottledLot() public {
        Market storage m = _m();
        uint256 t0 = block.timestamp;

        // 1. Bob supplies 100 normally -> throttled: activating = 100, active = 0.
        _fund(m.collateralToken, bob, LOT);
        vm.startPrank(bob);
        m.collateralToken.approve(address(m.core), LOT);
        m.core.supplyCollateral(m.params, LOT, bob, "");
        vm.stopPrank();
        assertEq(m.controller.positionOf(bob).activating, LOT, "bob lot is throttled");
        assertEq(m.controller.positionOf(bob).active, 0, "no active collateral");

        // 2. Router (instant caller) supplies an equal amount onBehalf of bob. credit books nothing:
        //    raw collateral = 200, activating stays 100, derived active jumps to 100.
        _fund(m.collateralToken, router, LOT);
        vm.startPrank(router);
        m.collateralToken.approve(address(m.core), LOT);
        m.core.supplyCollateral(m.params, LOT, bob, "");
        vm.stopPrank();
        assertEq(m.controller.positionOf(bob).active, LOT, "derived active jumped by the instant
        ↪  amount");

        // 3. Bob withdraws 100. _drain empties the activating bucket first and clears the timer.
        vm.prank(bob);
        m.core.withdrawCollateral(m.params, LOT, bob, bob);
        assertEq(m.controller.positionOf(bob).activating, 0, "throttled bucket drained");
        assertEq(m.controller.positionOf(bob).active, LOT, "original throttled lot is now active");
        assertEq(block.timestamp, t0, "no time elapsed");

        // 4. Bob borrows immediately against the laundered lot.
        vm.prank(bob);
        (uint256 borrowed,) = m.core.borrow(m.params, 1 ether, 0, bob, bob);
        assertEq(borrowed, 1 ether, "borrow against laundered collateral succeeds with zero delay");
```

```
    }
}
```

A helper the borrower controls, registered as the instant caller and authorized on the borrower's position, funds the whole round trip from a fee-free `LidoLend::flashLoan`, so it holds no capital and pays only gas:

```
function onMorphoFlashLoan(uint256 assets, bytes calldata) external {
    collateral.approve(address(core), assets);
    core.supplyCollateral(params, assets, victim, "");        // instant, this contract is
    ↪  instant-registered
    core.withdrawCollateral(params, assets, victim, address(this)); // drains the victim's throttled
    ↪  bucket
    collateral.approve(address(core), assets);                // repay the flash
}
```

Recommended Mitigation: Track instant-supplied collateral in its own counter, drained ahead of the dust-then-activating waterfall, so returning instant collateral can never consume a throttled lot's timer. Do not grant instant activation to any caller until this fix ships.

Lido: The instant route is intended for opening a loop position backed by already-activated collateral, and its rollout has been postponed. When enabled it would require existing activated collateral to seed the loop and would create debt that must be unwound at a cost, so the concern remains valid but is harder to exploit.

Unresolved. Defer only while the feature is disabled. The risk is acknowledged, so no trusted instant activators will be enabled without additional constraints and further audits.

8.2 Low Risk

8.2.1 Permissionless `GuardianSignals::prune` lifts a count-derived ban and bypasses the `ACCESS_RESTORER_ROLE` unban path

Description: `_isBanned` treats an account as banned when either the latched bit is set or the live flag count meets the current quorum: `signal.banned || signal.flaggedBy.length() >= _quorum` (`lido-lend-markets/src/GuardianSignals.sol:300-302`). The latch is only written by `_flag`, and only when the count reaches the quorum in force at flag time (`lido-lend-markets/src/GuardianSignals.sol:379-385`). So an account can be banned purely through the count path, with `signal.banned` still false, once `MANAGER_ROLE` lowers the quorum below a pre-existing flag count via `setQuorum` (`lido-lend-markets/src/GuardianSignals.sol:124-127`).

`prune` is permissionless (`lido-lend-markets/src/GuardianSignals.sol:245`) and removes flags cast by addresses no longer in the guardian set, decrementing exactly that live count. For a latched ban this is harmless: the latch holds `_isBanned` true at any count. For a count-derived ban it is not - dropping the count back below the quorum flips `_isBanned` to false. The sequence is: flags are placed while the quorum is above the resulting count, so nothing latches; `MANAGER_ROLE` lowers the quorum below that count, so the account becomes banned by count alone; a flagger is removed via `removeGuardian`; and then any caller invokes `prune`, dropping the count below the quorum and un-banning the account. The two preconditions are ordinary `MANAGER_ROLE` operations and the final step is open to anyone.

That outcome is the one the function's own NatSpec says is impossible - "The ban is never touched - the ban latch means a permissionless caller can lower the flag count but cannot unban" (`lido-lend-markets/src/GuardianSignals.sol:243-244`) - and it is the outcome the role model reserves for `ACCESS_RESTORER_ROLE`, the sole gate on `unban` (`lido-lend-markets/src/GuardianSignals.sol:276-281`).

Files:

`GuardianSignals::prune, setQuorum`

Impact: An account whose ban rests on the count path can be restored by any caller, bypassing the `ACCESS_RESTORER_ROLE` gate that exists to make unbanning a privileged, risk-increasing action. Once un-banned, the borrower's activating collateral is no longer pinned to a `type(uint256).max` unlock (`lido-lend-markets/src/libs/CollateralActivationLib.sol:368`), so it can mature and back new borrows, and the hard block on borrowing lifts, until current guardians re-flag the account up to quorum. The window is closable by the guardians, and reaching it requires two `MANAGER_ROLE` actions in a particular order, which bounds this to Low. The defect is that a permissionless function can move ban state at all, contrary both to its own documentation and to the role separation built around the ban.

Recommended Mitigation: Make the ban a pure latch, so no permissionless mutation of the flag count can change it: have `_isBanned` return `signal.banned` alone and leave `_flag` as its only writer, which it already is - it sets the latch the moment the count reaches the quorum in force. The quorum then decides only when a new flag latches a ban, and neither a config change nor a `prune` can silently create or destroy one. This also makes the NatSpec quoted above true, and makes `_unflag`'s "Never lifts a latched ban" comment (`lido-lend-markets/src/GuardianSignals.sol:387`) hold unconditionally. If the count term must remain in `_isBanned`, gate `prune` so that it cannot lower an effectively-banned account's count below the quorum, leaving `unban` as the only exit.

Lido: Fixed in commit d1200a4.

Cyfrin: Fix verified. `prune` and `_unflag` latch a live ban before dropping the count, so a permissionless prune cannot lift a count-derived ban. One inherent residual stays: a manager raising `setQuorum` can still unban an unlatched count ban.

8.2.2 `CollateralActivationLib::debit` does not decrement market inflow when an activating remainder is demoted to dust

Description: `debit` drains collateral from the un-matured buckets, reverses the inflow that collateral had added, and then - when the activating remainder is left below `minActivationAmount` with no lock in force

- demotes that remainder into `dust` (`lido-lend-markets/src/libs/CollateralActivationLib.sol:203-224`). The inflow decrement is computed before the demotion runs, from the drained amount alone (`lido-lend-markets/src/libs/CollateralActivationLib.sol:213-222`):

```
uint256 activatingAfter = position.activating;              // post-drain, pre-demote
_applyInflowDecrement($, borrower, activatingBefore - activatingAfter);

if (activatingLockAmount == 0 && activatingAfter != 0 && activatingAfter <
↪  $.config.minActivationAmount) {
    position.dust += activatingAfter.toUint128();
    position.activating = 0;
    ...
}
```

The demoted remainder leaves the `activating` bucket for `dust`, but its contribution to the market-wide `$.inflow` accumulator is never removed. That is an asymmetry with `credit`, which adds the whole `assetsWithDust` to inflow when dust folds up into `activating` (`lido-lend-markets/src/libs/CollateralActivationLib.sol:185-187`) and does not touch inflow at all when a deposit stays dust (`lido-lend-markets/src/libs/CollateralActivationLib.sol:172-176`). Dust is defined as off-throttle collateral - "Free collateral below the activation threshold, withdrawable, never timed" (`lido-lend-markets/src/libs/CollateralActivationLib.sol:108`) - yet after a demotion the accumulator still counts it.

It also breaks the exact-undo property the code claims for itself. `_applyInflowDecrement` states "within a tx no time elapses, so this nets the same-tx addition exactly" (`lido-lend-markets/src/libs/CollateralActivationLib.sol:356`). When a credit and a demoting debit land in the same transaction the netting is not exact: the credit adds the full deposit to inflow, the decrement removes only the drained portion, and the sub-threshold remainder stays counted.

Impact: The market's `$.inflow` accumulator - which `_computeDelay` turns into the activation delay that every subsequent depositor in the market inherits (`lido-lend-markets/src/libs/CollateralActivationLib.sol:380-383`) - is inflated by collateral that is no longer throttled. A borrower can supply just above `minActivationAmount` and withdraw down to a sub-threshold remainder within one transaction, ending with their collateral back as freely-withdrawable dust while the market keeps counting that remainder as recent inflow. Each cycle leaves at most `minActivationAmount` behind, the accumulator is capped at `maxDelayInflow` (`lido-lend-markets/src/libs/CollateralActivationLib.sol:336`), and it decays back down over `maxActivationDelay` (`lido-lend-markets/src/libs/CollateralActivationLib.sol:288-292`), so the effect is bounded and self-healing. Repeated cycles faster than the decay push the market-wide activation delay toward `maxActivationDelay`, lengthening how long every other borrower's supplied collateral takes to become borrowable.

Recommended Mitigation: Compute the inflow decrement after the demotion branch, from the final `position.activating`, so a remainder folded back into dust is removed from the accumulator along with the drained portion. Moving the `_applyInflowDecrement` call below the demote branch and passing `activatingBefore - position.activating` is sufficient, and it preserves the behaviour of every other path: with no demotion `position.activating` still equals `activatingAfter`, so the decrement is unchanged, and the locked-remainder branch - where the demotion is deliberately skipped so a locked balance cannot be turned into freely-withdrawable dust - is likewise unaffected.

Lido: Fixed in commit 0a41992.

Cyfrin: Fix verified. `debit` demotes the remainder to dust before reconciling inflow, so the demoted amount's same-tx inflow is now reversed.

8.2.3 `LidoLend::setFeeRecipient` does not accrue interest first so the outgoing recipient's earned fee shares are minted to the incoming recipient

Description: `_accrueInterest` mints a market's fee cut to whichever address the global `feeRecipient` holds at accrual time: `position[id][feeRecipient].supplyShares += feeShares` (`lido-lend/src/LidoLend.sol:709-718`). Accrual is lazy, so the fee for the whole open window `[market[id].lastUpdate, now]` is computed and

minted on the next touch of that market (lido-lend/src/LidoLend.sol:699-725).

setFeeRecipient overwrites that address without first closing the open window (lido-lend/src/LidoLend.sol:131-137). Its sibling in the same setter family does close it: setFee calls _accrueInterest before writing the new fee, with the comment "Accrue interest using the previous fee set before changing it" (lido-lend/src/LidoLend.sol:121-122). The two functions mutate the two inputs of the same feeShares expression and are treated differently. Because feeRecipient is a single global rather than per-market, one call redirects the open window of every fee-bearing market at once, and each market's window is as long as that market has gone untouched.

Files:

LidoLend::setFeeRecipient

Impact: The fee shares earned over the window preceding the rotation - accrued under the outgoing recipient's tenure - are minted to the incoming recipient on each market's next accrual, and the mint is atomic and irreversible once booked into totalSupplyShares. Both addresses are governance-designated: LidoLend::setFeeRecipient is onlyOwner, and the owner contract gates the call behind FEE_MANAGER_ROLE. So on the deployment as written the misallocation is between two protocol-controlled destinations rather than a loss to an outside party. It becomes a genuine value leak only if the recipient is ever pointed at a contract with external beneficiaries - a fee splitter, a revenue-share vault, a third-party integrator - in which case those beneficiaries are credited fees that accrued before their entitlement began. The amount at stake is the pending fee across every fee-bearing market at the moment of the call. The accrual also emits no Supply event for the mint - the fork's own event library carries the warning that "feeRecipient receives some shares during interest accrual without any supply event emitted" (lido-lend/src/libraries/EventsLib.sol:37) - so the misdirected shares leave no log to reconcile against off-chain.

Recommended Mitigation: Close the interest window before rotating the recipient, mirroring what setFee already does for the other input to the same expression. Because feeRecipient is global, the setter cannot accrue every market itself within a bounded gas budget, so there are two practical forms. Either make the rotation a batch - have governance call accrueInterest for each market with a non-zero fee in the same transaction that calls setFeeRecipient, and state that requirement in the setter's NatSpec - or move the fee recipient into per-market storage, at which point the setter can accrue that one market first exactly as setFee does. Note that this function is unchanged forked core, so an in-place fix carries the dependency-pin process cost the repository's fork policy describes, whereas the batch form needs no core change.

Lido: Acknowledged.

Conditional on the rotation procedure. Assumed that the both addresses is controlled by DAO, so the real risk is low. Also, can be countered on operational level.

8.2.4 LidoLendERC4626::settle, fallbackExit emit success events with all-zero amounts when the settle loop makes no progress

Description: Both vault exit paths emit their completion event on the function's only exit path, outside the guard that decides whether anything actually happened.

In settle, the loop over the named borrowers decrements remainingSupplyShares by whatever each borrower's cut consumed. If no borrower in the batch is settle-eligible, nothing is cut, remainingSupplyShares still equals settlingSupplyShares, and sharesBurned therefore evaluates to zero (lido-lend-markets/src/LidoLendERC4626.sol:212-216). The if (sharesBurned != 0) guard correctly skips the burn and the allowance spend (lido-lend-markets/src/LidoLendERC4626.sol:219-222) - and then emit Settled(caller, owner, receiver, 0, 0, 0) fires anyway on line 224. The call succeeds without reverting, having burned nothing, seized no collateral and cancelled no debt.

fallbackExit has the same shape: if the settle loop breaks on its first iteration because the queue root yields no cut (lido-lend-markets/src/LidoLendERC4626.sol:276), settledShares computes to zero, the if (settledShares != 0) guard at line 292 skips the burn, and emit FallbackExit(caller, owner, receiver, 0, 0, 0) fires on line 298.

Impact: The vault's exit log stream reports successful settlements and fallback exits that settled against nobody.

An indexer counting exits from `Settled` or `FallbackExit` over-counts, and a monitor watching for the vault to settle against borrowers fires on calls that cut nothing. The caller of a zero-progress call gets no signal either: the transaction succeeds, the gas is spent, no shares are burned and no collateral is received, and nothing reverts to tell them the exit did not happen - they have to inspect the returned tuple to notice. No on-chain accounting is wrong, since the burn and the allowance spend are correctly skipped, so this is an observability and caller-feedback defect rather than a loss of funds.

Recommended Mitigation: Move each emit inside the guard that already brackets the burn, so the event fires only when the exit made progress:

```
if (sharesBurned != 0) {
    if (caller != owner) _spendAllowance(owner, caller, sharesBurned);
    _burn(owner, sharesBurned);
    emit Settled(caller, owner, receiver, sharesBurned, collateralSeized, debtCanceled);
}
```

and the same for `FallbackExit` inside the `settledShares != 0` block. This preserves the paths that currently behave correctly: `fallbackExit`'s redeem leg can legitimately satisfy the whole exit on its own and return early before the settle loop (`lido-lend-markets/src/LidoLendERC4626.sol:263`), which already emits no `FallbackExit`, and guarding on `settledShares != 0` keeps that unchanged. If a caller-visible signal for the zero-progress case is also wanted, reverting on it is the stronger option, but that changes the semantics of a call that succeeds today and should be weighed against integrators that submit speculative settle batches.

Lido: Fixed in commit 2e5852e.

Cyfrin: Verified to be fixed

8.2.5 `RiskStewardDashboard::addMarketSteward, removeMarketSteward` emit `MarketStewardSet` on no-op calls and `revokeInstantActivation` emits nothing

Description: Two complementary gaps in the same contract's event surface.

`EnumerableSet.AddressSet`'s `add` and `remove` each return a bool saying whether the set actually changed. Both steward mutators discard that return value and emit unconditionally (`lido-lend-markets/src/periphery/RiskStewardDashboard.sol:62-72`):

```
function addMarketSteward(Id marketId, address steward) external onlyRole(STEWARD_ADMIN_ROLE) {
    _stewards[marketId].add(steward);                  // return value dropped
    emit MarketStewardSet(marketId, steward, true);    // fires even if already a steward
}

function removeMarketSteward(Id marketId, address steward) external onlyRole(STEWARD_ADMIN_ROLE) {
    _stewards[marketId].remove(steward);               // return value dropped
    emit MarketStewardSet(marketId, steward, false);   // fires even if never a steward
}
```

So calling `removeMarketSteward` for an address that was never appointed still emits a log asserting the revocation of a privilege that address never held, and re-adding an existing steward re-emits the grant. Neither call guards the zero address, so a revocation event can be emitted for `address(0)` too.

Separately, two of the three risk-steward levers emit a dashboard-level event and the third does not. `increaseActivationDelays` emits `ActivationDelaysIncreased` (`lido-lend-markets/src/periphery/RiskStewardDashboard.sol:98`) and `increaseActivatingLock` emits `ActivatingLockIncreased` (line 114), while `revokeInstantActivation` forwards to the controller and emits nothing of its own (`lido-lend-markets/src/periphery/RiskStewardDashboard.sol:118-120`). The controller does log the effect, but from the controller and with no indication that a risk steward rather than a `MANAGER_ROLE` holder pulled the lever, so the two paths are indistinguishable in the log stream.

Impact: The risk-steward surface is a separately-delegated privilege class, so its log stream is the audit trail governance uses to review what stewards did and who was able to act as one. The no-op emissions put entries in that trail for privileges that were never granted or never held: a monitoring rule counting steward revocations

over-reports, and a governance review reconstructing the steward set for a market from `MarketStewardSet` alone arrives at a history of grants and revocations that does not match the contract's own `marketStewards` view. The missing event leaves the third lever absent from that trail entirely. No on-chain authorization is wrong - `_stewards` is authoritative, and set semantics make the duplicate calls harmless - so this is an observability and audit-trail defect.

Recommended Mitigation: Emit only on an actual set change, and reject the zero address:

```
function addMarketSteward(Id marketId, address steward) external onlyRole(STEWARD_ADMIN_ROLE) {
    if (steward == address(0)) revert ZeroAddress();
    if (_stewards[marketId].add(steward)) emit MarketStewardSet(marketId, steward, true);
}

function removeMarketSteward(Id marketId, address steward) external onlyRole(STEWARD_ADMIN_ROLE) {
    if (_stewards[marketId].remove(steward)) emit MarketStewardSet(marketId, steward, false);
}
```

Emitting conditionally rather than reverting on a no-op keeps both calls idempotent, which is the behaviour the current code appears to intend. Then give the third lever a dashboard-level event so all three are attributable from one log stream, resolving the controller first (which already validates the market) and emitting before the external call:

```
event InstantActivationRevoked(Id indexed marketId, address indexed caller);
...
function revokeInstantActivation(Id marketId, address caller) external onlyMarketSteward(marketId) {
    ILidoLendMarketConfigurator controller = _configurator(marketId);
    emit InstantActivationRevoked(marketId, caller);
    controller.revokeInstantActivation(caller);
}
```

Lido: Fixed in commit dd36e6e.

Cyfrin: Verified.

8.2.6 `GuardianSignals::syncQuorumBlock` advances the collateral-maturation quorum gate with no event emitted

Description: `_syncQuorumBlock` is the sole writer of `_quorumFrontierTimestamp` and emits nothing, on either branch (`lido-lend-markets/src/GuardianSignals.sol:348-359`):

```
uint40 frontierTimestamp = Arrays.sort(timestamps)[n - quorum].toUint40();
if (frontierTimestamp > _quorumFrontierTimestamp) _quorumFrontierTimestamp = frontierTimestamp;
```

`GuardianSignals` emits an event for every other state write it performs - `GuardianAdded`, `GuardianRemoved`, `QuorumSet`, `SuspectWindowSet`, `Heartbeat`, `Flagged`, `Unflagged`, `FlagPruned`, `Banned`, `Unbanned`, `Suspected`, `SuspectCleared` and `SuspectPruned` - so the frontier is the one mutable field in the contract with no log behind it.

The frontier is not bookkeeping. It is the guardian-quorum gate on collateral maturation: `getOpinion` hands it to the controller (`lido-lend-markets/src/GuardianSignals.sol:291`), and `CollateralActivationLib::sync` refuses to mature a borrower's activating lot unless `unlockTime <= opinion.quorumFrontierTimestamp` or the liveness fallback has elapsed (`lido-lend-markets/src/libs/CollateralActivationLib.sol:240-243`). So the single input to a value-critical maturation decision moves with no record in the log, and `syncQuorumBlock` is permissionless (`lido-lend-markets/src/GuardianSignals.sol:171-173`), so any caller can move it at any time.

Files:

`GuardianSignals::syncQuorumBlock`

Impact: No off-chain consumer can recover the on-chain frontier from the log stream. `Heartbeat` events give every guardian's last-scanned timestamp, so an indexer can compute what the frontier would be if `syncQuorumBlock` were called now - but the stored value only moves when someone actually calls it, and that call produces no log, so the two diverge. A keeper or dashboard reconstructing the frontier from heartbeats reports a borrower's lot as

maturable while `sync` still declines to mature it, and a monitor watching for the maturation gate to open has nothing to trigger on. Every system that tracks when supplied collateral becomes borrowable must poll `getQuorumFrontier` on each block instead of reacting to an event. No on-chain value is affected: the in-protocol consumer reads the storage value directly.

Recommended Mitigation: Emit on the advancing branch of `_syncQuorumBlock`, carrying the new frontier and the caller:

```
event QuorumFrontierAdvanced(address indexed caller, uint40 frontierTimestamp);
...
if (frontierTimestamp > _quorumFrontierTimestamp) {
    _quorumFrontierTimestamp = frontierTimestamp;
    emit QuorumFrontierAdvanced(msg.sender, frontierTimestamp);
}
```

Emit only on the advancing branch, and leave the early return at line 351 and the no-op branch silent: neither writes anything, so logging them would reintroduce the spurious-emission problem in the other direction.

Lido: Fixed in commit 977176a.

Cyfrin: Verified.

8.2.7 Debt-free `LidoLend::withdrawCollateral` reverts with an opaque panic instead of a descriptive lock error

Description: `LidoLendMarketController::_handleWithdrawCollateral` computes `activeCollateral` unconditionally, even though it is used only when the position carries debt:

```
uint256 inactive = _activation.debit(borrower, hookData.assets);

position.collateral -= hookData.assets;
uint256 activeCollateral = position.collateral - inactive;    // computed even with zero debt

if (position.borrowShares != 0) {                              // ...but only USED here
    if (_debtAssets(marketView, position) > _getMaxBorrow(marketView, activeCollateral)) {
        revert InsufficientActiveCollateral();
    }
}
```

`inactive` intentionally preserves `activatingLockAmount` while the lock is active. A debt-free full withdrawal drives `position.collateral` to `0` while `inactive` stays positive, so `position.collateral - inactive` underflows to `Panic(0x11)` before the controller can return a descriptive lock error. The activation lock is meant to keep the locked amount unwithdrawable during its configured duration; the defect is the opaque panic, not the withdrawal being blocked. Any positive `activatingLockAmount` combined with a full-withdrawal request reproduces it.

Impact: A debt-free borrower requesting more than the currently unlocked collateral receives `Panic(0x11)` instead of a descriptive lock error. No funds are lost or stuck beyond the intended lock duration; the defect is entirely in the error surface presented to the caller.

Recommended Mitigation: Explicitly reject a request that would leave raw collateral below the inactive buckets, then compute `activeCollateral` only for debt-bearing positions:

```
  position.collateral -= hookData.assets;
- uint256 activeCollateral = position.collateral - inactive;
+ if (inactive > position.collateral) revert ActivationLockActive();

  if (position.borrowShares != 0) {
+     uint256 activeCollateral = position.collateral - inactive;
      MarketView memory marketView = _marketView(id);
```

Moving the subtraction alone (without the explicit check) would let the withdrawal proceed and leave the activation buckets above the actual core collateral balance.

Lido: Fixed in: https://github.com/lidofinance/lido-lend-markets/pull/93/changes/5ae086f03c18b6c56d79b494b3bd6d3667811a4c

Cyfrin: Fix verified; behaviour preserved, guard only fires under an active lock.

8.2.8 `LidoLendMarketController::positionOf` reports matured collateral as still activating until a hook flushes it

Description: Only `CollateralActivationLib::sync` evaluates the maturity condition and clears `position.activating` when it holds:

```
if (isUnlockTimePassed && (hasGuardiansQuorum || maxQuorumDelayElapsed)) {
    position.activating = 0;
    _clearTimer(position);
}
```

`positionOf` skips this. It reads the raw stored buckets and derives `active` directly from them, with no maturity check:

```
(dashboard.activating, dashboard.dust) = _activation.getActivationState(borrower);
dashboard.active = position.collateral - dashboard.activating - dashboard.dust;
```

Because `positionOf` is `view`, it cannot invoke the storage-mutating `sync`, and no read-only equivalent of the maturity projection exists. Once a lot has matured but no sync-triggering hook has touched the borrower, `positionOf` keeps reporting the pre-maturation `activating` amount and an understated `active`, contradicting the field's documented meaning of matured, borrowable collateral.

Impact: If a lot matures but no hook touches the borrower afterward, `positionOf` understates `active` and overstates `activating` until the next hook corrects it. No on-chain accounting or funds are affected.

Recommended Mitigation: Have `positionOf` account for collateral that has already matured, not just what the last hook synced.

Lido: Acknowledged

Cyfrin: Accepted Tradeoff

8.2.9 `settle` can be called on any position by manipulating available liquidity

Description: The `LidoLend` contract introduces a new option to swap supply tokens for collateral directly, provided the market has no liquidity. This is governed by the `_settleableSupply` function.

```
function _settleableSupply(Id id, address onBehalf) internal view returns (uint256) {
        uint256 availableLiquidity = UtilsLib.zeroFloorSub(market[id].totalSupplyAssets,
        ↪  market[id].totalBorrowAssets);
        uint256 onBehalfSupplyAssets = position[id][onBehalf].supplyShares
        .toAssetsDown(market[id].totalSupplyAssets, market[id].totalSupplyShares);
        return UtilsLib.zeroFloorSub(onBehalfSupplyAssets, availableLiquidity);
    }
```

The issue is that this function can get called unexpectedly, even when there is sufficient liquidity in the pool, via flash borrows. Liquidity is measured by the gap between supply and borrow. A user with supplied collateral can borrow out funds until there is no free liquidity, trigger a `settle`, and then repay the loan at no cost.

Impact: `settle` can actually unexpectedly. While on paper the swap is always at the correct oracle price, the issue is that this still allows anyone to mess with other user's positions. If any market has price higher than the oracle price, which is very common leading to MEV scenarios, MEV bots can take out a loan, settle some positions, and claim collateral and then swap them back.

Basically if uniswap pool swaps WETH-USDC at a higher price, MEV bots with mess with positions in lido lend to take advantage of this price discrepancy. This makes user positions extremely unpredictable and volatile.

Proof of Concept: Consider the following sequence of steps:

The market has 100 USDC supplied, and 900 USDC borrowed. You cannot settle Alice's position (100 USDC borrowed by Alice).

Say 1 ETH=1000 usdc, but uniswap is allowing 1 eth = 1020 usdc. Assume Bob already has collateral supplied to the market, due to some other position they own.

1. Bob borrows 100 USDC against their existing position.
2. Now liquid funds = 0, so `settlablesupply` equal Bob's suply limit.
3. Bob calls `settle` on Alice's position. Bob pays off 100 USDC and gets hold of Alice's 0.1 WETH
4. Bob sells 0.1 WETH for 102 USDC.
5. Bob pays off the borrow in step 1, keeping 2 usdc for themselves.

Bob profited 2 USDC. Alice's position is lost.

Recommended Mitigation: Consider adding more stringent criteria regarding `settle` calls, since 0 liquidity availability can be easily spoofed.

Lido: Acknowledged.

Conditional on the intended settlement policy. It's assumed that the settlement is always equal to or worse than direct swap of the loan token to collateral, so on these premises the manipulation is not profitable.

8.2.10 `LidoLendERC4626::initialize` does not verify the canonical controller

Description: The `LidoLendERC4626` vault is connected to a specific `LidoLend` core in its constructor, but its controller is supplied separately during initialization. `LidoLendERC4626::initialize` only checks that the controller reports the expected market ID.

```
controller = ILidoLendMarketController(controller_);
marketId = marketParams_.id();
if (Id.unwrap(controller.marketId()) != Id.unwrap(marketId)) {
    revert ControllerMarketMismatch();
}
```

This does not prove that the supplied controller is the controller registered in the core. A different contract can return the same market ID and pass initialization.

The vault then reads deposit limits, withdrawal limits, and settlement information from the supplied controller, while supply, withdrawal, and settlement operations are executed through its immutable `LIDO_LEND` core. This can make the vault read policy from one controller while the core enforces another.

A similar check is used by `LidoLend::_createMarket`. A controller created for another core can report the correct market ID, be registered, and then reject every hook because `LidoLendMarketController::onlyControlledMarket` expects calls from the other core.

Impact: An incorrectly initialized vault can report deposit or withdrawal limits that do not match what the core will accept. Settlement and fallback exits can also revert if the vault obtains borrower information from a different controller than the one used by the core.

Registering a controller created for another core can make the affected market unusable. The canonical deployment flow uses the correct controller, so this is a deployment validation and liveness issue rather than an accounting bypass or direct loss of funds.

Proof of Concept: Consider the following sequence of steps:

1. Core A creates market `M` and registers Controller A.
2. Controller B is deployed and returns the same market ID `M`, but it is not registered in Core A.
3. Controller B is configured so that `ControllerB::maxDeposit` returns `123`.

4. A vault connected to Core A is initialized with Controller B.
5. Initialization succeeds because Controller B reports the correct market ID.
6. `LidoLendERC4626::maxDeposit` returns `123` from Controller B, while actual deposits still execute through Core A and Controller A.

Recommended Mitigation: During `LidoLendERC4626::initialize`, require the supplied controller to equal `LIDO_LEND.marketController(marketId)`. `LidoLend::_createMarket` should also verify that the controller's immutable core equals `address(this)`.

Lido: Acknowledged; Market launch is two-phase: the controller and vault are deployed and wired first (the vault's initialize binds the controller here), and the market is created in the core only afterwards, via the governance activation motion's createMarket. So at initialize time the market doesn't exist yet — LIDO_LEND.marketController(marketId) returns address(0) — and the recommended check has no registered controller to compare against. The binding is instead validated at deploy time by the market builder.

8.2.11 `IrmRouter::setMarketIrm` cannot replace a reverting current IRM route

Description: `IrmRouter::setMarketIrm` is designed to let a live market migrate from one IRM implementation to another. The function comment explains that the new implementation is probed before the route is stored because a reverting new IRM would brick the market.

```
if (market.lastUpdate != 0) {
    IMorpho(MORPHO).accrueInterest(marketParams);
    market = IMorpho(MORPHO).market(id);

    irm.borrowRate(marketParams, market);
}

marketIrm[id] = irm;
```

This protects against switching into a broken new IRM, but it creates the opposite failure mode. A live market must successfully accrue through the current old route before the router can store the new route.

During `IrmRouter::setMarketIrm`, the call to `LidoLend::accrueInterest` still uses `marketParams.irm`, which is the router itself. The router then forwards `IrmRouter::borrowRate` to the currently stored implementation in `marketIrm[id]`.

```
IIrm irm = marketIrm[marketParams.id()];
require(address(irm) != address(0), ErrorsLib.IRM_NOT_SET);

return irm.borrowRate(marketParams, market);
```

If the old routed IRM starts reverting, becomes misconfigured, or otherwise cannot price the market, `LidoLend::accrueInterest` reverts before `IrmRouter::setMarketIrm` reaches `marketIrm[id] = irm`. The normal migration path therefore cannot be used to switch the market to a working IRM.

Impact: A live market can become stuck on a broken IRM route. Most core market actions accrue interest before continuing, so a reverting current IRM can block normal market interactions. The intended router migration path also fails because it depends on that same old route succeeding first.

Proof of Concept: Consider the following sequence of steps:

1. A market is created with `IrmRouter` as its immutable IRM.
2. `IrmRouter::setMarketIrm` routes the market to IRM A.
3. The market becomes live and accrues normally through IRM A.
4. IRM A later starts reverting for this market.
5. The router owner or manager tries to migrate the market to working IRM B.
6. `IrmRouter::setMarketIrm` calls `LidoLend::accrueInterest` before writing IRM B.

7. `LidoLend::accrueInterest` calls back into `IrmRouter::borrowRate`.
8. `IrmRouter::borrowRate` forwards to the still active IRM A.
9. IRM A reverts.
10. The transaction reverts before `marketIrm[id] = IRM B`.
11. The market remains routed to IRM A and cannot be recovered through `IrmRouter::setMarketIrm`.

Recommended Mitigation: Add an explicit emergency migration path that can replace a failing old route without requiring old-route accrual to succeed. This path should be tightly permissioned and should define how the skipped or failed accrual interval is handled.

Lido: Acknowledged.

8.2.12 `LidoLend::settle` rounds canceled borrow shares up in the supply-anchored path, deviating from the pool-favoring convention, but the deviation is bounded to dust

Description: In the supply-anchored branch of `LidoLend::settle`, `borrowSharesCanceled` is derived from `borrowAssetsCanceled` via `toSharesUp` instead of `toSharesDown`, which is the direction `LidoLend::repay` uses for the same "known assets, derive shares to burn" conversion. This burns marginally more of the targeted borrower's shares than the canceled assets justify, for a difference bounded to at most one raw share unit per call (the standard `mulDivUp`/`mulDivDown` gap).

```
borrowSharesCanceled =
                borrowAssetsCanceled.toSharesUp(market[id].totalBorrowAssets,
                ↪  market[id].totalBorrowShares);
```

Thus, this favors the borrower and increases the debt held by other borrowers marginally, which goes against the conventions generally followed in such applications, where the pool is always favored.

Impact: The borrower's loan is reduced by an amount higher than required, forcing other borrowers to bear that.

Recommended Mitigation: For consistency with `LidoLend::repay` and the rest of the codebase's rounding convention, round `borrowSharesCanceled` down in the supply-anchored branch instead of up.

Lido: Fixed in 7b0a432.

Cyfrin: Fix validated. Supply anchored path removed.

8.2.13 `GuardianSignals::getOpinion` can return a stale quorum frontier because nothing forces `GuardianSignals::syncQuorumBlock` before it is read

Description: `GuardianSignals::getOpinion` returns the stored quorum frontier without refreshing it first.

```
opinion.quorumFrontierTimestamp = _quorumFrontierTimestamp;
```

The frontier only advances when the permissionless `GuardianSignals::syncQuorumBlock` is separately invoked, which it cannot be from inside `getOpinion` since that is a view function. `LidoLendMarketController` never calls `syncQuorumBlock` before reading an opinion, so the value `CollateralActivationLib::sync` uses to compute `hasGuardiansQuorum` can lag behind guardians' real heartbeats, pushing maturation onto the slower `maxGuardiansQuorumDelay` fallback more than necessary.

Impact: Collateral can take longer to mature than it should because the fast guardian-quorum path is judged against a stale frontier, even though guardians have genuinely scanned past the position's unlock time.

Recommended Mitigation: Have the controller trigger `GuardianSignals::syncQuorumBlock` (or a combined state-mutating read) before consulting the opinion in the activation and borrow hooks, or operationally guarantee a keeper calls `syncQuorumBlock` on a fixed cadence so the frontier stays close to current.

Lido: Fixed in e8392f6.

Cyfrin: Fix verified. Updating heartbeat now also updates the quorum frontier.

8.2.14 GuardianSignals::_syncQuorumBlock cannot lower the quorum frontier after quorum is raised, silently keeping the weaker pre-raise assurance level

Description: GuardianSignals::_syncQuorumBlock computes the quorum-th largest guardian heartbeat timestamp and only ever raises _quorumFrontierTimestamp.

```
uint40 frontierTimestamp = Arrays.sort(timestamps)[n - quorum].toUint40();
if (frontierTimestamp > _quorumFrontierTimestamp) _quorumFrontierTimestamp = frontierTimestamp;
```

This one-directional ratchet exists so a guardian-set rotation that adds fresh zero-heartbeat members cannot pull the frontier backwards. However, raising the quorum through GuardianSignals::setQuorum makes the quorum-th largest timestamp computed from the very same heartbeats strictly smaller, since agreement now needs one more guardian, and the ratchet refuses that smaller, more honest value.

Say three guardians last heartbeat at t=100, t=90 and t=80 with quorum = 2: the frontier reaches 90, correctly meaning two guardians scanned past t=90. Governance then raises the quorum to 3 for stronger assurance; the true 3-guardian consensus is only t=80, but GuardianSignals::_syncQuorumBlock computes 80, sees it is not greater than the stored 90, and leaves _quorumFrontierTimestamp at 90. So GuardianSignals::getOpinion keeps reporting that a 3-guardian quorum scanned past t=90, when only two guardians actually have, and CollateralActivationLib::sync treats hasGuardiansQuorum as true up to t=90 on the old, weaker assurance level until real 3-guardian consensus naturally exceeds it.

Impact: Raising the guardian quorum does not immediately tighten the collateral-activation quorum check as governance intends; the weaker pre-raise assurance level silently stays in effect until enough time passes for genuine higher-quorum consensus to catch up.

Proof of Concept:

1. Guardian set = {G1, G2, G3}, quorum = 2; heartbeats land at t=100, t=90 and t=80 respectively.
2. Anyone calls GuardianSignals::syncQuorumBlock; the frontier becomes 90 (the 2nd-largest timestamp).
3. MANAGER_ROLE calls GuardianSignals::setQuorum(3) to require full-committee agreement going forward.
4. GuardianSignals::syncQuorumBlock is called again; the quorum-th (3rd) largest timestamp is 80, which is not greater than the stored 90, so _quorumFrontierTimestamp stays at 90.
5. A borrower whose unlockTime is 90 is judged hasGuardiansQuorum = true even though G3's last scan (t=80) never reached 90, i.e. only the old 2-of-3 bar is actually satisfied.

Recommended Mitigation: Recompute the frontier independent of its previous stored value whenever the quorum changes (e.g. force a fresh, un-ratcheted recomputation inside GuardianSignals::setQuorum), or maintain a per-quorum frontier so a stricter quorum cannot inherit a looser quorum's already-ratcheted value.

Lido: Acknowledged.

8.2.15 GuardianSignals::_unsuspect can extend the suspect surcharge window instead of shrinking it

Description: GuardianSignals::_unsuspect removes the caller's suspect report and then unconditionally re-snapshots the expiry window off the current timestamp.

```
signal.suspectedAt[msg.sender] = 0;
_snapshotSuspects(marketId, account, signal);
```

When other guardians still have live suspect reports on the account, GuardianSignals::_snapshotSuspects sets suspectReportedAt = block.timestamp, pushing suspectExpiryTimestamp (= suspectReportedAt + _suspectWindow) further into the future, even though a retraction should only ever shrink the remaining surcharge window, never extend it.

Impact: Retracting one guardian's suspicion can lengthen, rather than shorten, how long the remaining suspects' collateral-activation surcharge stays in effect for the account.

Proof of Concept:

1. Guardians A, B and C all suspect account X at `t=0` with `_suspectWindow = 10 days`; `suspectReportedAt = 0`, `suspectCount = 3`, expiry = 10 days.
2. At `t=9 days`, X's activating lot is close to falling outside the surcharge window.
3. Guardian A calls `GuardianSignals::unsuspect` intending to reduce suspicion; `suspectedBy` drops to `{B, C}` and `suspectCount` recomputes to 2, but `suspectReportedAt` is rewritten to 9 days, so expiry becomes 19 days.
4. `CollateralActivationLib::_getUnlockTime` keeps applying the 2-suspect surcharge to X's unlock for 10 more days than it would have without A's retraction.

Recommended Mitigation: Only refresh `suspectReportedAt` when a report is added, not when one is removed; on retraction, recompute `suspectCount` without moving the snapshot timestamp forward.

Lido: Fixed in 8329de6.

Cyfrin: Fix verified, report timestamp is now based on latest report time, not current time.

8.2.16 `CollateralActivationLib::credit` checks the dust threshold before the instant flag, so small instant deposits are not activated immediately

Description: `CollateralActivationLib::credit` checks the dust threshold before checking the `instant` flag.

```
if (assetsWithDust < $.config.minActivationAmount) {
    position.dust = assetsWithDust.toUint128();
    return;
}
...
if (instant) {
    emit CollateralActivated(borrower, assetsWithDust, position);
    return;
}
```

A sub-threshold deposit from a trusted instant caller hits the dust branch first and returns before ever reaching the instant branch, so it lands as untimed but inactive dust instead of immediately-active collateral, contrary to the NatSpec's stated behaviour that an instant deposit counts as active right away.

Impact: A trusted caller's small instant deposit is not usable as borrowable collateral immediately, contrary to the documented purpose of the instant path.

Proof of Concept:

1. `minActivationAmount = 10 ether`; the borrower has zero existing dust.
2. An instant-trusted caller credits 5 ether on the borrower's behalf.
3. `assetsWithDust = 5 ether < minActivationAmount`, so the dust branch runs and returns before the instant check is ever reached.
4. The 5 ether sits as dust — withdrawable, but excluded from active/borrowable collateral — even though instant deposits are meant to bypass the activation mechanism entirely.

Recommended Mitigation: Check the `instant` flag before the dust-threshold branch so a trusted deposit is always credited straight to active collateral regardless of its size.

Lido: Fixed in 3df6901.

Cyfrin: Fix validated.

8.2.17 `ExitBookBase::fillExitRequest` lets a lender's exit fill at a degraded price with no lender-side floor

Description: `ExitBookBase::fillExitRequest` settles a request using the `pricePerShare` frozen at placement, but only as a ceiling.

```
uint256 owed = Math.min(Math.mulDiv(filledShares, request.pricePerShare, WAD), assetsOut);
```

If the position's real redeemable value per share falls between request placement and fill, for example from bad debt realized in the backing market, `owed` is capped down to the lower `assetsOut` and the fill still proceeds, since neither `ExitBookLib::add` nor `ExitBookBase::fillExitRequest` let the request owner set a minimum acceptable price or otherwise block a degraded fill.

Impact: A lender's exit request can be filled at a materially worse rate than implied when it was placed, with no built-in floor and no way to react short of manually cancelling before a filler acts.

Proof of Concept:

1. Lender places an exit request when `pricePerShare = 1.05` (WAD-scaled) for 1000 shares, implying roughly 1050 assets.
2. Before any filler acts, bad debt is realized in the backing market, dropping the live redeemable value per share to 0.90.
3. A filler calls `ExitBookBase::fillExitRequest`; `assetsOut` measured from the redeem reflects the degraded 0.90 rate, so `owed = min(1000 * 1.05, assetsOut)` collapses to `assetsOut`.
4. The lender receives close to 900 assets instead of the roughly 1050 implied by their frozen price, with no revert and no chance to have declined the fill.

Recommended Mitigation: Let the owner supply a minimum acceptable price or minimum assets out at request-placement time, and have `ExitBookBase::fillExitRequest` revert the fill (not only the filler's own `minAssetsOut` check) when the lender's floor is not met.

Lido: Fixed in bb3a698.

Cyfrin: Fiix verified. Exits are now asset based, instead of shares based.

8.2.18 `ExitBookBase::fillExitRequest` deposits the filler's full assets before sizing the fill, exposing the excess to being consumed by others

Description: `ExitBookBase::fillExitRequest` pulls the filler's entire `depositAssets` into the source and only computes the fillable shares afterward.

```
if (depositAssets != 0) {
    asset.safeTransferFrom(msg.sender, address(this), depositAssets);
    _depositLiquidity(id, asset, depositAssets, msg.sender);
}

uint256 filledShares = _fillableShares(id, request);
```

`depositAssets` lets a filler top up idle liquidity by exactly the shortfall needed to redeem the request in one shot. But `ExitBookBase::_fillableShares` reads the live `request.sizeRemaining`, which another filler's `fillExitRequest` can shrink via `ExitBookLib::recordFill` between the off-chain estimate and execution. Since `_depositLiquidity` supplies before any of this is known, anything beyond what the smaller fill consumes becomes idle market liquidity under the filler's supply position.

So under certain situations, the filler can be providing more liquidity than necessary to a liquidity-constrained pool, risking getting locked out of more funds than expected.

Impact: A front-running second filler can shrink the fill, stranding most of the first filler's deposit as exposed idle liquidity that others can withdraw first.

Proof of Concept:

1. A request has 1,000 shares remaining backed by only 100 assets of idle liquidity; 900 are out on loans.
2. Filler A previews the fill and calls `ExitBookBase::fillExitRequest(id, 900, minOut)` to redeem all 1,000 shares.

3. Filler B lands first, filling 700 shares from existing idle liquidity, leaving `sizeRemaining = 300`.
4. A's transaction still supplies the full 900, then computes `filledShares` from the 300-share remainder and redeems only that.
5. Roughly 630 of A's 900 remain as freshly idle liquidity, immediately drawable by other withdrawals or pending exit requests ahead of A.

Recommended Mitigation: Compute the fillable shares first, size the deposit to only the shortfall that amount requires, and pull it immediately before the redeem; refund or skip any deposit that turns out unnecessary rather than leaving it in the pool.

Lido: Fixed in bb3a698.

Cyfrin: Fix verified. Depositing funds is now done by the filler themselves.

8.2.19 `CollateralActivationLib`'s sub-threshold demotion is inconsistent: `seize` skips it, and `debit`'s version lets `dust` exceed `minActivationAmount`

Description: `CollateralActivationLib::_drain` can leave `position.activating` below `minActivationAmount`. It only clears the timer when the bucket hits exactly zero.

```
position.activating = uint128(activating - fromActivating);
if (position.activating == 0) _clearTimer(position);
```

`CollateralActivationLib::debit` fixes this up afterwards. It folds a sub-threshold remainder into dust when no lock is in force. `CollateralActivationLib::seize` does not. It calls `_drain` directly and skips the fold. So a partial liquidation or settle leaves a small remainder sitting in the timed bucket.

The fold in `debit` has the opposite problem. It adds to whatever dust survived the drain and never re-checks the threshold:

```
if (activatingLockAmount == 0 && activatingAfter != 0 && activatingAfter <
↪  $.config.minActivationAmount) {
    position.dust += activatingAfter.toUint128();
```

`credit`'s dust branch has the same blind spot. It compares `assets + dust` only, and never reads `position.activating`:

```
if (assetsWithDust < $.config.minActivationAmount) {
    position.dust = assetsWithDust.toUint128();
```

Neither side bounds the sum. `dust` can end up just under `2 * minActivationAmount`. The library's own NatSpec says dust is "below the threshold, untimed, freely withdrawable", so this breaks a stated invariant.

Both halves are needed to get there. `_drain` takes from dust first, so the fold only inflates dust when `activating` is already below the threshold and dust is nonzero. The seize path is what leaves `activating` in that range.

Impact: A seizure remainder stays timed when it should have become withdrawable dust. And `dust` can exceed its own threshold.

Proof of Concept: Reference config: `minActivationAmount = 10 ether`, `activatingLockDuration = 0`, so the demotion branch always runs.

1. A liquidation or settle seizes a borrower down to `activating = 9.99e18`, `dust = 0`, timer still set. `seize` runs no demotion. The remainder stays timed. Even though `activating` amount is lower than threshold, it does not become dust and stays activating.
2. Any address supplies 9.99e18 on that borrower's behalf. `assetsWithDust` is under the threshold (10e18), so it lands in dust. `activating` is untouched. Both `activating` and `dust` are now 9.99e18, and below threshold.
3. The borrower withdraws 1 wei. `_drain` takes it from dust, so `activating` stays at 9.99e18 and the demotion fires. `dust` ends near 19.98e18, above the threshold. So now dust ends up as almost twice the defined limit.

Recommended Mitigation: Run the same demotion in `seize`. It never holds a lock floor, so there is no reason to skip it.

Lido: Fixed in 3df6901.

Cyfrin: Fix verified. Drain now correctly demotes the remaining amount.

8.2.20 `RiskStewardDashboard::increaseActivationDelays` can revert on a legitimate activation-delay increase

Description: `increaseActivationDelays` is the tighten-only lever a per-market steward uses to raise activation delays. It writes back the whole config struct but never touches `maxGuardiansQuorumDelay`:

```
// src/periphery/RiskStewardDashboard.sol - increaseActivationDelays
function increaseActivationDelays(Id marketId, uint40 newMin, uint40 newMax) external
↪   onlyMarketSteward(marketId) {
    CollateralActivationConfig memory cfg = controller.collateralActivationConfig();
    if (newMin < cfg.minActivationDelay || newMax < cfg.maxActivationDelay) revert NotTightened();

    cfg.minActivationDelay = newMin;
    cfg.maxActivationDelay = newMax;                        // maxGuardiansQuorumDelay left untouched
    controller.setCollateralActivationConfig(cfg);
}
```

The controller forwards the struct to `CollateralActivationLib::setConfig`, which enforces a cross-field invariant the lever never accounts for:

```
// src/libs/CollateralActivationLib.sol - setConfig
if (cfg.maxGuardiansQuorumDelay != 0 && cfg.maxGuardiansQuorumDelay <= cfg.maxActivationDelay) {
    revert InvalidMaxGuardiansQuorumDelay();
}
```

A directionally valid tightening that pushes `maxActivationDelay` to or above an existing nonzero `maxGuardiansQuorumDelay` therefore reverts. The default configuration ships with `maxGuardiansQuorumDelay = 0`, so this affects only markets where governance has enabled the documented quorum fallback. Full governance can still update both fields together through `setCollateralActivationConfig`, but the steward lever cannot.

Impact: On a market with a nonzero `maxGuardiansQuorumDelay`, the steward's tighten-only lever reverts `InvalidMaxGuardiansQuorumDelay` on a legitimate delay increase and blocks the intended risk reduction until governance separately updates the quorum delay through the general config path, with no loss of funds and no attacker path.

Proof of Concept: A market is deployed with the quorum fallback enabled (`maxGuardiansQuorumDelay = 3 hours`, above the 2-hour `maxActivationDelay`, so the initial config is valid). A registered steward raises both delays (a legitimate tightening that clears the lever's `NotTightened` check), pushing `maxActivationDelay` to `3 hours`. The lever leaves `maxGuardiansQuorumDelay` at `3 hours`, so `setConfig` now sees `3h <= 3h` and reverts `InvalidMaxGuardiansQuorumDelay`. A control tightening strictly below the quorum delay succeeds, isolating the cross-field gap.

```
// test/scenario/ZZ_PoC_Validate_StewardTightenReverts.t.sol
cfg.activation.maxGuardiansQuorumDelay = 3 hours; // deploy with the quorum fallback enabled
// ...
vm.prank(steward);
vm.expectRevert(CollateralActivationLib.InvalidMaxGuardiansQuorumDelay.selector);
dashboard.increaseActivationDelays(_m().id, 15 minutes, 3 hours); // legitimate tighten -> reverts

// Control: a tightening strictly BELOW the quorum delay succeeds, proving the revert is the
// cross-field gap, not the tightening itself being rejected.
vm.prank(steward);
dashboard.increaseActivationDelays(_m().id, 15 minutes, 2 hours + 59 minutes); // OK
```

Run with `forge test --match-path "test/scenario/ZZ_PoC_Validate_StewardTightenReverts.t.sol" -vv` from the repository root (needs no mainnet fork). Observed output:

```
Ran 1 test for
↪  test/scenario/ZZ_PoC_Validate_StewardTightenReverts.t.sol:PoC_Validate_StewardTightenReverts
[PASS] test_stewardTighten_toQuorumDelay_reverts() (gas: 96555)
Suite result: ok. 1 passed; 0 failed; 0 skipped
```

Recommended Mitigation: The quorum-fallback configuration itself is valid and documented. The bug is the steward lever writing an incomplete config, so fix the lever, not the invariant and not a blanket ban on raising delays:

- Prefer: when `newMax >= cfg.maxGuardiansQuorumDelay` and the fallback is nonzero, also raise `maxGuardiansQuorumDelay` (e.g. to `newMax + 1`) so the stored pair still satisfies `setConfig`, **or**
- Reject in the lever with an explicit error that names the quorum-fallback conflict, instead of forwarding a half-updated struct into `setConfig`.

Do not weaken `InvalidMaxGuardiansQuorumDelay` in `CollateralActivationLib`. That check is correct, and full governance can already update both fields together. Only the tighten-only dashboard path is broken.

Lido: Fixed in: https://github.com/lidofinance/lido-lend-markets/pull/100/changes/ad19beba91f944eed426040ef98898f54d63e307 and https://github.com/lidofinance/lido-lend-markets/pull/100/changes/df83b2d0fcdf6d22729d669c7fe8f49476ae6c4a

Cyfrin: Validated Fix

8.2.21 `GuardianSignals::_unsuspect` skips suspect snapshot re-computation after pruning an expired report, leaving a stale suspect count that inflates the activation delay

Description: `GuardianSignals::_unsuspect` first calls `_pruneExpiredSuspects`, which removes already-expired reports without re-snapshotting, then does `if (!signal.suspectedBy.remove(guardian)) return;`. When the guardian's report was already pruned due to expiry, `remove` returns false and the function returns early without calling `_snapshotSuspects`, leaving `suspectCount` and `suspectReportedAt` stale until the next signal on the account.

Impact: When another guardian's still-active report remains, the stale `suspectCount` over-counts the number of live suspects, so `CollateralActivationLib::_getUnlockTime` applies a larger suspect surcharge than the configured policy and inflates the borrower's activation delay (by up to `delayPerSuspect` per stale report) beyond the number of live reports.

Recommended Mitigation: Call `_snapshotSuspects` in `_unsuspect` even when `suspectedBy.remove(guardian)` returns false, or have `_pruneExpiredSuspects` re-snapshot after removing expired reports.

Lido: Fixed in commit 27de45f

Cyfrin: Verified.

8.2.22 Splitting a collateral deposit across fresh addresses under-throttles the early lots

Description: The activation delay scales with a market-wide inflow accumulator that rises with each deposit in a transaction. `CollateralActivationLib::credit` freezes each lot's delay from the accumulator as it stands at that lot's own deposit, and the timer restart is per-position and extend-only.

`LidoLend::supplyCollateral` is permissionless in `onBehalf`, so one contract can fan out a large deposit `S` to many fresh addresses in a single transaction. Each early lot is priced before the accumulator rises, so it freezes a short delay, while a single deposit of `S` would freeze the maximum. The spec notes that splitting "does not reset the accumulator", but that only prevents a reset. The early lots still keep the low delay they froze before it rose.

Impact: A split deposit's early tranches skip the size-scaled activation delay. The guardian ban still hard-blocks the borrow, so this shortens the review window rather than causing direct loss.

Proof of Concept: Foundry test on the shipped default config. `10,000 ether` as one deposit freezes the full 2-hour delay, but split into 50 lots of `200 ether` across fresh addresses the first lot freezes about 12 minutes (borrowable within the hour) while the last freezes 2 hours.

```
// Fans out N sub-deposits in one transaction, so the per-tx inflow accumulator builds monotonically and
// each lot freezes an increasing delay. Models one attacker splitting a large deposit across fresh
↪  addresses.
contract Splitter {
    function run(ILidoLend core, MarketParams calldata params, IERC20 collateral, uint256 n, uint256
    ↪  lot) external {
        collateral.approve(address(core), n * lot);
        for (uint256 i = 1; i <= n; ++i) core.supplyCollateral(params, lot, addressFor(i), "");
    }
    function addressFor(uint256 i) public pure returns (address) { return address(uint160(0xA0000 +
    ↪  i)); }
}

contract PoC_ActivationSplitUnderThrottle is LidoLendSetup {
    Splitter internal splitter;
    uint256 internal constant N = 50;
    uint256 internal constant LOT = 200 ether;      // S = N * LOT = 10_000 ether == maxDelayInflow

    function setUp() public {
        MockERC20 loan = new MockERC20("Loan", "LOAN", 18);
        MockERC20 coll = new MockERC20("Coll", "COLL", 18);
        _deployProtocol(_defaultConfig(address(loan), address(coll), address(coll)));
        splitter = new Splitter();
    }

    function _m() internal view returns (Market storage) { return protocol.markets[0]; }

    function test_splitAcrossAddresses_underThrottlesEarlyLots() public {
        Market storage m = _m();
        uint256 t0 = block.timestamp;

        // One attacker fans out S = 10_000 ether as 50 lots of 200 across 50 fresh addresses, in one
        ↪  tx.
        _fund(m.collateralToken, address(splitter), N * LOT);
        splitter.run(m.core, m.params, m.collateralToken, N, LOT);

        address first = splitter.addressFor(1);
        address last = splitter.addressFor(N);
        uint256 firstDelay = m.controller.positionOf(first).unlockTime - t0;
        uint256 lastDelay = m.controller.positionOf(last).unlockTime - t0;

        uint256 minDelay = m.config.activation.minActivationDelay;
        uint256 maxDelay = m.config.activation.maxActivationDelay;     // 2 hours
        uint256 maxInflow = m.config.activation.maxDelayInflow;

        // Last lot saw the full aggregate inflow -> froze the max delay (so would a single deposit of
        ↪  S).
        assertEq(lastDelay, maxDelay, "aggregate inflow reached the cap");
        // First lot was priced before the accumulator rose -> froze on the ramp floor for a single LOT.
        assertEq(firstDelay, minDelay + (maxDelay - minDelay) * LOT / maxInflow, "early-lot delay is
        ↪  the ramp floor");
        assertLt(firstDelay, maxDelay / 4, "early lot escaped the size scaling");

        // After one hour the early lot has matured and borrows; the last lot is still throttled.
        vm.warp(t0 + 1 hours);
        vm.prank(first);
        (uint256 borrowed,) = m.core.borrow(m.params, 1 ether, 0, first, first);
        assertEq(borrowed, 1 ether, "early split lot borrows within 1h");
```

```
        vm.prank(last);
        vm.expectRevert(ILidoLendMarketController.InsufficientActiveCollateral.selector);
        m.core.borrow(m.params, 1 ether, 0, last, last);
    }
}
```

Recommended Mitigation: Price each lot against the inflow accumulator observed at the end of the transaction or block, not at its own credit, so a split raises the delay of every tranche it contains.

Lido: Acknowledged

8.2.23 `LidoLendMarketController::initialize` never sets `supplyCap`, so every market launches uncapped until a separate governance call

Description: `LidoLendMarketController::initialize` binds the market id, admin, guardian registry, and configs but never assigns `supplyCap`, so it defaults to zero, which the controller treats as no cap. `maxDeposit` returns `type(uint256).max` and `_handleSupply` returns before enforcing any ceiling.

The only way to set a cap is `LidoLendOwner::setMarketSupplyCap` (MARKET_MANAGER_ROLE) after deployment. The factory's `MarketSpec` has no cap field, no deploy script sets one, and acceptance never asserts a value, so a market goes live uncapped until a separate governance transaction lands.

Impact: Every market launches uncapped. Lenders can supply unlimited amounts into a market whose risk parameters may assume a bounded supply, and a cap set later does not unwind supply already accepted above it. No direct loss, but the exposure ceiling is silently absent at launch.

Recommended Mitigation: Add a `supplyCap` field to the factory `MarketSpec` and set it in `initialize` so a market cannot be created without an explicit cap, and assert each deployed market's cap in acceptance.

Lido: Fixed in commit 6044447.

Cyfrin: Verified. `supplyCap` is now an `initialize` parameter.

8.2.24 `applyIrmConfig` drops the parameter validation the original audit baseline enforced

Description: The IRM lives in the vendored `lib/lido-lend-irm` submodule. It is in scope here because every market accrues interest through `IrmRouter` into it, so a bricked IRM bricks the market.

At the July baseline, `AdaptiveCurveIrm::setMarketCurveParams` always constructed `new AdaptiveCurveConfig(params)`, whose constructor validates every parameter, so the validation was unbypassable. The refactor replaced it with `applyIrmConfig(marketParams, config)`, which reads `getCurveParams()` off an arbitrary address and stores it with no validation and no factory-provenance check. `IConfigurableIrm` even documents that the parameters are not validated and the caller guarantees their validity.

Parameters that `AdaptiveCurveConfig` rejects but `applyIrmConfig` accepts brick accrual. `targetUtilization == 0` divides by zero in the rate math, and a `curveSteepness` below WAD or an oversized `maxRateAtTarget` overflows the interest cast. Core `_accrueInterest` calls the IRM with no try/catch and every market action accrues first, so a reverting IRM takes down the whole market with no recovery short of re-routing to a different IRM. `AdaptiveCurveIrm` re-reads the config on every accrual, so a mutable config that passes at apply time can drift out of band later with no further transaction. `LinearKinkIrm` copies its params by value at apply time and has only the one-shot brick.

Impact: Missing parameter validation lets an invalid config brick the market. Access is authorized and the honest deploy path uses the validating `IrmConfigFactory`, so this is a defense-in-depth regression: a mis-parameterized config the old constructor would have rejected now takes effect silently.

Recommended Mitigation: Re-validate the returned parameters at apply time (reuse `AdaptiveCurveConfig`'s checks), or require the config to have been produced by `IrmConfigFactory`, or snapshot validated params into the IRM's own storage instead of re-reading a mutable external config on every accrual.

Lido: Acknowledged.

Conditional on configuration controls. Assumed that it's covered on operations level.

8.2.25 `LidoLendMarketFactory` does not bind a market id to a single controller, and it accepts caller-supplied initial risk config, so controller provenance must be verified off-chain before activation

Description: A market id is a pure hash of the market parameters, fixed before any controller exists. `_validateMarketSpec` rejects a deployment only when the market already exists on the core, so before governance creates it, `deployMarketController` can be called any number of times for the same `MarketSpec`.

Each call mints a distinct controller proxy claiming the same market id, and each carries the caller's own `activationConfig` and `settlementConfig`. `_validateMarketController` checks role topology and bindings but never reads the config values, so a structurally valid controller with permissive risk parameters passes exactly like the intended one. The core resolves the ambiguity only at `createMarket`, when governance names one controller for the id. Nothing on-chain marks which controller is canonical before that moment.

Impact: A non-canonical controller with unvalidated risk config can be activated. Every candidate is wired with `ADMIN` as sole admin and the caller holds no role, so nothing is seizable. The risk is that the deployment treats the factory event or returned address as proof of correctness and names an attacker-deployed controller. Governance can change the config afterward, so this is a launch-configuration concern.

Recommended Mitigation: Verify controller provenance off-chain before activation: pin the exact controller address per market id and assert its `activationConfig` and `settlementConfig` in the deployment runbook and acceptance, rather than trusting the emitted event. Optionally, have `validateMarketController` assert the configs equal an expected specification.

Lido: Acknowledged.

Acceptable with activation checks. Assumed that it's covered on operations level.

8.2.26 `LidoLendERC4626::_deposit` and `LidoLendERC4626::_withdraw` can reduce incumbent managed backing during an ordinary round trip

Description: `LidoLendERC4626::_deposit` mints outer ERC4626 shares from the supplied asset amount before it knows how many core supply shares the same amount will mint.

```
super._deposit(caller, receiver, assets, shares);

(, uint256 suppliedShares) =
    LIDO_LEND.supply(_marketParams, assets, 0, address(this), "");

_managedSupplyShares += suppliedShares;
```

The outer conversion uses `LidoLendERC4626::totalAssets` and the vault virtual-share offset. The subsequent `LidoLend::supply` conversion rounds the same asset amount down into core supply shares.

`LidoLendERC4626::_withdraw` composes the inverse conversions.

```
(, uint256 withdrawnShares) =
    LIDO_LEND.withdraw(
        _marketParams,
        assets,
        0,
        address(this),
        address(this)
    );

_managedSupplyShares -= withdrawnShares;

super._withdraw(caller, receiver, owner, assets, shares);
```

The outer redemption conversion rounds vault shares down into assets. `LidoLend::withdraw` then converts those assets back into core supply shares through `SharesMathLib::toSharesUp`, rounding in the opposite direction.

At a sufficiently high core asset-per-share index, a depositor can enter through `LidoLendERC4626::deposit`, receive outer vault shares while creating only one managed core supply share, then immediately redeem those vault shares and cause `LidoLend::withdraw` to burn two managed core supply shares.

The extra burned core share comes from backing held for incumbent vault shareholders.

Impact: An ordinary deposit and redemption can reduce `LidoLendERC4626::_managedSupplyShares` even though no settlement, liquidation, fee change, or new interest accrual occurs between the entry and exit.

The entrant loses dust through the round trip. An unchanged incumbent keeps the same outer vault-share balance but has less managed backing behind it, so `LidoLendERC4626::totalAssets` and `LidoLendERC4626::previewRedeem` decrease.

Proof of Concept:

1. Alice deposits through `LidoLendERC4626::deposit` and holds vault shares backed by `LidoLendERC4626::_managedSupplyShares`.
2. A borrower opens a normal core borrow position. Interest accrues through `LidoLend::accrueInterest`, increasing the core asset-per-share index while the vault retains its managed core shares.
3. Bob selects a deposit at a rounding boundary where `LidoLendERC4626::_deposit` mints outer ERC4626 shares, while `LidoLend::supply` returns only one additional core supply share.
4. `LidoLendERC4626::_deposit` adds that one returned share to `LidoLendERC4626::_managedSupplyShares`.
5. Bob immediately redeems the outer shares minted by his deposit. The outer ERC4626 conversion rounds the redemption down to assets, then `LidoLendERC4626::_withdraw` calls `LidoLend::withdraw` for that asset amount.
6. `LidoLend::withdraw` uses `SharesMathLib::toSharesUp`. At the selected boundary it burns two core supply shares even though Bob's entry created only one.
7. `LidoLendERC4626::_managedSupplyShares` finishes below its pre-deposit value. Alice's vault-share balance is unchanged, but `LidoLendERC4626::totalAssets` now values it against less managed backing and her `LidoLendERC4626::previewRedeem` decreases.
8. Bob loses dust through the round trip. The remaining backing loss is imposed on Alice without action from her.

Recommended Mitigation: Due to the limited impact(dust amounts) and potential added complexity in attempting to fix this rounding issue, acknowledging this issue may be the best result to avoid complexity.

Lido: Acknowledged

8.2.27 `GuardianSignals::setDelegate` lets one delegate satisfy a multi-guardian quorum

Description: `GuardianSignals::setDelegate` allows every guardian to choose an arbitrary delegate address. The contract does not prevent several guardians from assigning the same delegate.

```
function setDelegate(address newDelegate) external {
    if (!_guardians.contains(msg.sender)) revert Unauthorized();
    address oldDelegate = _delegates[msg.sender];
    _delegates[msg.sender] = newDelegate;
    emit DelegateSet(msg.sender, oldDelegate, newDelegate);
}
```

`GuardianSignals::_isSenderAuthorized` treats the delegate as equivalent to the guardian for signalling calls.

```
function _isSenderAuthorized(address onBehalf) internal view {
    if (!_guardians.contains(onBehalf)) return false;
```

```
    return msg.sender == onBehalf || msg.sender == _delegates[onBehalf];
}
```

`GuardianSignals::flag`, `GuardianSignals::suspect`, and `GuardianSignals::heartbeat` receive an `onBehalf` guardian identity. The delegate can call each function separately for every guardian that selected it.

The stored signal is attributed to `onBehalf`, not to `msg.sender`. As a result, one delegate can create signals for several distinct guardian addresses, and the quorum logic counts those addresses separately.

Impact: A guardian quorum can appear to contain several independent guardian approvals while one delegate key controls all of them.

If multiple guardians share a delegate, compromise of that delegate can let one key create enough flags to ban an account, create enough suspect reports to increase activation delay, or submit heartbeats for multiple guardian identities toward the quorum frontier.

The issue reduces the guardian set's effective key independence to the number of distinct delegate keys, which can be materially smaller than the configured guardian quorum.

Proof of Concept:

1. Configure a guardian set containing Guardian A, Guardian B, and Guardian C with a quorum of two.
2. Guardian A calls `GuardianSignals::setDelegate(delegate)`.
3. Guardian B calls `GuardianSignals::setDelegate(delegate)`.
4. The delegate calls `GuardianSignals::flag(marketId, Guardian A, accounts)`.
5. The delegate calls `GuardianSignals::flag(marketId, Guardian B, accounts)`.
6. `GuardianSignals::_flag` adds Guardian A and Guardian B to `GuardiansSignal::flaggedBy` as separate identities.
7. `GuardiansSignal::flaggedBy.length()` reaches the quorum even though the delegate is the only key that submitted both signals.
8. `GuardianSignals::_isBanned` returns true and `LidoLendMarketController::_handleBorrow` rejects borrowing for the affected account.

The same delegate can submit `GuardianSignals::suspect` or `GuardianSignals::heartbeat` calls on behalf of every guardian that selected it.

Recommended Mitigation: Require active guardian delegates to be unique.

`GuardianSignals::setDelegate` can reject a delegate address already assigned to another active guardian. Governance should also define whether a guardian acting directly counts as a separate key from its delegate and enforce the same uniqueness rule when guardian membership changes.

If shared delegates are intentionally supported, quorum-sensitive actions should count distinct delegate keys rather than guardian identities.

Lido: Fixed in commit de7f986

Cyfrin: Verified.

8.3 Informational

8.3.1 `ExitBookBase::_priorityFeeRate` and the request deadline do not exclude market-pause time from their wall-clock windows

Description: A request's priority fee is a Dutch auction over wall-clock time. `_priorityFeeRate` computes `elapsed = block.timestamp - request.createdAt` and ramps the rate from `startRate` to `endRate` between `rampStart` and `rampEnd` (`lido-lend-markets/src/periphery/ExitBookBase.sol:308-321`). The request's `deadline` is enforced the same way, against raw `block.timestamp` (`lido-lend-markets/src/libs/ExitBookLib.sol:103-109`). Neither excludes time during which a fill was impossible.

A fill is impossible while the market is paused. `fillExitRequest` always reaches the source - a supply and a withdraw on the core for the Morpho book, a deposit and a redeem on the vault for the ERC-4626 book - and each of those fires a controller hook whose first statement is `_checkResumed`, which reverts while the controller is paused (`lido-lend-markets/src/controllers/LidoLendMarketController.sol:132-134`). So across a pause the ramp keeps climbing toward `endRate` and the deadline keeps running, through a window in which no filler could have filled. The pause span is already on-chain - `PausableUntil` stores the resume timestamp (`lido-lend-markets/src/utils/PausableUntil.sol:21`) - and is simply never subtracted from either window.

Impact: A lender whose request is open across a market pause is paid less on the eventual fill than the auction they configured would have charged for the same amount of fillable time. The priority fee is deducted from the lender's proceeds and handed to the filler (`lenderAssets = owed - fee`, `lido-lend-markets/src/periphery/ExitBookBase.sol:171-175`), and the ramp advances during the pause without rewinding on resume, so the first filler after the pause collects a rate the request reached without any filler having competed for it. The overcharge is bounded by `MAX_PRIORITY_FEE`, the 10% cap on the end rate checked at placement (`lido-lend-markets/src/periphery/ExitBookBase.sol:75` and `:289`), so the lender's worst case is the full ramp rather than an unbounded loss. In the other direction, a request whose `deadline` falls inside or shortly after the pause expires without ever having been fillable - `requireFillable` then reverts `RequestExpired` (`lido-lend-markets/src/libs/ExitBookLib.sol:108`) and the lender must place a new request. The lender's shares never leave their position, so nothing is locked or misdirected; the harm is a mispriced fee and a wasted request.

Recommended Mitigation: Subtract paused time from both windows. Have the concrete book expose the source controller's accumulated paused duration for the market, then compute the ramp against an adjusted elapsed value - `block.timestamp - request.createdAt` minus the paused time that fell inside the request's life - and compare the deadline against that same adjusted clock. That keeps the auction measuring fillable time, which is what it was designed to price, and it preserves every existing behaviour when the market was never paused, since the adjustment is then zero. If threading the pause span into the book is unwelcome, the lighter alternative is an owner-only extension: allow a request whose window elapsed while its source was paused to have its `deadline` extended and its ramp restarted, callable only by the request owner and only for the paused portion, so the lender is not forced to re-place and re-queue a request that never had a chance to fill.

Lido: Acknowledged.

8.3.2 `MorphoExitBook::placeExitRequest` omits the `ZeroPricePerShare` guard its sibling exit book enforces

Description: `Erc4626ExitBook::placeExitRequest` snapshots the source's price per share and rejects a zero snapshot outright, with a comment naming the harm: "A zero snapshot price pays the lender `owed = 0` on every fill and hands the whole redemption to the filler. Reject it." (`lido-lend-markets/src/periphery/Erc4626ExitBook.sol:77-81`). `MorphoExitBook::placeExitRequest` takes the same class of snapshot for the Morpho source - `pricePerShare = _supplySharesToAssets(marketParams, WAD)` (`lido-lend-markets/src/periphery/MorphoExitBook.sol:119`) - but declares no such error and performs no such check.

That snapshot is what every fill pays the lender against. `fillExitRequest` computes `owed = min(mulDiv(filledShares, request.pricePerShare, WAD), assetsOut)`, then `lenderAssets = owed - fee` and `fillerGross = (assetsOut - owed) + fee` (`lido-lend-markets/src/periphery/ExitBookBase.sol:170-175`). A `pricePerShare` of zero makes `owed` zero on every fill, so `lenderAssets` is zero and the entire redeemed amount is split between the filler and the service cut.

`_supplySharesToAssets` rounds down (`lido-lend-markets/src/periphery/MorphoExitBook.sol:175-179`), so it returns zero once the market's supply shares have grown large relative to its supply assets - the state a bad-debt write-down leaves behind, since it reduces `totalSupplyAssets` while `totalSupplyShares` is unchanged. The snapshot is frozen into the request at creation (`lido-lend-markets/src/libs/ExitBookLib.sol:55-69`) and never re-read, so a request placed in that state stays stamped with a zero price for its whole life.

Files:

`MorphoExitBook::placeExitRequest`

Impact: A lender who places a Morpho exit request while the market's supply price rounds to zero receives nothing from any fill of that request, and every share filled is paid out to the filler instead. The request cannot be re-priced, because `pricePerShare` is written once at placement, so the lender's only remedy is to notice the zero price before a fill lands and cancel the request (`lido-lend-markets/src/periphery/ExitBookBase.sol:191-194`). Reaching the state requires a bad-debt event severe enough to round the market's assets-per-share to zero, which is why this is rated Low. The defect is the asymmetry: the sibling book treats exactly this case as unacceptable and rejects it at placement, while this one accepts it silently.

Recommended Mitigation: Apply the guard the sibling book already carries. The narrow fix is to declare a `ZeroPricePerShare` error on `MorphoExitBook` and revert on it in `placeExitRequest`:

```
uint256 pricePerShare = _supplySharesToAssets(marketParams, WAD);
if (pricePerShare == 0) revert ZeroPricePerShare();
```

The better fix is to move the check into the shared `ExitBookBase::_createRequest`, which already validates the request's other terms (`lido-lend-markets/src/periphery/ExitBookBase.sol:288-294`) and already receives `pricePerShare` as an argument. Putting it there closes the gap for both books and for any book added later, and it leaves every currently-valid placement unaffected, since a non-zero snapshot is a precondition both books already depend on.

Lido: Fixed in commit 2653b5b.

Cyfrin: Fix verified. The zero-price guard now lives in the shared `_createRequest` (`ZeroAssetQuote`), covering `MorphoExitBook`. It comes with the EXT-02 quote rework, which also changes the `Request` struct layout.

8.3.3 `CollateralActivationLib::_drain` emits `CollateralDrained` unconditionally and carries no drained amount

Description: `_drain` works out how much comes out of each un-matured bucket, writes the results, and emits on its only exit path regardless of whether either bucket moved (`lido-lend-markets/src/libs/CollateralActivationLib.sol:307-327`):

```
uint256 fromDust = Math.min(dust, amount);
uint256 fromActivating = Math.min(amount - fromDust, Math.saturatingSub(activating, lockAmount));

position.dust = uint128(dust - fromDust);
position.activating = uint128(activating - fromActivating);

if (position.activating == 0) _clearTimer(position);

emit CollateralDrained(borrower, position);   // outside any branch
```

When the borrower's un-matured buckets are already empty, `fromDust` and `fromActivating` are both zero, no storage slot changes value, and the event still fires with an all-zero position. That is the steady state rather than an edge case: collateral only sits in these buckets during the activation window, and once `sync` matures a lot both `activating` and `dust` are zero while the collateral itself is active. Every later operation on that borrower still routes through `_drain - debit` on a withdrawal (`lido-lend-markets/src/controllers/LidoLendMarketController.sol:402`), and `seize` on a liquidation or a settle, which calls it with no lock floor (`lido-lend-markets/src/libs/CollateralActivationLib.sol:256-258`).

`CollateralDrained(address indexed borrower, Position position)` (lido-lend-markets/src/libs/CollateralActiva is also the only one of the library's five position events with no `amount` parameter: `CollateralActivating`, `CollateralActivated`, `CollateralDusted` and `CollateralDemoted` all carry one (lido-lend-markets/src/libs/CollateralActivationLib.sol:53-66). A consumer therefore cannot read the drained quantity off the event at all, and must diff the emitted position against the state it inferred from the preceding event.

Files:

`CollateralActivationLib::_drain`

Impact: `CollateralDrained` is the log a monitor would subscribe to in order to detect un-matured collateral leaving the activation throttle - which is exactly the behaviour the throttle exists to constrain. Because it fires on every routine withdrawal, liquidation and settle against any borrower whose collateral has already matured, that signal is buried in a continuous stream of no-op events, and because the event carries no amount, telling a real drain apart from a no-op requires the consumer to independently track each borrower's prior bucket state. The emitted position is the true resulting state, so no on-chain accounting is wrong and no consumer is misled about the position itself. The defect is that the event is unusable for the purpose its name and its NatSpec - "Collateral is drained from the un-matured buckets" (lido-lend-markets/src/libs/CollateralActivationLib.sol:61) - advertise.

Recommended Mitigation: Carry the drained amount, and emit only when the drain actually moved collateral:

```
event CollateralDrained(address indexed borrower, uint256 amount, Position position);
...
uint256 drained = fromDust + fromActivating;

position.dust = uint128(dust - fromDust);
position.activating = uint128(activating - fromActivating);

if (position.activating == 0) _clearTimer(position);

if (drained != 0) emit CollateralDrained(borrower, drained, position);
return position.dust + position.activating;
```

Guarding only the emit, and leaving the writes, the `_clearTimer` call and the return outside it, keeps every current behaviour intact - callers rely on the post-drain inactive total being returned on every path, including the no-op one. Adding the `amount` parameter also brings the event into line with the other four position events in the library, all of which already carry it.

Lido: Fixed in commit 02cb5b0.

Cyfrin: Verified.

8.3.4 `GuardianSignals::setQuorum, prune, unflag` flip effective ban status with no `Banned` or `Unbanned` event

Description: The effective ban is a derived predicate, not the latched bit alone: `_isBanned` returns `signal.banned || signal.flaggedBy.length() >= _quorum` (lido-lend-markets/src/GuardianSignals.sol:300-302). `Banned` is emitted only from `_flag`, when a new flag crosses the then-current quorum (lido-lend-markets/src/GuardianSignals.sol:380-383), and from `ban` (line 271); `Unbanned` is emitted only from `unban` (line 280). But the predicate also depends on `_quorum` and on `flaggedBy.length()`, and three paths move those two operands across the threshold while emitting nothing about ban state:

- `setQuorum` (lido-lend-markets/src/GuardianSignals.sol:124-127). Lowering the quorum makes every account whose live flag count already sits at or above the new value effectively banned. `_flag` never ran at the new quorum, so the latch was never written and no `Banned` log exists anywhere. Only `QuorumSet` is emitted.
- `prune` (lido-lend-markets/src/GuardianSignals.sol:245-261). Permissionless, and removing an ex-guardian's flag can drop a count-derived ban back below the quorum, silently lifting it. Only `FlagPruned` is emitted.

- `unflag` (`lido-lend-markets/src/GuardianSignals.sol:210-218`, via `_unflag` at lines 388-392). The same shape, with a current guardian retracting its own flag. Only `Unflagged` is emitted.

Files:

`GuardianSignals::setQuorum, prune, unflag`

Impact: Off-chain consumers of the guardian-signal log stream - the risk dashboard, ban monitoring and alerting, and any keeper deciding whether a borrower's activating lot is worth a maturation call - diverge from on-chain `isBanned` in both directions. An indexer that reconstructs ban state as "saw `Banned` and no later `Unbanned`", the reading the events invite, reports a silently-banned borrower as healthy while the contract hard-blocks its borrows and pins its activating collateral's unlock to `type(uint256).max` (`lido-lend-markets/src/libs/CollateralActivationLib.sol:368`), and reports a silently-unbanned borrower as banned while the contract permits both again. No on-chain value is misdirected: every in-protocol consumer reads `getOpinion` directly, so the contract's own behaviour stays self-consistent. This is an observability defect.

Recommended Mitigation: Emit the transition where the derived predicate actually flips, rather than only where the latch is written. The cleanest form removes the derived term from `_isBanned` altogether: make the ban a pure latch (`return signal.banned;`) and leave `_flag` as its only writer, which it already is. The quorum then decides only when a new flag latches a ban, and no config change or flag removal can silently create or destroy one, so the log is complete by construction. If the derived term must stay, `prune` and `_unflag` must each evaluate `_isBanned` before and after the mutation and emit `Banned` or `Unbanned` on a flip. Note that `setQuorum` cannot do that in bounded gas - it would have to iterate every market-and-account signal to find the accounts whose status flipped - which is itself an argument for the latch-only form.

Lido: Acknowledged.

8.3.5 The adaptive-curve rate math is duplicated verbatim between `AdaptiveCurveIrm` and `AdaptiveCurveIrmLib`

Description: `AdaptiveCurveIrmLib` re-implements the whole rate computation of `AdaptiveCurveIrm` - the `err` derivation, the trapezoidal average, the `_curve` formula, the `_newRateAtTarget` exp-and-bound step, and the `maxBorrowRate` cap - rather than sharing it. The two copies are byte-for-byte equivalent today, but nothing links them: the `_curve` and `_newRateAtTarget` of `AdaptiveCurveIrm` are `private`, so the library cannot call them, and the library's own copies are `internal` to a separate file. Any future change to the curve (a new cap, a different `errNormFactor`, an extra bound) applied to only one copy makes the periphery preview silently disagree with the rate the core actually charges, with no compile error and no test that necessarily catches it.

The library is explicitly the integrator-facing surface (its NatSpec describes it as the library to compute `borrowRateView` and expected market balances from only the market id), so a divergence is a divergence between what integrators quote and what borrowers pay.

```
lido-lend-irm/src/adaptive-curve-irm/AdaptiveCurveIrm.sol
188:         uint256 rate = uint256(_curve(avgRateAtTarget, err, curveParams.curveSteepness));
189:
190:         // The cap bounds only the reported rate. Adaptation is err-driven, not rate-driven, so
↪   `endRateAtTarget`
191:         // stays untouched and the state machine is unchanged.
192:         if (curveParams.maxBorrowRate != 0 && rate > curveParams.maxBorrowRate) rate =
↪   curveParams.maxBorrowRate;
201:     function _curve(int256 _rateAtTarget, int256 err, int256 curveSteepness) private pure returns
↪   (int256) {
202:         // Non negative because 1 - 1/C >= 0, C - 1 >= 0.
203:         int256 coeff = err < 0 ? WAD - WAD.wDivToZero(curveSteepness) : curveSteepness - WAD;
204:         // Non negative if _rateAtTarget >= 0 because if err < 0, coeff <= 1.
205:         return (coeff.wMulToZero(err) + WAD).wMulToZero(_rateAtTarget);
206:     }

lido-lend-irm/src/adaptive-curve-irm/libraries/periphery/AdaptiveCurveIrmLib.sol
119:         // Safe "unchecked" cast because avgRateAtTarget >= 0.
120:         uint256 rate = uint256(_curve(avgRateAtTarget, err, p.curveSteepness));
```

```
121:
122:         // Mirrors the IRM's cap: bounds only the reported rate, `endRateAtTarget` untouched.
123:         if (p.maxBorrowRate != 0 && rate > p.maxBorrowRate) rate = p.maxBorrowRate;
141:     function _curve(int256 _rateAtTarget, int256 err, int256 curveSteepness) internal pure returns
↪  (int256) {
142:         // Non negative because 1 - 1/C >= 0, C - 1 >= 0.
143:         int256 coeff = err < 0 ? WAD - WAD.wDivToZero(curveSteepness) : curveSteepness - WAD;
144:         // Non negative if _rateAtTarget >= 0 because if err < 0, coeff <= 1.
145:         return (coeff.wMulToZero(err) + WAD).wMulToZero(_rateAtTarget);
146:     }
```

Recommended Mitigation: Extract the pure curve math into a single shared internal library that both the IRM and the periphery library import, so a change can only be made in one place.

```
// lido-lend-irm/src/adaptive-curve-irm/libraries/CurveMathLib.sol
library CurveMathLib {
    function err(Market memory market, int256 targetUtilization) internal pure returns (int256) { ... }
    function curve(int256 rateAtTarget_, int256 err_, int256 curveSteepness) internal pure returns
    ↪  (int256) { ... }
    function newRateAtTarget(int256 startRateAtTarget, int256 linearAdaptation, AdaptiveCurveParams
    ↪  memory p)
        internal pure returns (int256) { ... }
    function rateFrom(AdaptiveCurveParams memory p, int256 startRateAtTarget, Market memory market)
        internal view returns (uint256 rate, int256 endRateAtTarget) { ... }  // incl. the
        ↪  maxBorrowRate cap
}
```

`AdaptiveCurveIrm::_borrowRate` then reduces to reading `marketCurveConfig[id]` and `rateAtTarget[id]` and calling the shared helper, and `AdaptiveCurveIrmLib::_borrowRate` to fetching the same two values over the `IAdaptiveCurveIrm` interface and calling the identical helper. The duplicated `_err`, `_curve` and `_newRateAtTarget` bodies in both files are then deleted.

Lido: Fixed in commit ad62ada.

Cyfrin: Verified.

8.3.6 Zero-address validation is applied inconsistently across constructors and initializers

Description: Nearly every deployment entry point in this codebase rejects the zero address for its wiring arguments, but `LidoLendMarketController` and `RiskStewardDashboard` do not. Both silently accept `address(0)` for the core, the admin, and (in the controller's case) the guardian-signals registry - the two contracts whose bricking is hardest to recover from, since the controller is bound into a market at creation and cannot be re-pointed afterwards.

Missing:

```
lido-lend-markets/src/controllers/LidoLendMarketController.sol
103:     constructor(IMorpho lidoLend) {
104:         LIDO_LEND = lidoLend;                          // no zero check on the core
105:         _disableInitializers();
106:     }

110:     function initialize(
111:         Id marketId_,
112:         address admin,                                 // no zero check
113:         IGuardianSignalsView guardianSignals,          // no zero check
114:         CollateralActivationConfig calldata activationConfig,
115:         SettlementConfig calldata settlementConfig
116:     ) external initializer {
117:         __AccessControl_init();
118:         _grantRole(DEFAULT_ADMIN_ROLE, admin);
```

```
119:         marketId = marketId_;
120:         _guardianSignals = guardianSignals;
121:         _settlementQueue.init();
122:         _settlementQueue.setSettlementConfig(settlementConfig);
123:         _activation.setConfig(activationConfig);
124:     }

lido-lend-markets/src/periphery/RiskStewardDashboard.sol
53:     constructor(address admin, ILidoLend lidoLend) {
54:         LIDO_LEND = lidoLend;                        // no zero check on the core
55:         _grantRole(DEFAULT_ADMIN_ROLE, admin);       // no zero check on the admin
56:     }
```

Present everywhere else in scope:

```
lido-lend-markets/src/LidoLendERC4626.sol
74:         if (lidoLend == address(0)) revert ZeroLidoLend();
88:         if (controller_ == address(0)) revert ZeroController();
90:         if (marketParams_.loanToken == address(0)) revert ZeroLoanToken();
92:         if (marketParams_.collateralToken == address(0)) revert ZeroCollateralToken();

lido-lend-markets/src/GuardianSignals.sol
94:         if (admin == address(0)) revert ZeroAddress();

lido-lend-markets/src/periphery/LidoLendOwner.sol
44:         if (address(core) == address(0) || admin == address(0)) revert ZeroAddress();

lido-lend-markets/src/periphery/MorphoExitBook.sol
69:         if (morpho == address(0)) revert ZeroMorpho();

lido-lend-markets/src/oracle/UsdyStableOracle.sol
88:         if (usdyOracle == address(0)) revert ZeroAddress();
101:         if (feed == address(0) || loanToken == address(0)) revert ZeroAddress();

lido-lend-markets/src/oracle/WstEthRateOracle.sol
48:         if (wstETH_ == address(0)) revert ZeroAddress();

lido-lend-irm/src/adaptive-curve-irm/AdaptiveCurveIrm.sol
64:         require(router != address(0), OwnerManagerErrors.ZERO_ADDRESS);

lido-lend-irm/src/irm-router/IrmRouter.sol
44:         require(morpho != address(0), OwnerManagerErrors.ZERO_ADDRESS);
```

Recommended Mitigation: Add the missing guards so every deployment entry point follows the same convention.

```
// LidoLendMarketController
 constructor(IMorpho lidoLend) {
+    if (address(lidoLend) == address(0)) revert ZeroAddress();
     LIDO_LEND = lidoLend;
     _disableInitializers();
 }

 function initialize(
     Id marketId_,
     address admin,
     IGuardianSignalsView guardianSignals,
     CollateralActivationConfig calldata activationConfig,
     SettlementConfig calldata settlementConfig
 ) external initializer {
+    if (admin == address(0) || address(guardianSignals) == address(0)) revert ZeroAddress();
+
     __AccessControl_init();
```

```
    ...
}
```

```
// RiskStewardDashboard
 constructor(address admin, ILidoLend lidoLend) {
+    if (admin == address(0) || address(lidoLend) == address(0)) revert ZeroAddress();
     LIDO_LEND = lidoLend;
     _grantRole(DEFAULT_ADMIN_ROLE, admin);
 }
```

Lido: Fixed in commit 5ba07d3.

Cyfrin: Verified.

8.3.7 The `LidoLendMarketController` guardian-registry binding is observable neither by a getter nor by an event

Description: `LidoLendMarketController` stores the `IGuardianSignalsView` registry that decides who is banned and how long collateral activation is delayed. That registry is `private`, has no accessor anywhere in the contract or its interfaces, and `initialize` emits no event when it binds it. It is also bound once and never re-settable. Every other dependency and parameter of the controller is observable - `LIDO_LEND` (public immutable), `marketId`, `supplyCap`, `isFrozen`, `hasInstantActivation`, `collateralActivationConfig`, `settlementConfig` - and the two config structs even emit on write (`CollateralActivationConfigSet`, `SettlementConfigSet`). The guardian registry is the one wired dependency that falls through both surfaces.

The practical consequence: an on-chain consumer (a governance guard, the `RiskStewardDashboard`, another market's controller) cannot verify which guardian committee gates a given market, and an off-chain indexer cannot recover it from logs. Only a raw `eth_getStorageAt` against a `private` slot works, which is exactly the read that breaks silently on the next storage-layout change.

```
lido-lend-markets/src/controllers/LidoLendMarketController.sol
// private, no getter anywhere in the contract or in ILidoLendMarketController /
↪  ILidoLendMarketConfigurator
74:     /// @dev When set, new borrows and debt-bearing collateral withdrawals revert and activation
↪  maturation
75:     ///      defers. Debt-free positions can still withdraw.
76:     IGuardianSignalsView private _guardianSignals;

// bound here, with no event
110:     function initialize(
111:         Id marketId_,
112:         address admin,
113:         IGuardianSignalsView guardianSignals,
114:         CollateralActivationConfig calldata activationConfig,
115:         SettlementConfig calldata settlementConfig
116:     ) external initializer {
117:         __AccessControl_init();
118:         _grantRole(DEFAULT_ADMIN_ROLE, admin);
119:         marketId = marketId_;
120:         _guardianSignals = guardianSignals;
121:         _settlementQueue.init();
122:         _settlementQueue.setSettlementConfig(settlementConfig);
123:         _activation.setConfig(activationConfig);
124:     }

// while every neighbouring piece of state is exposed
61:     IMorpho public immutable LIDO_LEND;
68:     Id public marketId;
71:     uint256 public supplyCap;
74:     bool public isFrozen;
```

```
165:     function collateralActivationConfig() external view returns (CollateralActivationConfig memory)
↪  {
192:     function hasInstantActivation(address caller) external view returns (bool) {
238:     function settlementConfig() external view returns (SettlementConfig memory) {
```

Recommended Mitigation: Expose the binding, and log it at initialization so indexers can pick it up.

```
// ILidoLendMarketController
+    /// @notice The guardian-signal registry this controller reads bans and strikes from.
+    function guardianSignals() external view returns (IGuardianSignalsView);
+
+    /// @notice Emitted once, at initialization, when the guardian-signal registry is bound.
+    event GuardianSignalsSet(IGuardianSignalsView guardianSignals);
```

```
// LidoLendMarketController
 IGuardianSignalsView private _guardianSignals;
+
+    /// @inheritdoc ILidoLendMarketController
+    function guardianSignals() external view returns (IGuardianSignalsView) {
+        return _guardianSignals;
+    }

 // in initialize()
     _guardianSignals = guardianSignals;
+    emit GuardianSignalsSet(guardianSignals);
```

Consider emitting `marketId_` in the same initialization event (or an `Initialized` event carrying the market id, the admin and the guardian registry) - `marketId` has a public getter, but no log currently ties a deployed controller address to the market it was bound to.

Lido: Fixed in commits 3bc6ae8 and cf543a5.

Cyfrin: Verified.

8.3.8 `AdaptiveCurveIrm::_borrowRate` reverts when a market's curve config is unset, a failure mode the base IRM interface did not have

Description: `AdaptiveCurveIrm::_borrowRate` requires a config to be registered for the market before computing a rate: `require(address(curveConfig) != address(0), ErrorsLib.CONFIG_NOT_SET);`.

`LidoLend::_accrueInterest` and `LidoLend::_createMarket` both call `IIrm.borrowRate` unconditionally whenever `marketParams.irm != address(0)`, with no try/catch, so every state-changing action on such a market (supply, withdraw, borrow, repay, settle, liquidate, accrueInterest, and market creation itself) reverts entirely until the config is set.

This is a new failure mode introduced, since the original IRM contracts never reverted on `_borrowRate` calls.

Impact: A market routed to `AdaptiveCurveIrm` (directly or via `IrmRouter`) whose curve config is not yet set is completely unusable, including unable to be created, until an owner/manager transaction sets the config, a new failure mode versus an IRM that always returns a rate.

Recommended Mitigation: Document and operationally enforce that curve config must be set before a market is routed to `AdaptiveCurveIrm`, and consider ordering deployment scripts/governance actions so config-setting always precedes market creation or routing.

Lido: Acknowledged.

8.3.9 `AdaptiveCurveIrmLib` does not detect a market that has migrated away from `AdaptiveCurveIrm`, returning misleading rates from stale config

Description: `AdaptiveCurveIrmLib::_borrowRate` only checks that a curve config exists for the market id: `require(address(config) != address(0), ErrorsLib.CONFIG_NOT_SET);`.

`IrmRouter::setMarketIrm` lets a market be repointed to a different `IIrm` implementation (e.g. `LinearKinkIrm`) without ever touching `AdaptiveCurveIrm.marketCurveConfig[id]`, so a market that has fully migrated off `AdaptiveCurveIrm` still has a non-zero config entry there and this library keeps computing and returning a rate as if the market were still routed to it, per its own documented precondition ("Works only for markets routed to an AdaptiveCurveIrm").

Impact: Off-chain or on-chain consumers of this library (e.g. vault preview/accounting adapters) can silently receive a plausible but incorrect rate/balance projection for a market that no longer uses `AdaptiveCurveIrm`, instead of a clear revert.

Recommended Mitigation: Have `IrmRouter::setMarketIrm` clear the outgoing IRM's per-market config when migrating away from it, or have `AdaptiveCurveIrmLib` cross-check the market's currently routed IRM before trusting its stored config.

Lido: Acknowledged.

8.3.10 `LinearKinkIrm::setMarketKinkParams` can never write `optimalUtilization = 0`, so a market's config can never be unset

Description: `setMarketKinkParams` requires `require(params.optimalUtilization > 0, ...)` before writing `marketKinkParams[id]`, while `_borrowRate` treats `optimalUtilization == 0` as the sentinel for "no config" (`require(p.optimalUtilization != 0, ErrorsLib.CONFIG_NOT_SET)`).

Since the setter can never store a zero `optimalUtilization`, once a market's kink config has been set once, it can only be replaced with another valid config, never cleared back to the unset state through this function.

Impact: There is no way to fully unset/disable a market's `LinearKinkIrm` config once set; stale config permanently remains in storage even after a market is no longer intended to use it.

Recommended Mitigation: Add a separate function (or a documented sentinel path) that allows an authorized caller to explicitly clear a market's `marketKinkParams` entry back to its zero/unset state.

Lido: Acknowledged.

8.3.11 `GuardianSignals::setQuorum` does not resync the quorum-scan frontier

Description: `GuardianSignals::setQuorum` updates `_quorum` but never calls `_syncQuorumBlock`.

```
function setQuorum(uint256 newQuorum) external onlyRole(MANAGER_ROLE) {
    _setQuorum(newQuorum);
    emit QuorumSet(newQuorum);
}
```

`_quorumFrontierTimestamp` keeps reflecting the frontier computed under the old quorum until someone separately calls the permissionless `syncQuorumBlock`, so `getOpinion` can return a frontier inconsistent with the just-updated quorum size.

Impact: Activation maturation via the guardian-quorum path can rely on a frontier that does not correspond to the currently configured quorum size, delaying or prematurely allowing maturation.

Recommended Mitigation: Call `_syncQuorumBlock` at the end of `setQuorum` so the frontier is recomputed under the new quorum in the same transaction as the change.

Lido: Acknowledged.

8.3.12 `CollateralActivationLib::sync` anchors the guardian-quorum fallback on `activatingStart` instead of `unlockTime`

Description: In `CollateralActivationLib::sync` a lot matures once its unlock time has passed and either the guardian quorum has scanned past it, or a fallback deadline has elapsed:

```
bool hasGuardiansQuorum = unlockTime <= opinion.quorumFrontierTimestamp;
bool maxQuorumDelayElapsed = block.timestamp >= position.activatingStart +
↪  $.config.maxGuardiansQuorumDelay;
```

The two branches use different anchors. `unlockTime` includes the strike/suspect surcharge; `activatingStart` does not. `setConfig` enforces `maxGuardiansQuorumDelay > maxActivationDelay`, so an unsurcharged lot always gets a real window in which the quorum branch is the only way to mature. Once a surcharge pushes `unlockTime` past `activatingStart + maxGuardiansQuorumDelay`, that window is zerom the fallback is already true when the unlock passes, and the quorum never matters.

This is not exploitable: a surcharged lot still waits strictly longer in absolute terms. But it is counter-intuitive that the more suspicious a lot is, the less its maturation depends on an actual guardian scan.

Recommended Mitigation: Anchor the fallback on `unlockTime`, so it is a fixed grace period granted to the quorum after the lot could first mature:

```
bool maxQuorumDelayElapsed = block.timestamp >= unlockTime + $.config.maxGuardiansQuorumDelay;
```

If the current behaviour is intended as an absolute ceiling on activation latency rather than a quorum grace period, document that surcharges consume it.

Lido: Acknowledged.

8.3.13 `LidoLend::accrueInterest` changes the Adaptive rate used by future borrowers while the market is empty

Description: `LidoLend::accrueInterest` calls the configured IRM even when `totalBorrowAssets` is zero. `AdaptiveCurveIrm::borrowRate` then advances its persistent `rateAtTarget` state from the elapsed time and the market's zero utilization state.

Because accrual is permissionless, a caller can divide the same idle horizon into multiple checkpoints. The `AdaptiveCurveIrm` update uses discrete fixed point calculations at each checkpoint, so a partitioned history and a single combined update can finish with equal market balances and equal final timestamps but different stored `rateAtTarget` values.

```
// LidoLend::_accrueInterest
uint256 borrowRate = IIrm(marketParams.irm).borrowRate(marketParams, market[id]);
uint256 interest = market[id].totalBorrowAssets.wMulDown(borrowRate.wTaylorCompounded(elapsed));

// AdaptiveCurveIrm::borrowRate
(uint256 avgRate, int256 endRateAtTarget) = _borrowRate(id, market);
rateAtTarget[id] = endRateAtTarget;
```

The next borrower consequently enters a market whose initial rate was partly selected by an unrelated caller's checkpoint schedule. Later accrual can preserve or amplify that state difference.

Impact: A permissionless caller can slightly influence the interest rate state inherited by future borrowers without supplying, borrowing, or retaining a position.

Proof of Concept:

1. Configure two identical markets with `AdaptiveCurveIrm` and zero `totalBorrowAssets`.
2. Start both markets from the same core state, timestamp, and `rateAtTarget`.
3. Over a chosen idle horizon, call `LidoLend::accrueInterest` at intermediate checkpoints in the first market.
4. Leave the second market untouched during the same horizon and call `LidoLend::accrueInterest` only at the end.
5. Confirm that both markets still have zero borrow interest and finish with the same final `lastUpdate`.

6. Compare `AdaptiveCurveIrm::rateAtTarget` between the markets. The partitioned and combined histories can produce different stored values.
7. Open identical borrow positions in both markets at the same final time.
8. Apply the same subsequent accrual schedule to both markets.
9. Compare their borrow rates and interest totals. The different idle period endpoint is inherited by the otherwise identical borrowers.

Recommended mitigation:

Preserve zero debt IRM updates while making idle evolution independent of checkpoint partitioning. The IRM can derive the current endpoint from a stable reference state and total elapsed time, or otherwise bound and document any unavoidable approximation difference.

Lido: Acknowledged

8.3.14 `LidoLend::setFeeRecipient, setOwner` accept the zero address

Description: `LidoLend::setFeeRecipient` validates only that `newFeeRecipient` differs from the current `feeRecipient` and never rejects `address(0). setOwner` validates only `newOwner != owner` and also accepts the zero address. Both are `onlyOwner`-gated and both lack the zero-address guard that `initialize` applies to its `newOwner` parameter. The `feeRecipient` slot defaults to zero, and `_accrueInterest` unconditionally mints fee shares to `position[id][feeRecipient]` when the market fee is non-zero.

Impact: * When the owner enables a non-zero fee before setting a real recipient, or passes `address(0)` to `setFeeRecipient`, every subsequent `_accrueInterest` mints fee shares to `address(0)`, which nobody can withdraw. The total supply shares increase by the fee amount, diluting every other supplier, while the fee value itself is permanently locked.

- Once `owner` is set to zero, every `onlyOwner` function (`createMarket`, `setFee`, `setFeeRecipient`, and `setOwner` itself) is permanently inaccessible because `msg.sender == address(0)` can never hold.

Recommended Mitigation: Add `require(newFeeRecipient != address(0), ErrorsLib.ZERO_ADDRESS)` to `setFeeRecipient` and `require(newOwner != address(0), ErrorsLib.ZERO_ADDRESS)` to `setOwner`.

Lido: Acknowledged

8.3.15 Single-step ownership transfer in `OwnerManagerBase::transferOwnership` and `LidoLend::setOwner` can permanently brick owner-only functions

Description: `OwnerManagerBase::transferOwnership` and `LidoLend::setOwner` both implement a one-step ownership rotation with no pending-acceptance handshake. `OwnerManagerBase::_transferOwnership` rejects only the zero address before writing `owner = newOwner`, and `LidoLend::setOwner` rejects only the idempotency guard `newOwner != owner` before assigning. Neither requires the incoming owner to actively acknowledge the role transfer. The inherited contracts `AdaptiveCurveIrm`, `LinearKinkIrm`, and `IrmRouter` use `OwnerManagerBase` for owner and manager rotation, so a single mistyped or wrong non-zero address permanently transfers the owner role on all three.

Impact: If the current owner transfers ownership to a wrong non-zero address (a typo, a contract with no owner capability, or a dead EOA), every owner-gated function becomes permanently unreachable with no recovery path. On the core this includes `createMarket`, `setFee`, and `setFeeRecipient`; on the IRM contracts it includes `transferOwnership` and `setManager` (via `OwnerManagerBase`), plus `applyIrmConfig` on `AdaptiveCurveIrm`/`LinearKinkIrm` and `setMarketIrm` on `IrmRouter`. The harm requires the owner to act erroneously.

Recommended Mitigation: Adopt a two-step ownership rotation. `setOwner` and `transferOwnership` should store a `pendingOwner`, and a separate `acceptOwnership` callable only by that pending address should finalize the transfer, proving the incoming owner controls the key before the role changes hands. Additionally, add a zero-address guard `require(newOwner != address(0), ...)` to `LidoLend::setOwner` to match the guard in `initialize`.

Lido: Acknowledged

8.3.16 In `LidoLendERC4626::fallbackExit` event `FallbackExit` reports only the settle-leg burn, omitting the liquid-leg shares burned

Description: `LidoLendERC4626::fallbackExit` emits `FallbackExit(..., settledShares, ...)`, passing only the settle-leg burn. The function's `sharesBurned` return value also includes the liquid-leg burn from the redeem or withdraw in the first step, and the `loanAssets` return value is not emitted at all. The event therefore under-counts the total shares burned.

Impact: Off-chain monitors and integrators that key on the `FallbackExit` event under-count total shares burned and cannot observe the liquid-leg `loanAssets`, causing accounting discrepancies in any reporting built on the event log.

Recommended Mitigation: Emit the full `sharesBurned` including the liquid-leg burn and the `loanAssets` value, either by extending the `FallbackExit` event parameters or by emitting a distinct event for the liquid leg.

Lido: Fixed in commit 268d3b6

Cyfrin: Verified.

8.3.17 `RiskStewardDashboard::increaseActivationDelays` never checks `newMinActivationDelay` is at most `newMaxActivationDelay`, deferring to a confusing downstream revert

Description: `RiskStewardDashboard::increaseActivationDelays` enforces a tighten-only guard against the current config for each delay individually but never validates `newMinActivationDelay <= newMaxActivationDelay`. A steward can pass `newMinActivationDelay` greater than `newMaxActivationDelay`, and the call only reverts downstream in `CollateralActivationLib.setConfig`, which enforces the ordering.

Impact: No state corruption occurs because the downstream revert protects the config, but the steward receives a confusing controller-side `InvalidActivationDelays` revert instead of a clear dashboard rejection, and the tighten-only intent is not expressed locally.

Recommended Mitigation: Validate `newMinActivationDelay <= newMaxActivationDelay` in `increaseActivationDelays` and revert with a dashboard-specific error before forwarding to the controller.

Lido: Acknowledged

8.3.18 `AssetRecoverer` recovery functions validate the recipient but not the token, yielding an obscure revert or a silent no-op

Description: `AssetRecoverer::recoverERC20, recoverERC721, recoverERC1155` check `recipient == address(0)` but never validate `token == address(0)`. The behavior on a zero token differs by path: `recoverERC20` reaches `SafeERC20::safeTransfer`, whose code-existence check reverts with `SafeERC20FailedOperation`; `recoverERC1155` reads `balanceOf` first, whose return value cannot be decoded from the empty returndata, so it reverts; `recoverERC721` calls `safeTransferFrom` (no return value) as a silent no-op that emits a success event without moving anything.

Impact: A recoverer who accidentally passes the zero address as the token receives a generic revert (ERC20/ERC1155) or a silent, misleading success (ERC721) rather than a clear zero-token error. No third-party funds are at risk and no assets are lost; the harm is an obscure failure mode from an admin-side error.

Recommended Mitigation: Define a `ZeroToken` error on `AssetRecoverer` and add `if (token == address(0)) revert ZeroToken();` to `recoverERC20`, `recoverERC721`, and `recoverERC1155` before the `recipient` check.

Lido: Acknowledged

8.3.19 Apply mechanical Solidity conventions consistently

Description: Several source locations use mechanically detectable conventions that differ from the codebase's canonical Solidity style. Some `mapping` declarations omit names for both the key and value parameters - for example `mapping(Id => int256)` and `mapping(Id => IAdaptiveCurveConfig)` in `AdaptiveCurveIrm`, `mapping(Id => IIrm)` in `IrmRouter`, the `market/position/isAuthorized/nonce/idToMarketParams` mappings in `LidoLend`, and the heap mapping in `MaxHeapLib`.

Other mappings name the key but omit the value name - for example `mapping(address caller => bool)` in `LidoLendMarketController`, `mapping(address guardian => address delegate)` in `GuardianSignals`, and `mapping(address borrower => Position)` in `CollateralActivationLib`. Additionally, `LidoLend.sol` imports `./libraries/ConstantsLib.sol` as a whole source unit rather than using an explicit named import.

Recommended Mitigation: Name the mapping key and value parameters consistently (e.g. `mapping(Id id => int256 rate)` and `mapping(address caller => bool enabled)`), and use an explicit named import for `ConstantsLib`, consistent with the named-import style used elsewhere in the codebase.

Lido: Partially fixed in PR 128, named the mapping key/value parameters across the `lido-lend-markets` contracts. The `ConstantsLib` named import and the mappings in `lido-lend` and `lido-lend-irm` are unchanged.

Cyfrin: Verified.

8.3.20 `LidoLendMarketFactory` omits fail-fast zero-address and code-size checks on the controller-only token inputs and on two constructor role recipients

Description: Two fail-fast checks are missing from the factory.

`_validateMarketSpec` code-checks the oracle and the guardian registry but not `loanToken` or `collateralToken`, so the controller-only path (`deployMarketController`) wires a controller to a zero or codeless token, producing a dead market id, instead of reverting at the point the bad token was supplied.

The constructor binding-checks four addresses but stores `CIRCUIT_BREAKER` and `ADMIN` with no zero-check, even though `ADMIN` becomes every proxy's admin and sole `DEFAULT_ADMIN_ROLE` holder and `CIRCUIT_BREAKER` becomes every controller's `PAUSER_ROLE` holder. A zero here is baked into every future deployment.

Impact: Bad tokens or a zero `ADMIN` / `CIRCUIT_BREAKER` are accepted instead of rejected. Both are deployer-misconfiguration guards, not attacker paths. The value is failing fast rather than surfacing the mistake late or silently across every deployment.

Recommended Mitigation: Add nonzero and `code.length` checks for `loanToken` and `collateralToken` in `_validateMarketSpec`, and require `admin` and `circuitBreaker` nonzero in the constructor.

Lido: Fixed in commit b236ef5b.

Cyfrin: Verified. The factory now validates the loan and collateral tokens and rejects a zero admin or circuit breaker.

8.3.21 `LidoLendERC4626::settle` and `fallbackExit` transfer seized collateral to a caller-chosen receiver before updating vault accounting, with no reentrancy guard

Description: Both `LidoLendERC4626::settle` and `LidoLendERC4626::fallbackExit` run the external core `settle` call, which ends by sending seized collateral to a caller-chosen `receiver`, before the vault decrements `_managedSupplyShares` and burns the owner's shares. The core sends the collateral as its final line, after its hook reentrancy lock has already cleared, so that transfer is unguarded, and the vault itself has no reentrancy guard.

A collateral token with a transfer callback would therefore hand `receiver` control while the vault is mid-update. No value is extractable, because settle is net-asset-value invariant per vault share: `_managedSupplyShares` and `totalSupply()` are stale in the same ratio while the core totals are already post-settle, so a reentrant `redeem` or `withdraw` gets the correct price and the core caps any withdraw to the vault's actual reduced position. The defect is the CEI violation itself. Shipped markets (wstEth, Usdy) use no callback token and the vault documents that assumption, but `LidoLendMarketFactory` accepts an arbitrary collateral token and does not enforce it.

Impact: `settle` and `fallbackExit` violate checks-effects-interactions ordering with no reentrancy guard. No value is extractable today, so this is defense-in-depth hardening.

Recommended Mitigation: Add a `nonReentrant` guard to `settle` and `fallbackExit` (and the shared deposit and withdraw paths), or have the factory reject a collateral token that advertises a transfer callback.

Lido: Acknowledged

8.4 Gas Optimization

8.4.1 Cache storage values read more than once instead of re-reading the slot

Description: All three projects compile with `via_ir` disabled and the legacy optimizer enabled (`via_ir = false`, `optimizer = true`, `optimizer_runs = 200` in each project's `foundry.toml`), so the optimizer's stack cache does not survive branches, external calls, or internal/library helper calls - the legacy common-subexpression eliminator only caches within a single basic block. In the sites below the same slot is loaded two or more times on the success path, separated by one of those boundaries, with no possible change in between: each repeat is a fresh SLOAD (2,100 gas cold, 100 gas warm). Cache the value in a local, or thread it into the helper, and re-use it.

```
lido-lend-markets/src/controllers/LidoLendMarketController.sol
// `isFrozen` read twice in _handleWithdrawCollateral (hot path: every withdrawCollateral hook)
396:         if (isFrozen && position.borrowShares != 0) revert MarketIsFrozen();
401:         _activation.sync(isFrozen, borrower, _guardianSignals.getOpinion(id, borrower));

// `isFrozen` read twice in _handleBorrow (hot path: every borrow hook)
425:         if (isFrozen) revert MarketIsFrozen();
448:         uint256 inactive = _activation.sync(isFrozen, hookData.onBehalf, opinion);

// `supplyCap` read twice in maxDeposit, separated by an external call the optimizer cannot see through
213:         if (supplyCap == 0) return type(uint256).max;
214:         (uint256 totalSupplyAssets,,,) = _marketTotals(LIDO_LEND.idToMarketParams(marketId));
215:         return Math.saturatingSub(supplyCap, totalSupplyAssets);

// _handleSettle: peekRoot() runs here, and allowedSlice() calls peekRoot($) again internally
// (SettlementQueueLib.sol:155) -- two heap `nodes.length` SLOADs plus a duplicate root-node SLOAD
503:         address riskiestBorrower = _settlementQueue.peekRoot();
509:         uint256 allowedShares = _settlementQueue.allowedSlice(marketView, borrower, position)

// positionOf: settlementLtvOf(borrower) and peekRoot() are both recomputed inside allowedSlice()
// (SettlementQueueLib.sol:155-156)
349:         dashboard.settlementLtv = _settlementQueue.settlementLtvOf(borrower);
350:         dashboard.isNextToSettle = _settlementQueue.peekRoot() == borrower;
351:         dashboard.slice = _settlementQueue.allowedSlice(marketView, borrower, position);

lido-lend-markets/src/libs/CollateralActivationLib.sol
// sync(): `position.activatingStart` is read at 237 and 241, then _getUnlockTime re-derives the
// `$.positions[borrower]` pointer (a fresh keccak) and reads `activatingStart` twice more (372, 375).
237:         if (!isFrozen && position.activatingStart > 0) {
238:             uint256 unlockTime = _getUnlockTime($, borrower, opinion);
241:             bool maxQuorumDelayElapsed = block.timestamp >= position.activatingStart +
↪  $.config.maxGuardiansQuorumDelay;
370:         Position storage position = $.positions[borrower];
372:         if (position.activatingStart < opinion.suspectExpiryTimestamp) {
375:         return position.activatingStart + position.activatingDelay + surcharge;

// getUnlockTime(): reads activatingStart, then _getUnlockTime re-derives the pointer and reads it again
271:         if ($.positions[borrower].activatingStart == 0) return 0;
272:         return _getUnlockTime($, borrower, opinion);

// _drain(): both fields are written at 320-321, then re-read at 326 (and by the emit at 325, which
// copies the whole struct back out of storage). The post-write values are already computable from the
// locals `dust - fromDust` / `activating - fromActivating`.
320:         position.dust = uint128(dust - fromDust);
321:         position.activating = uint128(activating - fromActivating);
325:         emit CollateralDrained(borrower, position);
326:         return position.dust + position.activating;

lido-lend-markets/src/libs/MaxHeapLib.sol
// _siftUp(): both node keys are loaded at 144, then _swap re-loads BOTH full nodes at 168-169.
144:             if (self.nodes[idx].key <= self.nodes[parent].key) break;
```

```
145:             _swap(self, idx, parent);

// _siftDown(): nodes[largest].key is a repeat of nodes[idx].key (largest starts == idx) and, when
// largest was just set to `left`, line 159 re-reads nodes[left].key. _swap then re-loads both nodes.
158:             if (self.nodes[left].key > self.nodes[largest].key) largest = left;
159:             if (right < len && self.nodes[right].key > self.nodes[largest].key) largest = right;
161:             _swap(self, idx, largest);
168:         Node memory nodeA = self.nodes[indexA];
169:         Node memory nodeB = self.nodes[indexB];

// upsert(): nodes[slot].key is read at 106, written at 109, and re-read by the _siftUp/_siftDown call
106:         uint96 oldKey = self.nodes[slot].key;
109:         self.nodes[slot].key = packedKey;

lido-lend-markets/src/GuardianSignals.sol
// getOpinion(): _isBanned reads signal.flaggedBy.length() (line 301) and _signalCounters reads the
// same slot again (line 434). getOpinion is called on-chain from the controller's Supply/Withdraw/
// Borrow hooks, so this is a hot path.
289:         opinion.banned = _isBanned(_signals[marketId][account]);
290:         (opinion.strikeCount, opinion.suspectCount, opinion.suspectExpiryTimestamp) =
↪  _signalCounters(marketId, account);

lido-lend-markets/src/periphery/ExitBookBase.sol
// fillExitRequest(): `request.owner` read at 158 and again at 177, with external calls in between
158:         _redeem(id, request.owner, filledShares);
177:         if (lenderAssets != 0) asset.safeTransfer(request.owner, lenderAssets);

lido-lend-markets/src/periphery/Erc4626ExitBook.sol
// _availableShares(): `request.owner` read twice across an external call. Additionally `vaultOf[id]`
// is re-read once per source operation, i.e. up to 3x per fillExitRequest (_depositLiquidity at 109,
// _availableShares at 98, _redeem at 105).
99:         uint256 redeemable = vault.maxRedeem(request.owner);
100:         uint256 allowed = vault.allowance(request.owner, address(this));
```

Recommended Mitigation: Load each slot once into a local (or a function parameter) and re-use it. Two representative diffs; apply the same shape at every location above.

```
// LidoLendMarketController._handleBorrow
 function _handleBorrow(Id id, BorrowHookData memory hookData) internal {
-    if (isFrozen) revert MarketIsFrozen();
+    bool frozen = isFrozen;
+    if (frozen) revert MarketIsFrozen();
     ...
-    uint256 inactive = _activation.sync(isFrozen, hookData.onBehalf, opinion);
+    uint256 inactive = _activation.sync(frozen, hookData.onBehalf, opinion);
 }
```

```
// CollateralActivationLib._drain -- compute the post-drain values once, store them, re-use the locals
 uint256 fromDust = Math.min(dust, amount);
 uint256 fromActivating = Math.min(amount - fromDust, Math.saturatingSub(activating, lockAmount));

-position.dust = uint128(dust - fromDust);
-position.activating = uint128(activating - fromActivating);
-
-if (position.activating == 0) _clearTimer(position);
-
-emit CollateralDrained(borrower, position);
-return position.dust + position.activating;
+uint256 newDust = dust - fromDust;
+uint256 newActivating = activating - fromActivating;
+
```

```
+position.dust = uint128(newDust);
+position.activating = uint128(newActivating);
+
+if (newActivating == 0) _clearTimer(position);
+
+emit CollateralDrained(borrower, position);
+return newDust + newActivating;
```

For `CollateralActivationLib`, change `_getUnlockTime` to take the `Position storage position` pointer its callers already hold instead of re-deriving it from `$` and `borrower`; that removes both the duplicate keccak and the duplicate `activatingStart` loads in `sync` and `getUnlockTime`.

For `MaxHeapLib`, have `_siftUp` and `_siftDown` pass the `Node` values they already loaded into `_swap`, so `_swap` writes and re-indexes without re-loading either node.

For `SettlementQueueLib`, add an internal variant of `allowedSlice` that accepts an already-resolved root address and settlement LTV, so `LidoLendMarketController::_handleSettle, positionOf` can pass the values they just read instead of having the library recompute them.

Lido: Fixed in commit 5aa818a.

Cyfrin: Verified.

8.4.2 `CollateralActivationLib::_computeDelay` copies the whole `CollateralActivationConfig` from storage but uses only slot 0

Description: `_computeDelay` loads the entire `CollateralActivationConfig` struct into memory, but reads only three of its nine fields: `maxDelayInflow`, `maxActivationDelay` and `minActivationDelay`. Walking the declaration order in `lido-lend-markets/src/types/CollateralActivationTypes.sol:32-42`, the struct occupies three slots:

```
slot 0: minActivationDelay (uint40, bytes 0-4)
        maxActivationDelay (uint40, bytes 5-9)
        maxDelayInflow     (uint128, bytes 10-25)
slot 1: activatingLockAmount   (uint128, bytes 0-15)
        activatingLockDuration (uint40,  bytes 16-20)
slot 2: minActivationAmount     (uint128, bytes 0-15)
        maxGuardiansQuorumDelay (uint40,  bytes 16-20)
        delayPerStrike          (uint40,  bytes 21-25)
        delayPerSuspect         (uint40,  bytes 26-30)
```

All three fields `_computeDelay` actually reads live in slot 0. The struct copy therefore also loads slots 1 and 2 for nothing. On the `credit` path (the `SupplyCollateral` hook, i.e. every collateral deposit) slot 2 has already been warmed by the `$.config.minActivationAmount` read at line 172, but slot 1 is never otherwise touched, so the copy pays a full cold SLOAD (~2,100 gas) purely to discard the value.

```
lido-lend-markets/src/libs/CollateralActivationLib.sol
380:     function _computeDelay(State storage $, uint256 effectiveInflow) private view returns (uint256)
↪  {
381:         CollateralActivationConfig memory cfg = $.config;
382:
383:         if (effectiveInflow >= cfg.maxDelayInflow) return cfg.maxActivationDelay;
384:
385:         uint256 delaySpan = cfg.maxActivationDelay - cfg.minActivationDelay;
386:         return cfg.minActivationDelay + Math.mulDiv(delaySpan, effectiveInflow, cfg.maxDelayInflow);
387:     }

// callers
188:         uint256 delay = _computeDelay($, newInflow);            // credit() -- SupplyCollateral hook
297:         return _computeDelay($, Math.saturatingAdd(calcDecayedInflow($), inflowIncrement));
```

Recommended Mitigation: Read the three fields directly. They share one slot, so the compiler issues a single SLOAD for all three and slots 1 and 2 are never touched.

```
 function _computeDelay(State storage $, uint256 effectiveInflow) private view returns (uint256) {
-    CollateralActivationConfig memory cfg = $.config;
+    // These three fields share slot 0; reading them directly avoids loading the config's other two
↪  slots
+    uint256 maxDelayInflow = $.config.maxDelayInflow;
+    uint256 maxActivationDelay = $.config.maxActivationDelay;
+    uint256 minActivationDelay = $.config.minActivationDelay;

-    if (effectiveInflow >= cfg.maxDelayInflow) return cfg.maxActivationDelay;
+    if (effectiveInflow >= maxDelayInflow) return maxActivationDelay;

-    uint256 delaySpan = cfg.maxActivationDelay - cfg.minActivationDelay;
-    return cfg.minActivationDelay + Math.mulDiv(delaySpan, effectiveInflow, cfg.maxDelayInflow);
+    uint256 delaySpan = maxActivationDelay - minActivationDelay;
+    return minActivationDelay + Math.mulDiv(delaySpan, effectiveInflow, maxDelayInflow);
 }
```

Lido: Fixed in commit bff438f.

Cyfrin: Verified.

8.4.3 The storage `MarketParams` struct is re-encoded from storage on every core call instead of once per transaction

Description: `MarketParams` is five full storage slots (`loanToken`, `collateralToken`, `oracle`, `irm`, `lltv`). Passing a storage `MarketParams` as an argument to an external call makes the compiler ABI-encode it straight from storage, paying five SLOADs per call. Both exit paths below issue that call repeatedly inside one transaction while the params provably do not change, so the second and subsequent encodes are pure waste (~500 gas per repeat once warm, ~10,500 when cold).

```
lido-lend-markets/src/LidoLendERC4626.sol
// _settle is invoked once per loop iteration -- up to `borrowers.length` in settle() and up to
// maxIterations (default DEFAULT_MAX_SETTLEMENT_ITERATIONS = 150, line 38) in fallbackExit().
// Each iteration re-encodes the 5-slot `_marketParams` from storage.
331:         (collateralSeized, debtCanceled,/* borrowSharesCanceled */, cutSupplyShares) =
332:             LIDO_LEND.settle(_marketParams, supplyShares, borrowShares, address(this), borrower,
↪  receiver);

200:         for (uint256 i = 0; i < borrowers.length && remainingSupplyShares != 0; ++i) {
273:         for (uint256 i = 0; i < maxIterations && remainingSupplyShares != 0; ++i) {

lido-lend-markets/src/periphery/MorphoExitBook.sol
// _requestMarket() loads `_marketIdOf[requestId]` (1 SLOAD) plus the 5-slot `_marketParams[id]`
// (5 SLOADs). Within a single ExitBookBase::fillExitRequest it runs up to three times: once from
// _depositLiquidity, once from _availableShares (via _fillableShares), once from _redeem.
133:     function _requestMarket(uint256 requestId) internal view returns (MarketParams memory) {
134:         return _marketParams[_marketIdOf[requestId]];
135:     }
142:         MarketParams memory marketParams = _requestMarket(id);
158:         MORPHO.withdraw(_requestMarket(id), 0, shares, account, address(this));
163:         MORPHO.supply(_requestMarket(id), depositAssets, 0, filler, "");
167:         return _supplySharesToAssets(_requestMarket(id), shares);

// _availableShares additionally recomputes the market id with `marketParams.id()` -- a keccak256 over
// the 160-byte struct (MarketParamsLib.sol:16-20) -- even though `_marketIdOf[id]` already holds it
// and _requestMarket just read it.
148:         uint256 ownerShares = MORPHO.position(marketParams.id(), request.owner).supplyShares;
```

Recommended Mitigation: Hoist the params into memory once per external entrypoint and thread the memory value down.

```
// LidoLendERC4626 -- load once in settle()/fallbackExit(), pass down through _settleBorrower /
// _settleNextAllowedBorrower / _settle instead of reading `_marketParams` inside the loop.
 function _settle(
     uint256 budgetSupplyShares,
     address borrower,
+    MarketParams memory params,
     uint256 supplyShares,
     uint256 borrowShares,
     uint256 remainingBorrowShares,
     address receiver
 ) internal returns (uint256 cutSupplyShares, uint256 collateralSeized, uint256 debtCanceled) {
     ...
     (collateralSeized, debtCanceled,/* borrowSharesCanceled */, cutSupplyShares) =
-        LIDO_LEND.settle(_marketParams, supplyShares, borrowShares, address(this), borrower, receiver);
+        LIDO_LEND.settle(params, supplyShares, borrowShares, address(this), borrower, receiver);
 }
```

Apply the same shape in `MorphoExitBook`: resolve `Id marketId = _marketIdOf[id]` and `MarketParams memory params = _marketParams[marketId]` once at the top of the fill flow and pass both down, so `_availableShares`, `_redeem` and `_depositLiquidity` consume the memory copies. That also removes the `marketParams.id()` keccak at line 148 - use the market id already loaded from `_marketIdOf[id]`, which is by construction the same value, since `setMarketAllowed` keys `_marketParams` by `marketParams.id()` (lines 80 and 85).

Lido: Acknowledged.

8.4.4 Cache known storage values to prevent identical storage reads

Description: The same storage location is read more than once on a single execution path with no intervening write or state-changing call, so every read after the first repeats work already done. Some locations are instead written and then read back. Affected locations:

- `LidoLend::settle` reads `market[id].totalBorrowAssets` and `market[id].totalBorrowShares` in the `toAssetsUp` conversion, then re-reads both in the `toAssetsDown` conversion with no intervening write.
- `LidoLend::liquidate` writes `market[id].totalBorrowAssets` via `zeroFloorSub`, then reads it back twice in the bad-debt branch.
- `LidoLend::_accrueInterest` reads `market[id].fee` in the zero check, then re-reads it for the `feeAmount` computation.

Recommended Mitigation: Cache the storage value in a local variable once, then reuse the local for the subsequent reads on that path. Where the value is written first, compute it into a correctly-typed local, store the local once, and reuse that local instead of reading the slot back.

Lido: Acknowledged

8.4.5 Use `calldata` for read-only external parameters

Description: Several `LidoLend` external functions declare reference-type parameters in `memory` but only read them, requiring an avoidable calldata-to-memory copy. This applies to the `MarketParams` parameter of `setFee`, `createMarket`, `supply`, `withdraw`, `borrow`, `repay`, `supplyCollateral`, `withdrawCollateral`, `settle`, `liquidate`, and `accrueInterest`, and to the `Authorization` parameter of `setAuthorizationWithSig`.

The clean saving applies only to `supplyCollateral` (which does not forward `marketParams` to an internal helper) and `setAuthorizationWithSig`; the other functions forward `marketParams` to the internal `memory`-typed helpers `_accrueInterest`, `_createMarket`, or `_isHealthy`, which cannot take `calldata`, so declaring those external parameters `calldata` merely relocates the copy to the internal call.

Recommended Mitigation: Declare the external parameters that are not forwarded to a `memory`-typed internal helper as `calldata` (`supplyCollateral`'s `marketParams` and `setAuthorizationWithSig`'s `authorization`). For the remaining functions, converting the parameter is only beneficial if the internal helper signatures are also revisited.

Lido: Acknowledged

8.4.6 Pack reorder-safe state variables into fewer storage slots

Description: `LidoLend`'s storage layout places `bool isInitialized` after nine `mapping` declarations, so it occupies a full slot by itself. Since `isInitialized` (1 byte) can pack with an `address` (20 bytes), reordering it adjacent to `owner` or `feeRecipient` reduces the contract's storage footprint by one slot. This is only safe before first deployment: `LidoLend` runs behind an upgradeable `OssifiableProxy`, so once the proxy is deployed and initialized the storage layout is frozen, and any later reordering would corrupt existing state.

Recommended Mitigation: Reorder `isInitialized` to be adjacent to `owner` (or `feeRecipient`) before the first deployment, and treat the resulting layout as the canonical layout that all future upgrades must preserve.

Lido: Acknowledged